

MEDHAVA

Medhava

One Business Operating System. Any trade. One shared data core.

22 modules · 113 apps · 19 technical layers · compiled 2026-08-24

What this document is

Two documents in one, because they answer two different questions and people ask both.

Part One — The System is the reader's tour: what Medhava is, how one order moves through it, every module and every app, how a trade is added as a row of configuration, and the rules that hold everywhere.

Part Two — The Plan of Action is the builder's document: what Medhava is, the tenancy model, the industry pack engine, the eight build phases, the free-first tool register, onboarding, security, risks and what a customer pays for.

Part Three — How It Is Built is the technical design: the architecture, the database, the backend, the frontend, storage, memory, sign-in, integrations and how it is run — every layer with what it is built on and what can replace it.

Both are generated from `brand/site/modules.js`, the one canonical list. Neither this page nor either part contains a module count, an app name or an app order typed by hand — which is why they cannot disagree with each other or with the software.

What this document is

This describes a design. Everything in it is what the system is being built to be. Nothing in it claims to already exist, and no part of it is presented as finished.

Two rules run through every page. **No capability depends on one tool** — 19 technical layers, 57 named alternatives between them, each behind an interface so a supplier can be changed without a rebuild. **Nothing is static and the past stays correct** — 18 things a business changes itself, instantly, each carrying the date it starts from so closed months never move.

The claim, and where it is checked

"Any industry" is a statement about the code, so it is checked in the code.

A trade is a row, not a fork. What a business calls things, the stages its work moves through, the extra fields its records need, the documents it issues and the reference data it starts with all arrive as one configuration file — and a pack may never contain executable code, invent a concept the engine does not have, extend a table that does not exist, declare money as anything but integer paise, switch off an immutable rule, or be applied in part.

	How many	The design
Industry packs shipped	6	a directory, no ceiling
Companies	as many as you have	a table; the shipped plan caps a subscription at 20, the software has none
Channels per company	as many as you sell on	a table, read from your data
Stock	one number per SKU	one number per SKU — never per channel
Group	sum minus inter-company trade	sum minus inter-company trade

`core/tests/packs.test.js` invents a **commercial laundry** during the test run — a trade that appears nowhere in this software — loads it from a JSON string, and requires every screen to answer in that trade's words while still refusing it the audit trail. A final assertion fails the build if the engine file ever contains a single trade word.

`core/tests/core.test.js` does the matching thing for scale: ten companies with ten channels each, then eleven by eleven with no code changed.

The honesty rules this document is written under

1. Nothing is described as finished. This is a design, and it says so on its first page.
2. No count is typed from memory. Every figure is read from the canonical list when this is written.
3. No figure is invented. Where a rate or a price is missing, the tool posts zero and names the item rather than guessing — a guessed rate is a wrong payment to a real person.
4. Where something could not be verified, it says so instead of implying it was.

PART ONE — THE SYSTEM

One Business Operating System for any trade: 22 modules and 113 apps over one shared data core.

This file is the whole system in plain text — every module, every app, and what each one reads and writes. It is generated from `brand/site/modules.js`, the same file the website and every PDF read, so nothing here can disagree with them. The counts below are not typed in; they are counted from that file each time this page is built.

Modules	22, in dependency order — a module comes only after everything it draws on
Apps	113
Companies	As many as you have. A company is a row, not a setting — the shipped plan caps a subscription at 20 and the software itself has no ceiling
Shared data core	Company · Item/SKU · Party · Stock · Ledger/Voucher · Order
Key difference	Not a suite of integrated apps. One application over one database, so there is no sync step and no second copy of any master record
Compliance	Double-entry books with CGST/SGST/IGST, TDS, TCS, input credit on accepted goods, GSTR-1 and GSTR-3B, filed per registration
Channels	Any storefront, marketplace, counter, wholesale desk or export buyer — read from your own data, never from a list inside the code
Security	Row-level isolation per company; outside services connect with scoped, revocable keys — never account passwords
Deployment	Hosted, or single-file apps that run by double-clicking with no install and no internet

The one idea

Every module reads and writes the **same six records**. That is the physical reason a single goods receipt can touch stock, the books, quality and sourcing in the same instant.

UNIFIED DATA CORE
Company · Item/SKU · Party ·
Stock · Ledger/Voucher · Order

▲ ▼

every one of the 113 apps reads and writes these, and only these

One stock number, not one per channel. The last unit sold at the counter disappears from every marketplace you sell on in the same instant — not three hours later as a cancellation, because cancellations are what a seller rating is lost to.

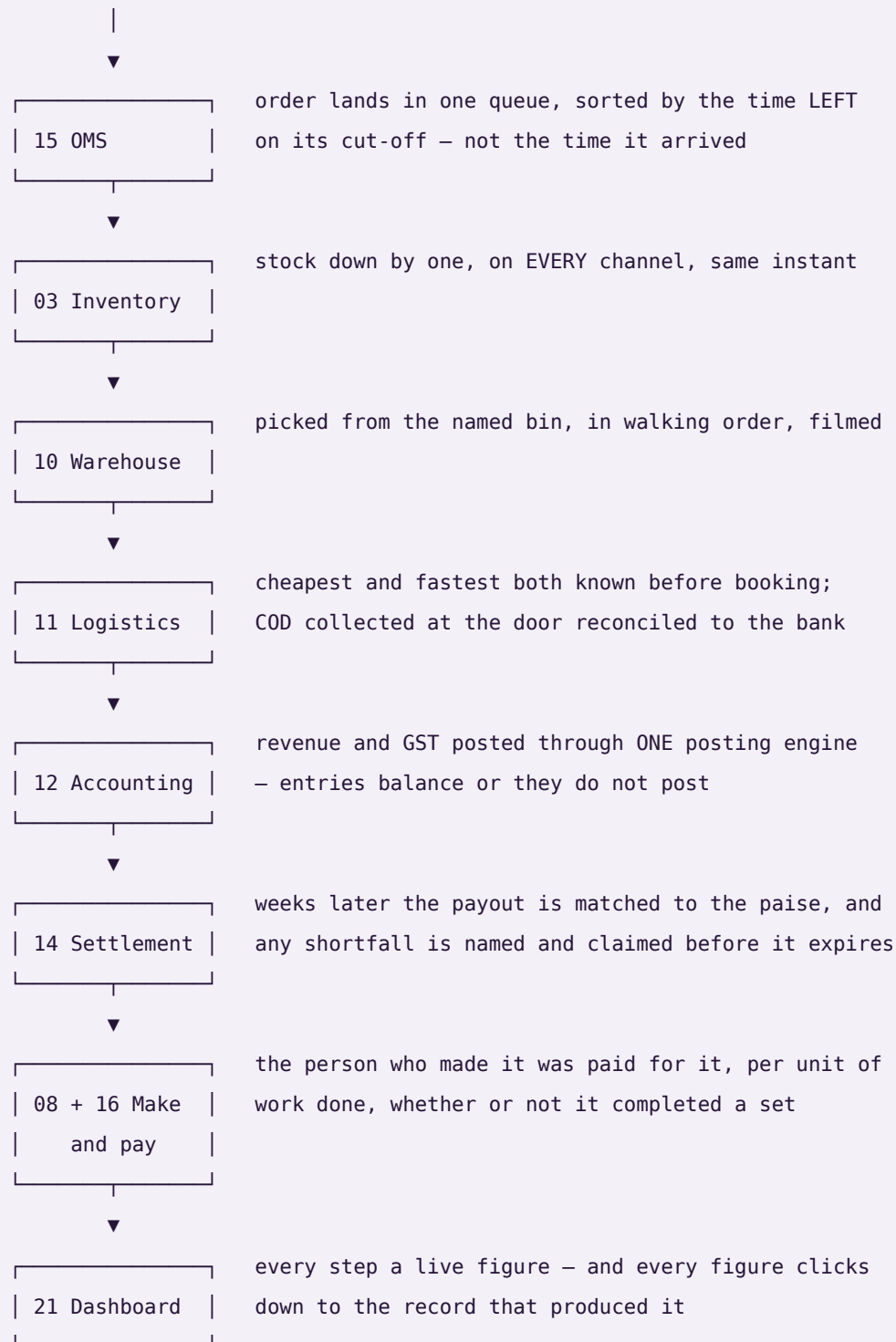
Accepted — not ordered — is what counts. You order 100 units. 100 arrive. Quality accepts 96. Most systems increase stock by 100 and claim tax credit on 100. This one increases stock by **96**, claims input credit on **96**, raises a debit note for the 4 rejected, and lowers that supplier's accept rate — automatically.

Nothing derived is ever stored. Outstanding, ageing, risk, promise dates, cost per piece and profit per design are recomputed on read. A stored total is a number that can drift away from the documents underneath it; a computed one cannot.

How one order moves through it

The test of whether this is one system or 113 programs sharing a login: take a single order and follow it. The words below are a product business's; a clinic, a law practice and a freight desk run the same eight modules with their own words on them.

sold on a marketplace



one transaction · eight modules · one database

Every industry, as a row of configuration

This is the part that makes "any trade" a fact rather than a claim. A trade is **not** a fork of the software, a branch, or a bespoke build. It is a file: what this trade calls things, the stages its work moves through, the extra fields its records need, the documents it issues, which discretionary rules apply, and the reference data it starts with.

Rank	Pack	Sector	Its words for customer · order · worker	Pipelines	Documents
1	manufacturing	Manufacturing	buyer · sales order · operator	2	4
2	wholesale-distribution	Distribution	dealer · sales order · salesman	2	5
3	retail-ecommerce	Retail	shopper · order · associate	2	4
4	professional-services	Services	client · matter · fee-earner	1	4
5	healthcare-clinic	Healthcare	patient · appointment · clinician	2	4
6	logistics-3pl	Logistics	shipper · consignment · driver	2	5

The order is not taste. Manufacturing is the largest ERP user base — around a fifth of all users and roughly a third of market revenue. Professional and financial services is next at 13.86%, and has no stock at all, which makes it the hardest case for the claim. Distribution is 9.90%. Retail and e-commerce is the largest warehouse-management segment at about 28%. Healthcare is under 5% of ERP users today and the fastest-growing of them all at 22.37% a year. Transportation is the most-served market among third-party logistics providers at 90%, and **order management ranks first** among the technology services those providers offer.

What a pack may never do. A configuration file that can do anything is not configuration, it is a hole. A pack may not contain executable code at any depth, invent a concept the engine does not have, add a field to a table that does not exist, declare money as anything but integer paise, switch off an immutable rule — company scoping, the audit trail, the posting rules, group elimination, roster privacy — or be applied in part. Each of those refusals is a named test.

One default worth stating on its own: a rule a pack never mentions is **on**. The rulebook is the default and a pack is an exception list, never a permission list. The other way round, every rule added after a pack was written would silently apply to nobody using it.

The test that decides whether this is a product. `core/tests/packs.test.js` invents a **commercial laundry** — a trade that appears nowhere in this software, in no pack, in no module and in no rule — hands the engine a JSON string while the tests are running, and requires the whole system to answer in that trade's words: an order reads as a docket, a work order as a wash load, a customer as an account. Its pipeline resolves ordered and terminating, its fields land on real tables, its rule switches resolve against the real rulebook, and it is refused the audit trail exactly as the shipped packs are. A final assertion fails the build if the engine file ever contains a single trade word — because an engine that knows one trade's words has an opinion about which trades are normal.

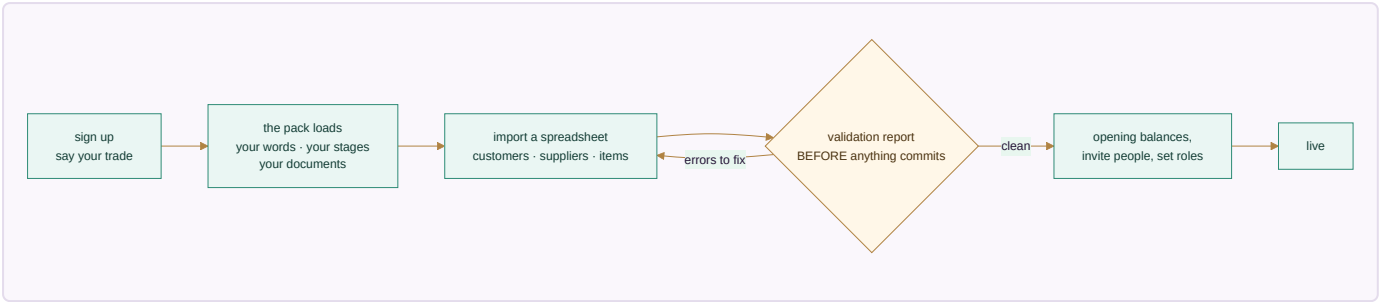
And the product screens are drawn from 12 sectors, not one: Drone & precision manufacturer, Freight forwarder, HVAC service firm, Law practice, Restaurant group, Interior contractor, Homeware brand · D2C, Precision components maker, Multi-doctor clinic, Training institute, Creative agency, Dairy co-operative. The same module is shown with each of their figures, one under the other, so the argument is made where it can be checked rather than believed.

How you actually use it — a walkthrough

The sections above say what Medhava is. This one follows a person through it, because "22 modules over one shared data core" is a true sentence that tells you nothing about your Tuesday.

Every screen below is a real render of the software, not an artist's impression — the same markup and the same stylesheet the product uses. The figures on them are illustrative.

Day one — from signing up to working, without a consultant



Nothing is blank when you arrive. Pick manufacturing and the system says sales order; pick professional services and the same screen says matter; pick the clinic pack and it says appointment. Same columns underneath, every time.

A day in the life — one order, followed all the way

An order arrives · Module 15

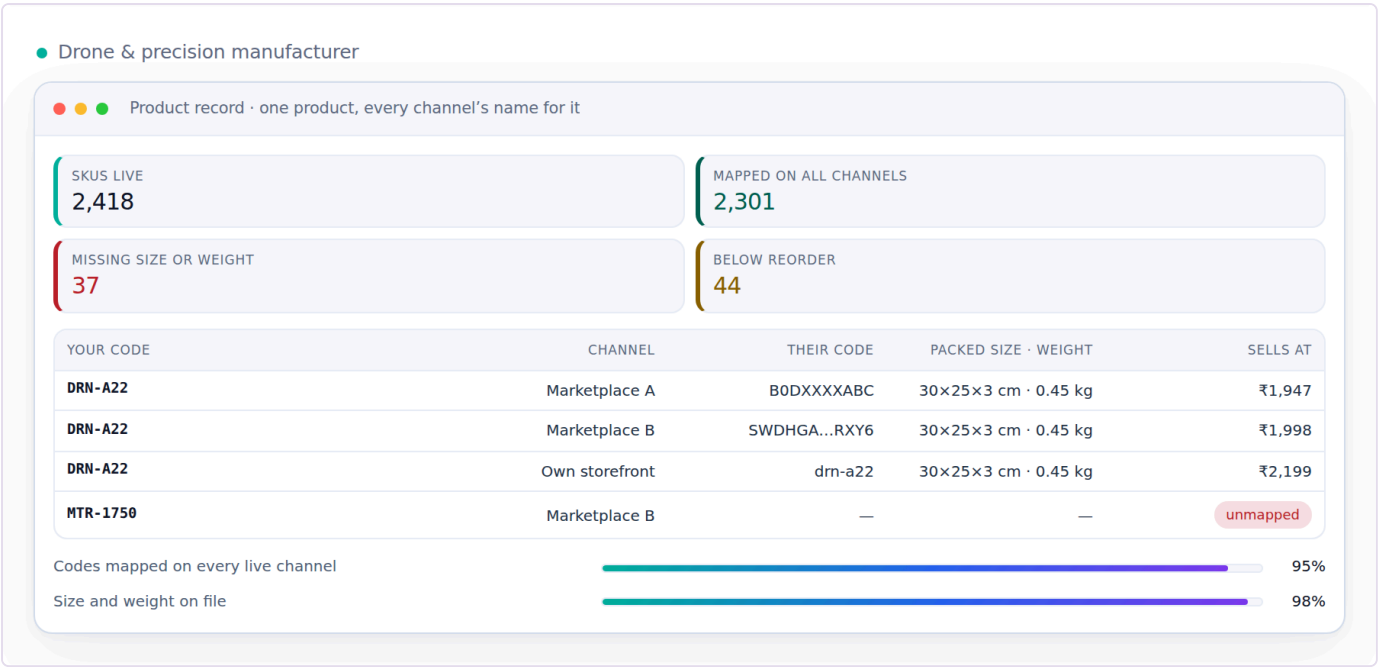
It lands in one queue with every other channel's orders, sorted by the time **left** on its cut-off rather than the time it arrived. The order that must leave in forty minutes is above the one that came in first and has all day.



Order queue · 9 channels · dispatch cut-off running — illustrative figures

Stock moves — everywhere at once · Module 03

One number per SKU. The unit that just sold disappears from every other channel in the same instant, which is the only way to stop the cancellation that costs you a seller rating.



Drone & precision manufacturer · Product record · one product, every channel's name for it — illustrative figures

It gets picked and packed · Module 10

A pick list in walking order, confirmed against the bin it came from. A short pick stops the pack rather than quietly reducing the order — because an order silently shipped short is a claim you will pay for later.



Drone & precision manufacturer · Pick wave 22 · Zone A → C — illustrative figures

It ships, and the money is chased · Module 11

The courier rate is checked against the packed weight before booking, and cash collected at the door stays a receivable until it is actually remitted to your bank.

Drone & precision manufacturer

Handover · what went out against what they took

EXPECTED OUT
1,046

HANDED OVER
1,012

LEFT BEHIND
34

COD NOT REMITTED
₹3.8 L

COURIER · SERVICE	EXPECTED	HANDED OVER	LEFT	CODE
Courier A · large	126	126	0	confirmed
Courier A · standard	604	598	6	confirmed
Courier B · standard	246	240	6	confirmed
Courier C · remote	70	48	22	awaiting

Manifests signed the same day



Parcels traced after handover



Drone & precision manufacturer · Handover · what went out against what they took — illustrative figures

The books post themselves · Module 12

Revenue and tax go through one posting engine. Entries balance or they do not post — there is no third option, and no month-end scramble to find out which.

Drone & precision manufacturer

Trial balance · it always ties

REVENUE YTD
₹11.8 Cr

GROSS MARGIN
38.2%

GST PAYABLE
₹4.1 L

ITC AVAILABLE
₹3.6 L

HEAD	DEBIT	CREDIT	THIS MONTH
Sales	—	₹1,42,08,400	+12%
Purchases	₹86,20,110	—	+9%
Direct expenses	₹12,44,900	—	-3%
Difference	₹0	₹0	Balanced

GSTR-1 lines matched



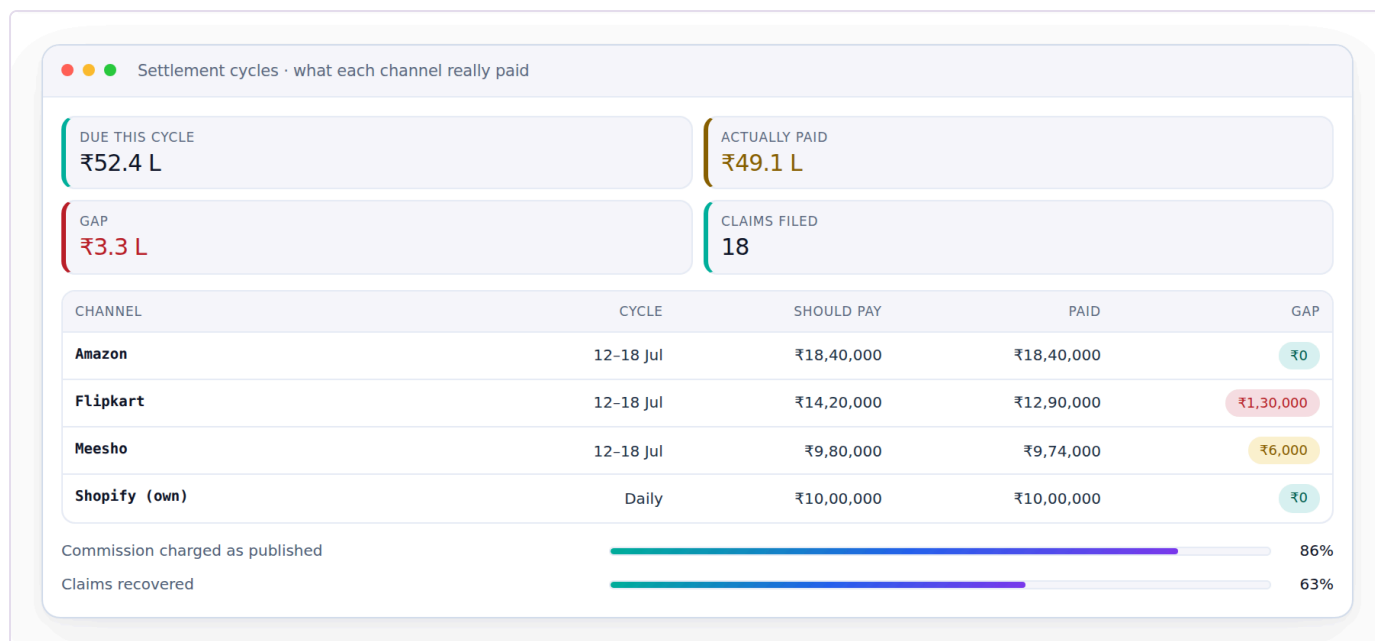
Vouchers posted automatically



Drone & precision manufacturer · Trial balance · it always ties — illustrative figures

Weeks later, the payout is checked · Module 14

What the channel said it would pay, against what arrived, line by line. A shortfall is named and claimed before the window to claim it closes.



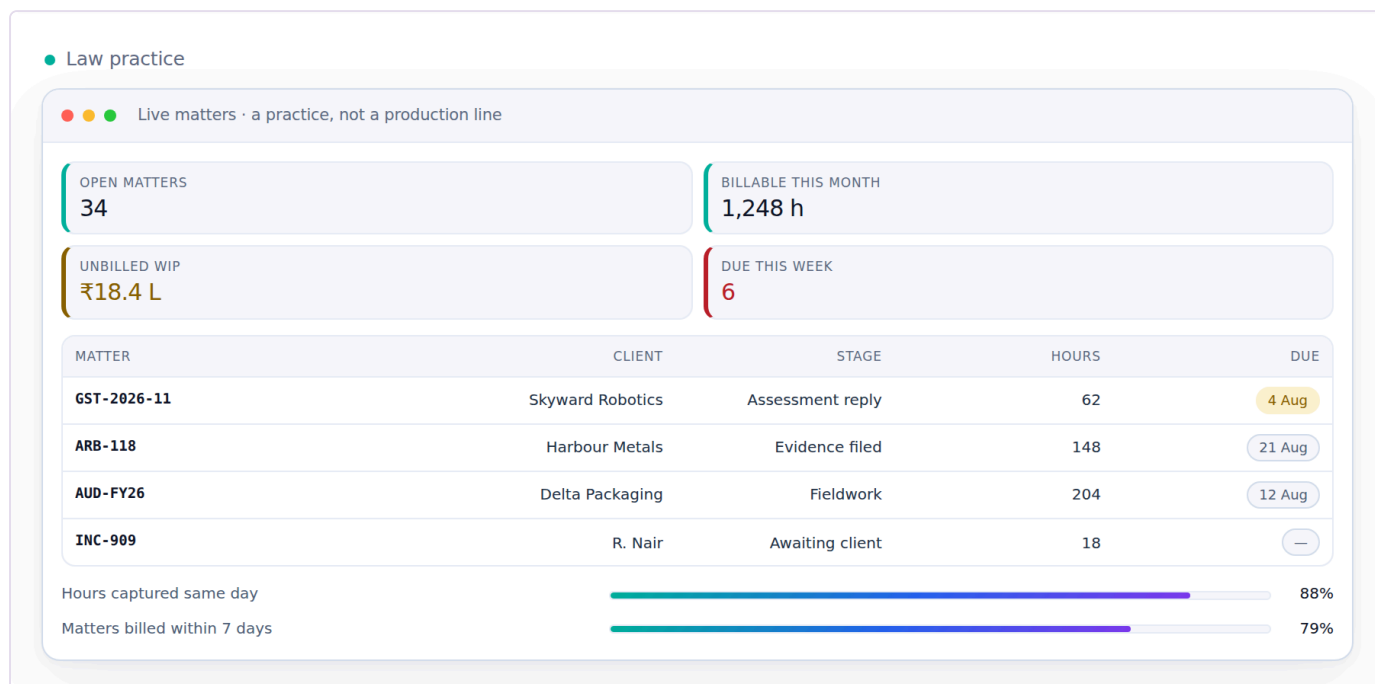
Settlement cycles · what each channel really paid — illustrative figures

The same day, in three trades that have nothing in common

This is the whole argument, and it is easier to see than to read:

A law practice runs matters · Module 20

Same record, same columns, same ledger underneath. A matter instead of an order, a fee-earner instead of an operator, hours instead of units.



Law practice · Live matters · a practice, not a production line — illustrative figures

A clinic runs appointments · Module 19

A patient instead of a customer, an appointment instead of an order, a clinician instead of a salesman.

Multi-doctor clinic

Local search · what patients actually search

TRACKED QUERIES
128

IN THE MAP PACK
41

CITED BY AN ASSISTANT
12

PAGES WITH SCHEMA
96%

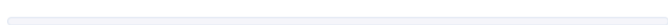
QUERY	POSITION	ASSISTANT CITES US	PAGE
dentist near Koregaon Park	2	Yes	/dental
root canal cost pune	7	No	/treatments/rct
paediatrician sunday open	4	Yes	/paediatrics
orthodontist consultation	11	No	/ortho

Pages whose markup matches what is on them



100%

Claims marked up that the page does not make



0%

Multi-doctor clinic · Local search · what patients actually search — illustrative figures

A restaurant group watches its cash · Module 13

Four sites, one cash position, fourteen days ahead. No stock module was removed and no code was forked to make any of these three work.

Restaurant group

Cash position · four sites · next 14 days

CASH + BANK
₹18.6 L

DUE OUT
₹22.4 L

EXPECTED IN
₹26.1 L

TIGHTEST DAY
Day 9

WHEN	OUT	IN	RUNNING
This week	₹9.8 L	₹11.2 L	₹20.0 L
Rent + salaries, day 9	₹8.4 L	₹2.1 L	₹13.7 L
Supplier terms, day 11	₹4.2 L	₹6.0 L	₹15.5 L
Aggregator payout, day 13	—	₹6.8 L	₹22.3 L

Forecast lines with a named assumption



100%

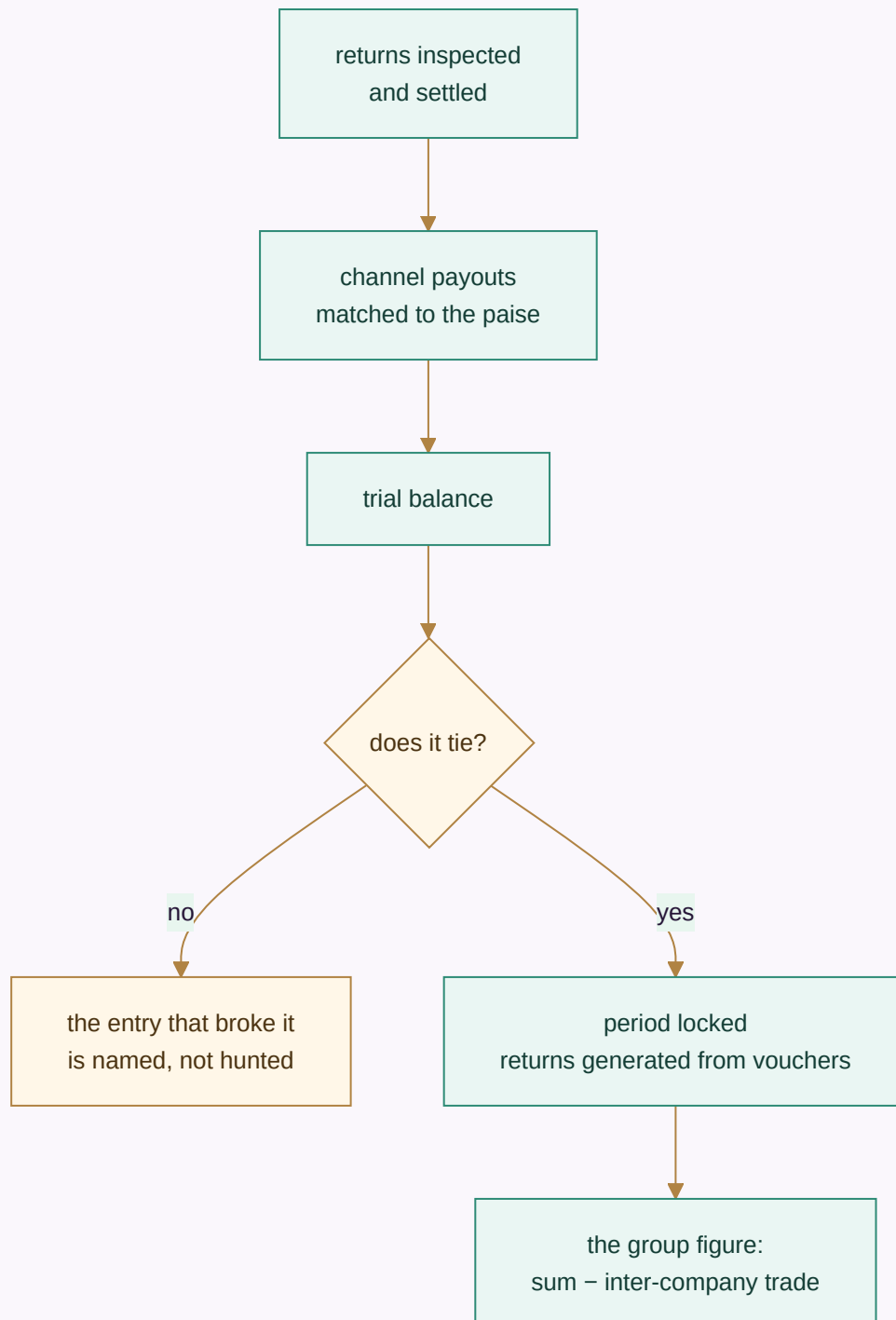
Projections posted to the ledger



0%

Restaurant group · Cash position · four sites · next 14 days — illustrative figures

Month end



Then the next month opens, and nothing about the close depended on anybody remembering to run it.

Every module and every app

Listed in build order. is described as finished that is not.

Module 01 · Platform

The spine every module runs on.

Not a module you open — the layer underneath all 22. Who can see what, how the system is configured, and a record of everything that ever happened. Foundation first: nothing downstream can be built before identity, roles and the audit trail exist.



Roles and permissions · who may do what — illustrative figures

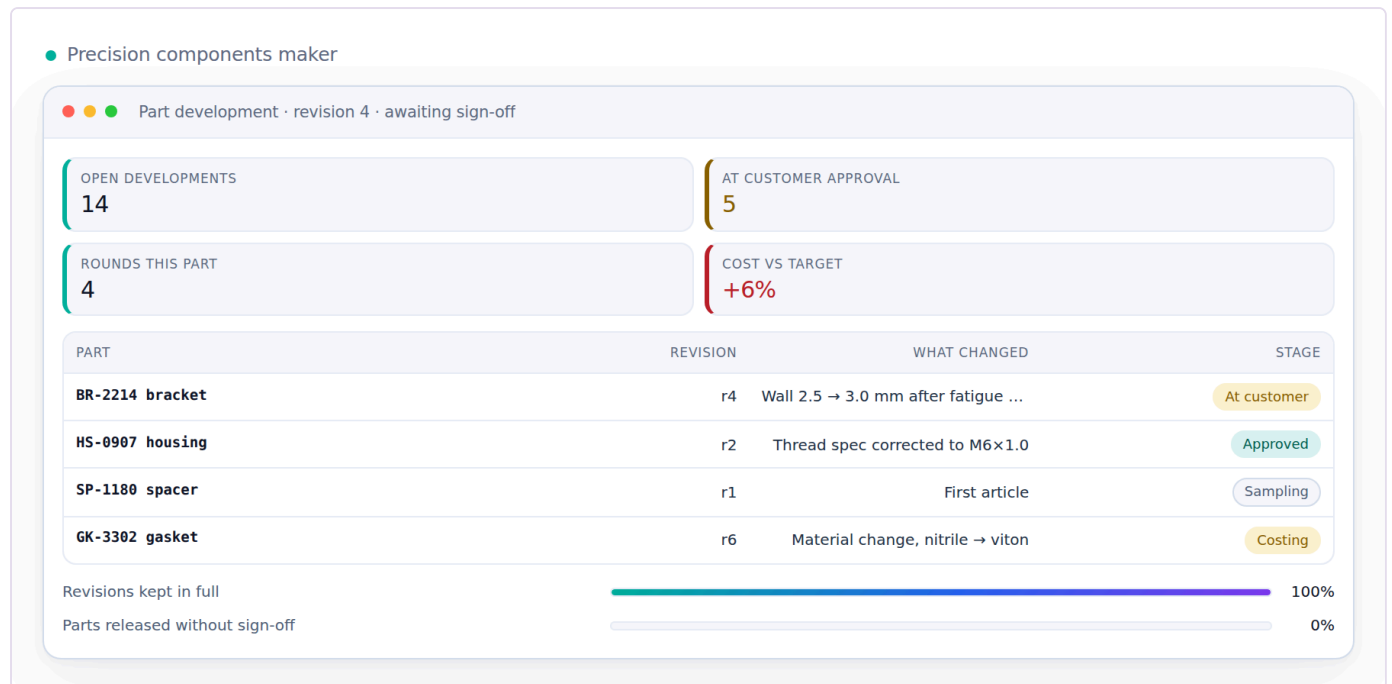
Reads from: Every module **Writes to:** Every module

App	What it does
Identity, Settings & Audit	Users, roles and permissions, company switching, tax and numbering setup — and an immutable record of everything that ever happened.
Industry Packs	What your trade calls things, the stages your work moves through, the extra fields your records need, the documents you issue and the reference data you start with — all of it loaded as a row of configuration, never as a separate version of the software. Pick the pack that matches your trade and the whole system changes its words: the same order screen reads as a job, a matter, a consignment or an appointment, with the same columns underneath. A pack may rename what exists, add fields to tables that exist, and switch discretionary rules off; it may never invent a concept, add a field to a table that does not exist, contain any executable code, or switch off the guarantees — company scoping, the audit trail, money never being a float — because a trade may change its vocabulary and may not opt out of the things the books are trusted for. Adding a trade nobody anticipated is therefore a file somebody writes, not a release somebody ships.
Ask & Print	Ask from your phone, anywhere: a ledger, a bill, today's packing slips. It comes back as a PDF — or prints on the office printer, with nothing plugged into your phone.
Communications	WhatsApp commands and broadcasts, email and SMS, and the handful of scheduled jobs that carry a nudge to the right person without anyone remembering to send it. Every capability here has more than one interchangeable provider — none of them is ever the source of a figure the business reports.
WhatsApp Command Console	The shop floor does not open a laptop. A worker or a staff member sends a short message — in, out, today's pieces, leave, an advance — and it becomes a real record: attendance with the time and place it was marked, a production report against the design, a request in the approvals queue. The end-of-day prompt asks what is still open and how long it needs, so tomorrow starts from what is actually pending rather than from memory. Marking attendance checks the phone is at the unit within the radius set for it, with a grace window; a person outside it is not refused, they are flagged for the manager, because a system that locks someone out of being paid for being early at the wrong gate has failed at its job. Every override is recorded with who made it. The words a worker types are in the language they speak, and nothing here ever asks for a password, a bank detail or a document number.
Data Privacy & Consent	What a person's data may be used for, captured as consent at the point it is given and honoured everywhere downstream — including the right to have it corrected or removed. Retention (how long a record is kept) and deletion (a person's right to have their own data removed) are two different policies, tracked separately, because a rule that keeps records for the law and a request from a person to be forgotten do not resolve the same way.
Provider Router & Cost Guard	The rule that no capability depends on one outside service, enforced at the moment it matters instead of merely promised. Every capability has an ordered fallback list ending on an option that needs nothing connected, so a courier API that stops answering at 9pm, or an AI key that hits its quota mid-catalogue, drops to the next option instead of stopping the work. A provider that keeps failing is tripped out of the list entirely and retried once after a cooldown rather than hammered; retries inside one provider wait twice as long each time. And every paid call is counted in paise against a ceiling you set — over the ceiling the paid provider is refused, not warned about, and the work completes on a free one. Because every capability is guaranteed a built-in or by-hand option, a spent budget can stop the spending without ever stopping the business.
Payment Data Scope	A written statement of exactly which systems ever see a card or bank credential, and which never do — because every card-capable screen in this system hands that moment to a payment provider's own secured field and never stores or even passes the number through application code. The statement is what an auditor or a partner asks for before they will connect to this system.

Module 02 · Design & Sampling

A style exists on paper before it exists as stock.

A product moves from a first idea to something the business can actually make and sell — specification, sample rounds, costed trials and sign-off — before it is ever entered as a catalog record. Building this first means Inventory & Catalog never has to invent a style it has no real specification for.



Precision components maker · Part development · revision 4 · awaiting sign-off — illustrative figures

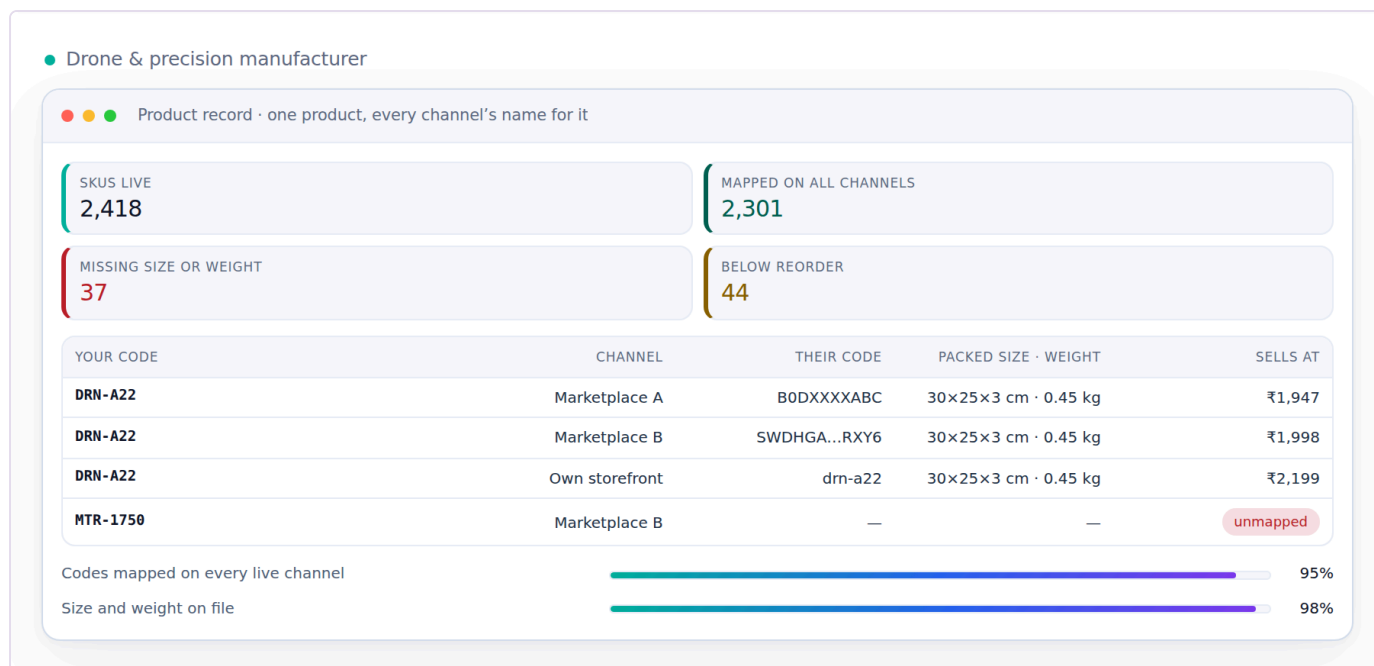
Reads from: CRM **Writes to:** Inventory & Catalog · Manufacturing

App	What it does
PLM & Development	First idea to something you can actually make: specification, sample rounds, costed trials and sign-off, with every version kept. A product, a machined part, a formulation or a service package all move through stages you set yourself.
Design / IP Register	What protects a design once it exists — trademark or copyright status, the date it was first shown, and a flag the moment a near-identical listing turns up elsewhere. Every version already kept by PLM & Development gets a filed status here instead of just a date stamp, because a design with no ownership record on file is a design nobody can defend.

Module 03 · Inventory & Catalog

One number everyone trusts.

The most important number in the system: one quantity per SKU, per location, per stage — read and written by every other module. And one product record that every channel lists from.



Drone & precision manufacturer · Product record · one product, every channel's name for it — illustrative figures

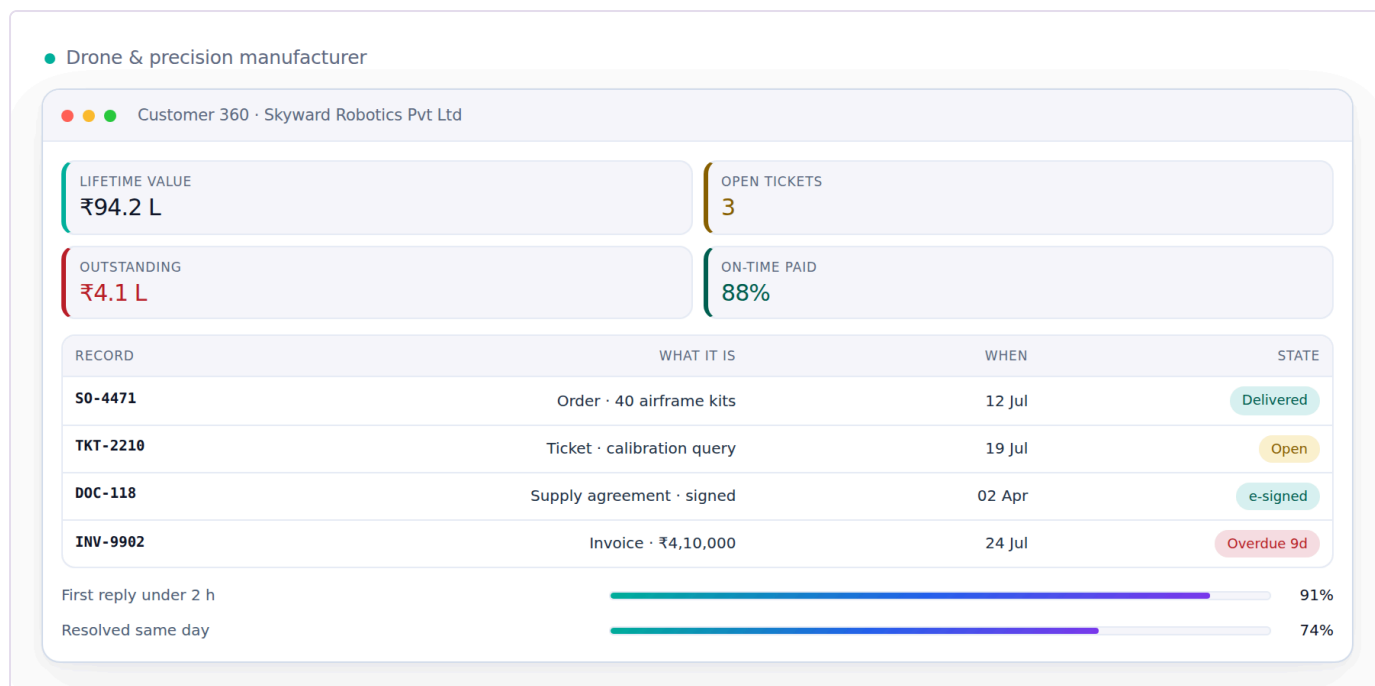
Reads from: Design & Sampling · Every module **Writes to:** Every module

App	What it does
Stock	Live quantity by SKU, location and stage, with reorder alerts, batches, kits and dead-stock. Goods you still own but that sit in a channel's own warehouse are a location like any other, so consignment and sale-or-return stock is counted, valued and aged with everything else instead of disappearing off the books until it sells.
Catalog / PIM	One product record — attributes, images, HSN, MRP and the price each channel actually sells at — pushed to every marketplace and to your own storefront, and scored for each channel's rules before it lists. It also holds the two things everything downstream depends on: the code each channel knows this product by, mapped to yours, and the packed size and weight that decide the courier rate and settle every weight dispute. Every listing's state on every channel is here too — live, waiting for your approval, blocked, archived — with the quality score that decides whether anyone sees it.
Kit & Combo SKU	A sellable SKU made of component SKUs — a three-piece set sold as one listing. Selling the kit decrements each component at order time, so stock is right for every piece in the set, not just the set itself.
Master-Data Hygiene	Duplicate detection and merge across customers, vendors and designs, and a dead-stock register for what has not moved in months. Protects every downstream report from the same record existing twice under two names.

Module 04 · CRM

Know every customer completely — and answer them fast.

One record per customer carrying every lead, order, return, document and conversation, whichever channel it came from. Whoever picks up the next question can already see everything that came before it.



Drone & precision manufacturer · Customer 360 · Skyward Robotics Pvt Ltd — illustrative figures

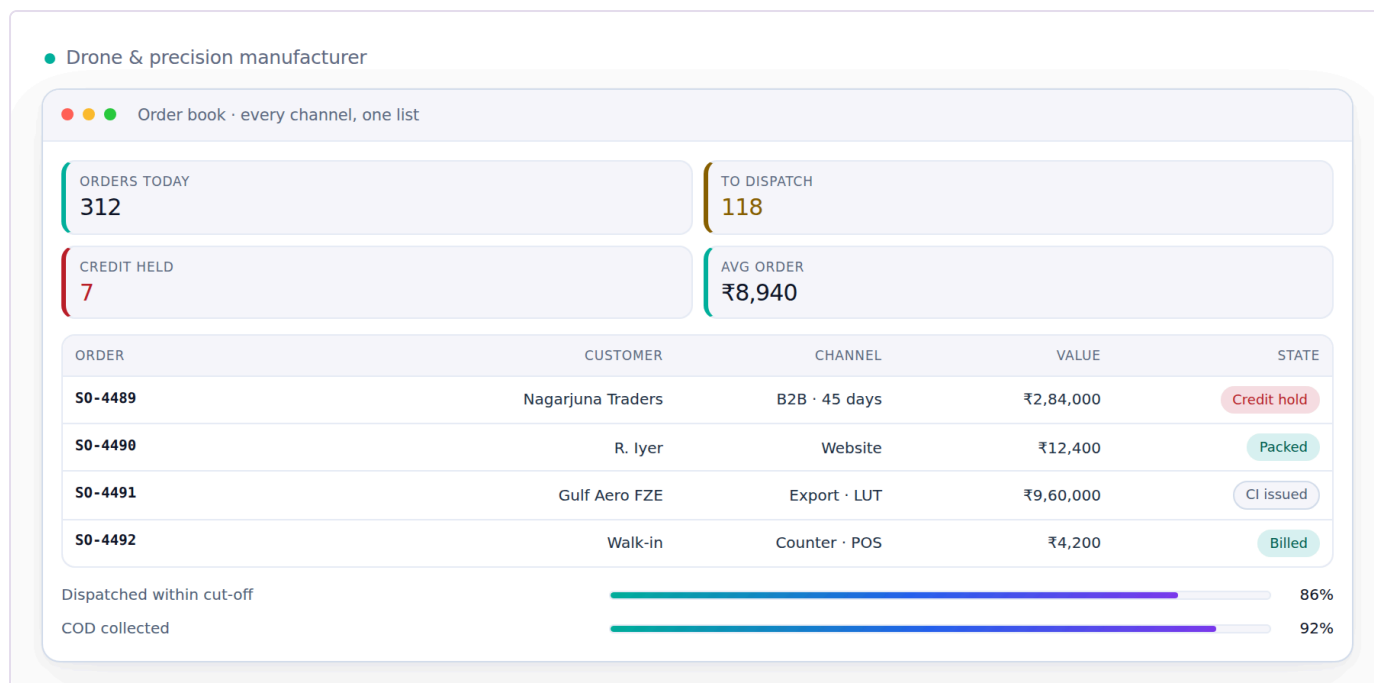
Reads from: Every module **Writes to:** Sales · E-commerce / OMS · Marketing

App	What it does
CRM & Customer 360	Lead to won, then the full lifetime: orders, returns, value and what to offer next.
Documents & eSign	Every agreement, receipt, certificate and scan filed against the record it belongs to — an order, a party, a case, an employee — so it is found by that record instead of by remembering a folder. Send one out for signature and the signed copy files itself back.
Helpdesk & Live Chat	Questions arriving by chat, email or phone become tickets tied to the order or the account they are about, with the whole history already on the screen.
Forms & Feedback (NPS)	A short form after delivery, and the score it produces attached to the design or item it is actually about — not just the buyer — so a complaint-prone item surfaces as a pattern instead of a scatter of individual gripes.

Module 05 · Sales

Every way you sell, one order book — to the doorstep.

Retail counter, wholesale, export and your own website all write to the same order and draw on the same stock number. The courier side lives here too, so a sale is not finished when it is billed — it is finished when it is delivered and the COD money is in.



Drone & precision manufacturer · Order book · every channel, one list — illustrative figures

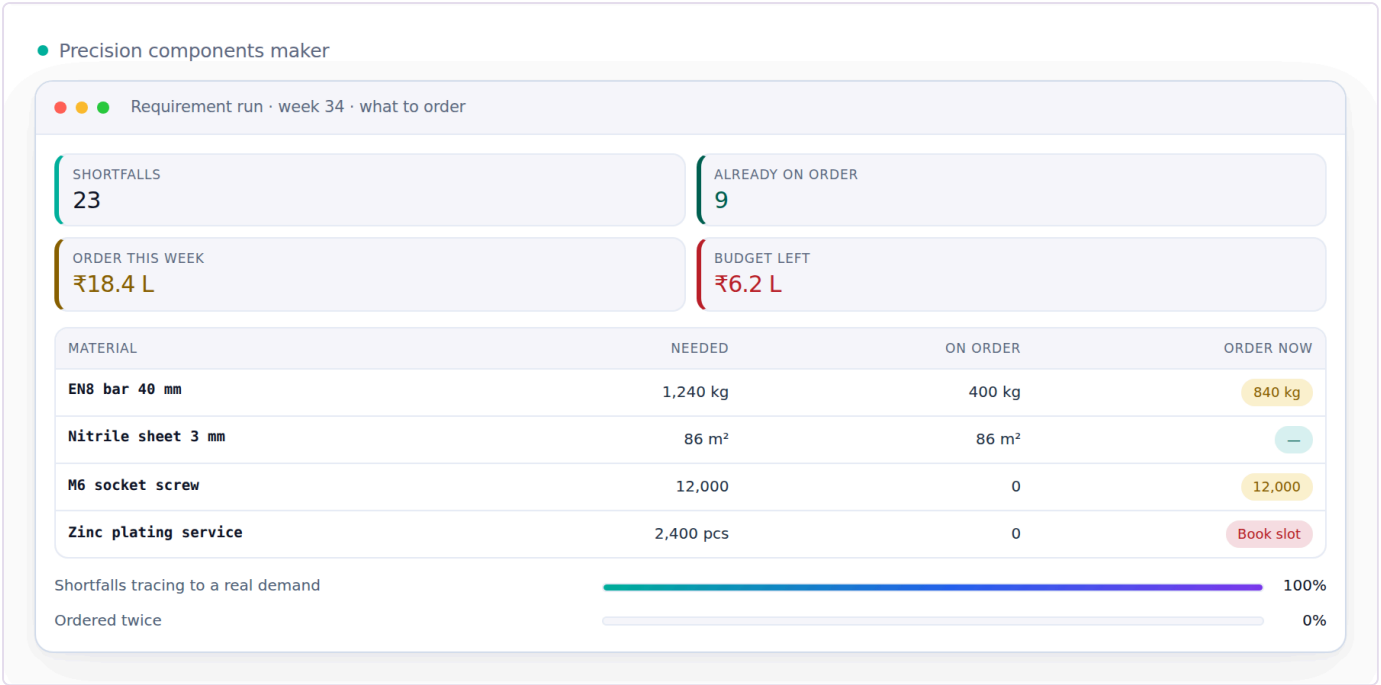
Reads from: Inventory & Catalog · CRM · Warehouse · Logistics **Writes to:** Inventory & Catalog · Accounting & GST · Warehouse · Logistics

App	What it does
D2C Sales	Orders from your own storefront — Shopify, WooCommerce or a custom site — cart to dispatch, with loyalty and partial COD.
B2B & Credit	Wholesale orders with credit limits, tier pricing and outstanding ageing.
Export	Commercial invoice, packing list, LUT bond and IGST-refund tracking.
POS	Counter billing that draws on the same stock as your website.
Quotes & Proforma	Send a quote, convert it to a confirmed order in one click.
Couriers & AWB	Book the shipment on the order itself, compare couriers, print the label and follow the AWB to the door.
Subscriptions	A schedule that raises its own invoice on its cycle and follows up on its own when a payment fails — for anything sold as a standing order rather than a one-off.
Customisation & Made-to-Measure	The order that does not exist in the catalogue: a buyer sends reference pictures and their own measurements, a price is agreed over several messages, and the piece is made for them. All of it is one record — the references, the measurement set, every quote in the negotiation and what was finally agreed, the advance taken to start work and the balance taken before dispatch. When the order is accepted it opens a production order like any other, so a bespoke piece is costed, made, checked and posted exactly as a catalogue piece is. Two legs of money on one order is the part most systems get wrong: the advance is earned when the work starts, the balance is owed until the piece ships, and the ledger shows both separately rather than one payment appearing when the whole thing is over.

Module 06 · Planning & Requirements (MRP)

Turn what is selling into what to buy and make.

Confirmed orders and demand history have to become a plan before Purchase can buy anything or Manufacturing can start anything — otherwise buying and making are both just guessing. This module sits between the two: it reads what is actually selling and what is already committed, and turns that into requirement, not the other way around.



Precision components maker · Requirement run · week 34 · what to order — illustrative figures

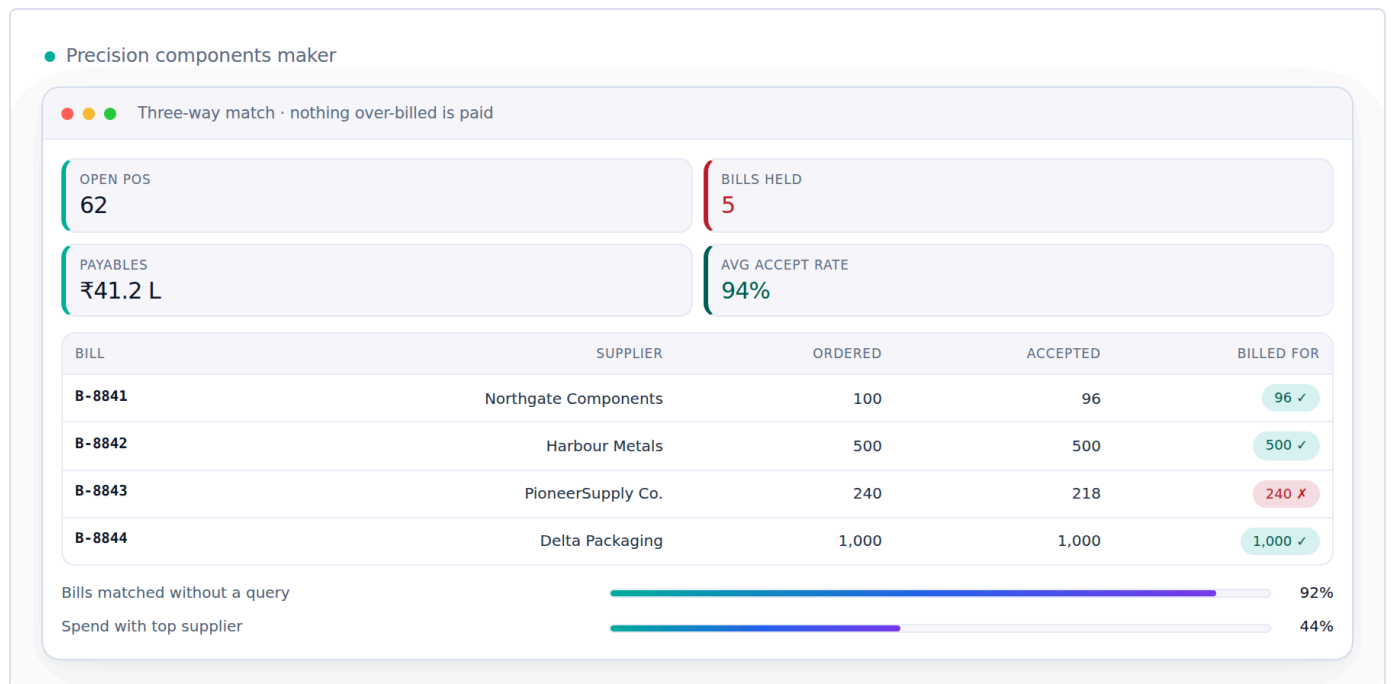
Reads from: Sales · E-commerce / OMS · Inventory & Catalog **Writes to:** Purchase · Manufacturing

App	What it does
Demand Forecast & Signal	What sold, by SKU and by period, turned into a short-term forecast — the number every requisition and every production order downstream is measured against instead of a guess.
Requirement Explosion (MRP run)	Confirmed demand exploded through the bill of materials into what raw material to buy and what to produce, and by when — so a purchase requisition or a production order always traces back to real demand, never to a feeling that stock is getting low.
Open-to-Buy / Budget Ceiling	A spending ceiling set per period regardless of what the demand signal suggests, so a real signal cannot on its own commit more money than the business has decided to risk this season. Every requisition the MRP run drafts is checked against what is left of the ceiling before it goes to Purchase.

Module 07 · Purchase

Nothing over-billed gets paid.

The buy side end to end — and the control that stops you paying for goods you rejected.



Precision components maker · Three-way match · nothing over-billed is paid — illustrative figures

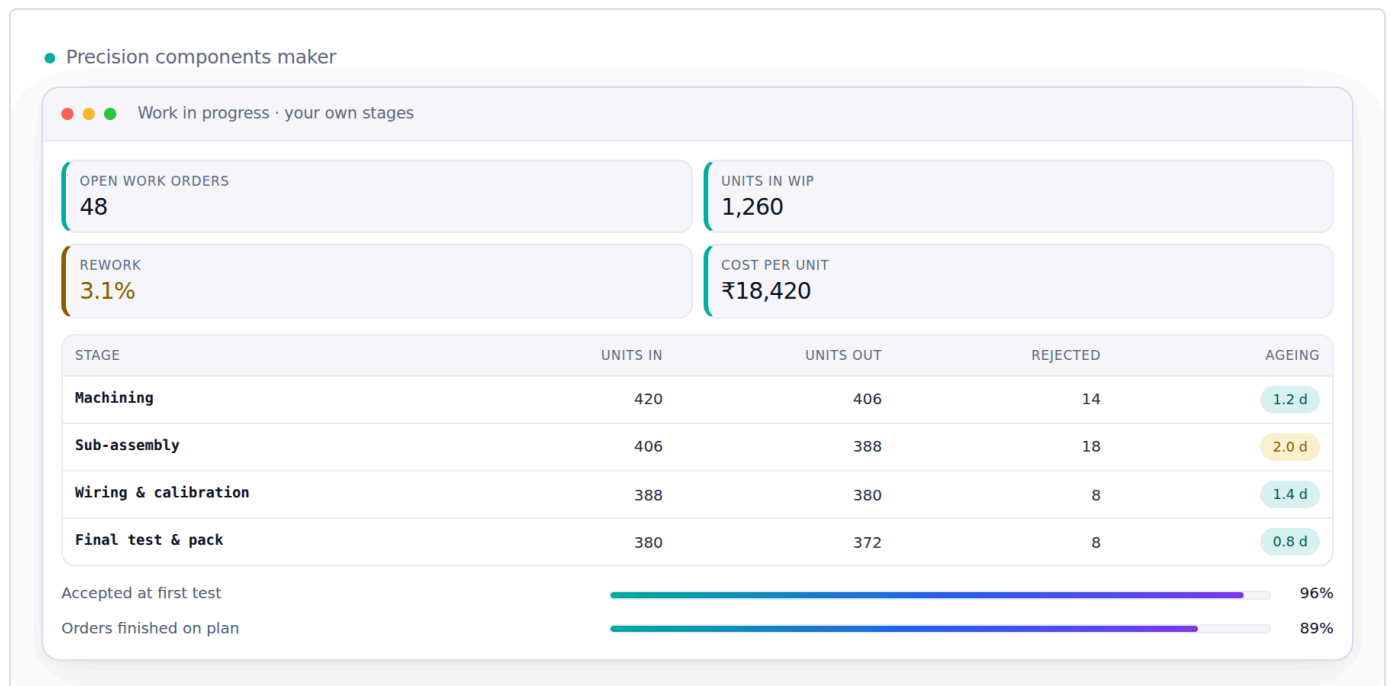
Reads from: Inventory & Catalog · Planning & Requirements (MRP) · Manufacturing **Writes to:** Inventory & Catalog · Accounting & GST · Quality & Compliance

App	What it does
Procurement	RFQ to purchase order to goods receipt, with a strict three-way match before any bill is paid.
Vendor Management	Vendor 360, payables, ageing, a real risk score, and sourcing that follows performance.
Insurance Register	What cover exists over stock in transit, stock in the warehouse and product liability, matched against what is actually moving through Purchase and Warehouse right now — so real value in transit is never quietly running with no cover tracked anywhere.

Module 08 · Manufacturing

Know what a unit really costs to make.

From material in the door to the finished unit — including what every worker earned and what each product actually cost. You define the stages, the rates and the rules; nothing here is fixed to one trade. Design and sample sign-off happen upstream now, and quality inspection and the compliance record live in their own module downstream — this module is purely the making.



Precision components maker · Work in progress · your own stages — illustrative figures

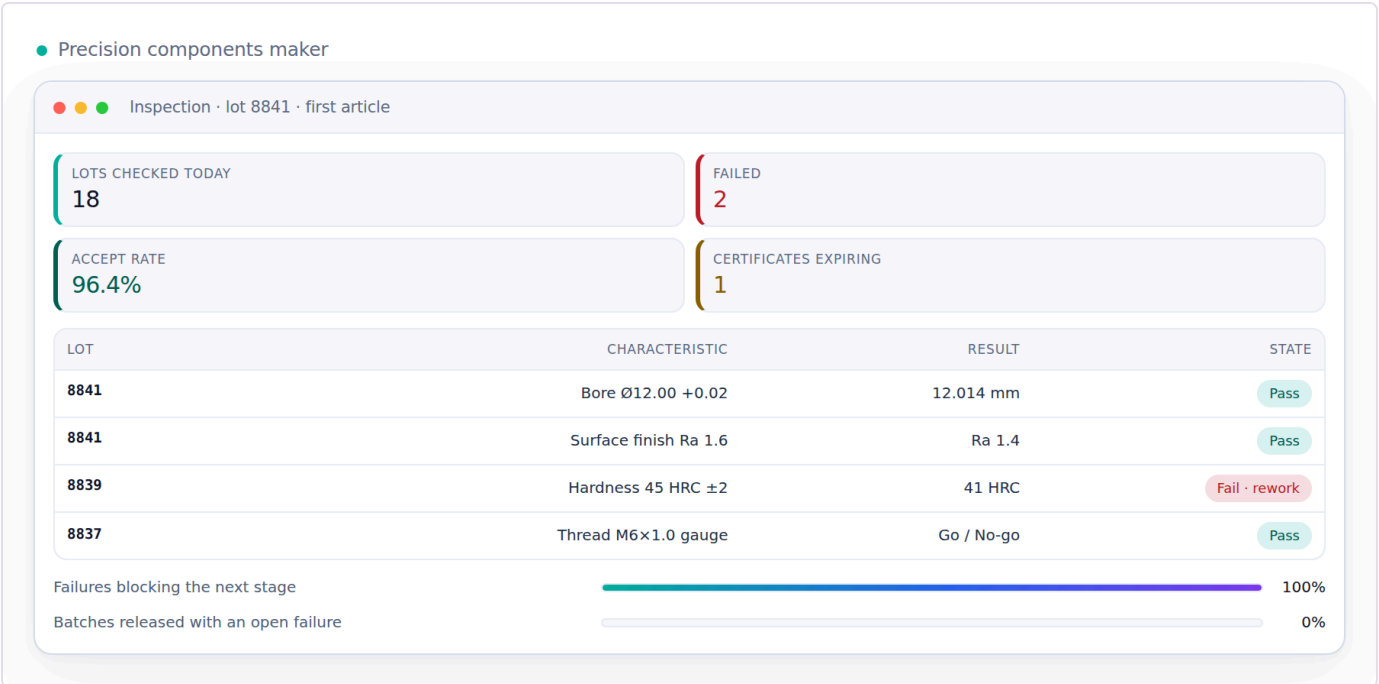
Reads from: Purchase · Planning & Requirements (MRP) · Design & Sampling **Writes to:** Inventory & Catalog · HR & Payroll · Accounting & GST · Quality & Compliance

App	What it does
Production Orders	Your own stages from first operation to finished goods, with work-in-progress visible at each one.
Piece-rate & Contractors	Output-based pay for anyone paid by the piece rather than the hour — pooled completion, per-unit rates, rework and advances resolved into a single payout.
BOM & Consumption	What each product consumes, costed at today's material rates.
Maintenance	Machines, tools and assets: what is due for service, when it was last done, what it cost, and what stopped while it was down. Planned and breakdown work against the same asset record.

Module 09 · Quality & Compliance

Certify what was received and what was made.

Quality inspection used to live buried as one step inside Manufacturing; it stands on its own here because a rejection at goods-receipt and a rejection on the production floor are the same discipline, and because the certificates a business holds — the proof it follows a standard — belong next to the inspections that back them up, not scattered across email.



Precision components maker · Inspection · lot 8841 · first article — illustrative figures

Reads from: Purchase · Manufacturing Writes to: Purchase · Manufacturing · Inventory & Catalog

App	What it does
Quality Control	Accept, reject or rework — on goods received and on goods made — with reasons that feed the supplier scorecard and the performance flags alike.
Certificate & Compliance Register	Every standard the business or a vendor holds — a safety, labour or environmental certification — with its issue date, its expiry, and the audit that backs it, so a certificate about to lapse is visible before a buyer asks for it and finds it already expired. What this register tracks is what a sustainability report downstream is actually built from.

Module 10 · Warehouse

Pick right the first time — and prove what you sent.

Bin-level instructions and barcode scanning, so the right item leaves the building and stock stays honest — and a recording of each parcel being packed, so an argument about what was in it is settled by footage instead of by memory.



Drone & precision manufacturer · Pick wave 22 · Zone A → C — illustrative figures

Reads from: Sales · E-commerce / OMS · Inventory & Catalog **Writes to:** Inventory & Catalog · Sales · E-commerce / OMS

App	What it does
Picking & Bins	Pick lists that tell staff exactly which bin to walk to, in walking order.
Barcode Operations	Scan to pick, pack, dispatch and run a physical stock count from a phone — the same scan whether the order came from a marketplace, your Shopify site or the counter.
Packing Video	Every parcel recorded as it is packed and indexed by its order number, so a wrong-item claim is answered with the clip. The footage attaches itself to the claim that needs it.

Module 11 · Logistics

The courier network itself — rates, failures and the COD money.

Booking one parcel happens on the order, in Sales. This module is the network behind it: what every courier charges before you pick one, what happens to a delivery that fails, and whether the cash collected at the door actually reached your bank.



Drone & precision manufacturer · Handover · what went out against what they took — illustrative figures

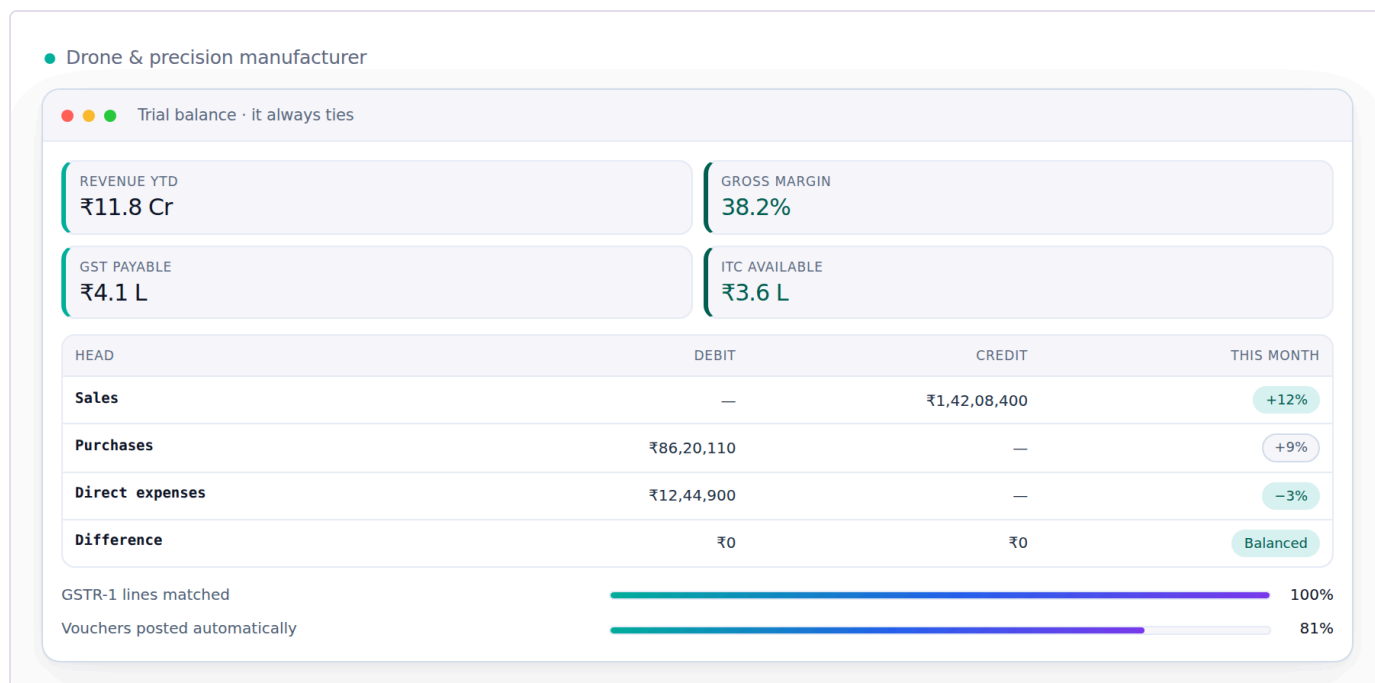
Reads from: Sales · E-commerce / OMS · Warehouse **Writes to:** Accounting & GST · Sales · E-commerce / OMS

App	What it does
Rates & Zones	Every courier's rate card by zone, weight slab and service — so the cheapest and the fastest option for this parcel are both known before it is booked.
NDR & RTO Rescue	A failed delivery worked while it can still be saved — reattempt, call, correct the address — before it becomes a return you pay for twice.
COD Remittance	What the courier collected at the door against what reached your bank, parcel by parcel, with every shortfall named and aged.
Handover & Manifest	What is expected out today against what the courier actually took, counted per courier and per service. The manifest to hand over, the one-time code to confirm it, and a signed record of the parcels that were left behind — so a parcel lost between your table and their van has an owner.
Fleet	Your own delivery vehicles, if you run any — what each trip cost, what is due for service, and what that adds to freight. Optional; most businesses run couriers only.

Module 12 · Accounting & GST

Books that always balance.

A full double-entry ledger built for Indian compliance — not a tax report bolted onto a spreadsheet. Medhava keeps the books on its own: no other accounting package is required, ever.



Drone & precision manufacturer · Trial balance · it always ties — illustrative figures

Reads from: Every module **Writes to:** Finance Reports · Treasury & Financial Planning

App	What it does
Accounting	Double-entry books where every voucher balances and the trial balance always ties.
Invoicing	GST tax invoices and receipts, totals computed from the lines to the paise. Where a channel raises its own invoice, both numbers live on the order — theirs and yours — so the panel's paperwork and your books point at the same sale and neither has to be re-keyed to find the other.
Expenses	Spend captured by category with approvals, and bill OCR to save typing.
GST & Tax	CGST, SGST, IGST, TDS, TCS, input credit, GSTR-1 and GSTR-3B — filed per registration, so a group where one company is registered and another is not files exactly what each one owes and nothing it does not.
ITC Reconciliation	Every input credit matched against the government's own GSTR-2A/2B before a return is filed, so a credit claimed is a credit that is actually on record.
Receivables, Payables & PDC	Payments and receipts allocated against named open invoices, and a register for post-dated cheques that posts only on the date they are realised, not the date they were written.
Fixed Assets & Depreciation	The asset register with both Straight-Line and Written-Down-Value depreciation tracked side by side, and disposal posting its gain or loss straight to the books.
Year-End Close & Period Lock	Profit-and-loss accounts reset and balance-sheet accounts carry forward at year end, and a reviewed period locks against backdated edits until an admin unlocks it — an act that is itself logged.
Finance Reports	P&L, balance sheet, and profit by channel, product and SKU.

Module 13 · Treasury & Financial Planning

Know what cash is coming, not just what already arrived.

Accounting records what happened; this module is concerned with what happens next — how much cash is actually expected, when, and whether spend against a budget is on track before the month closes and turns the answer into history.



Restaurant group · Cash position · four sites · next 14 days — illustrative figures

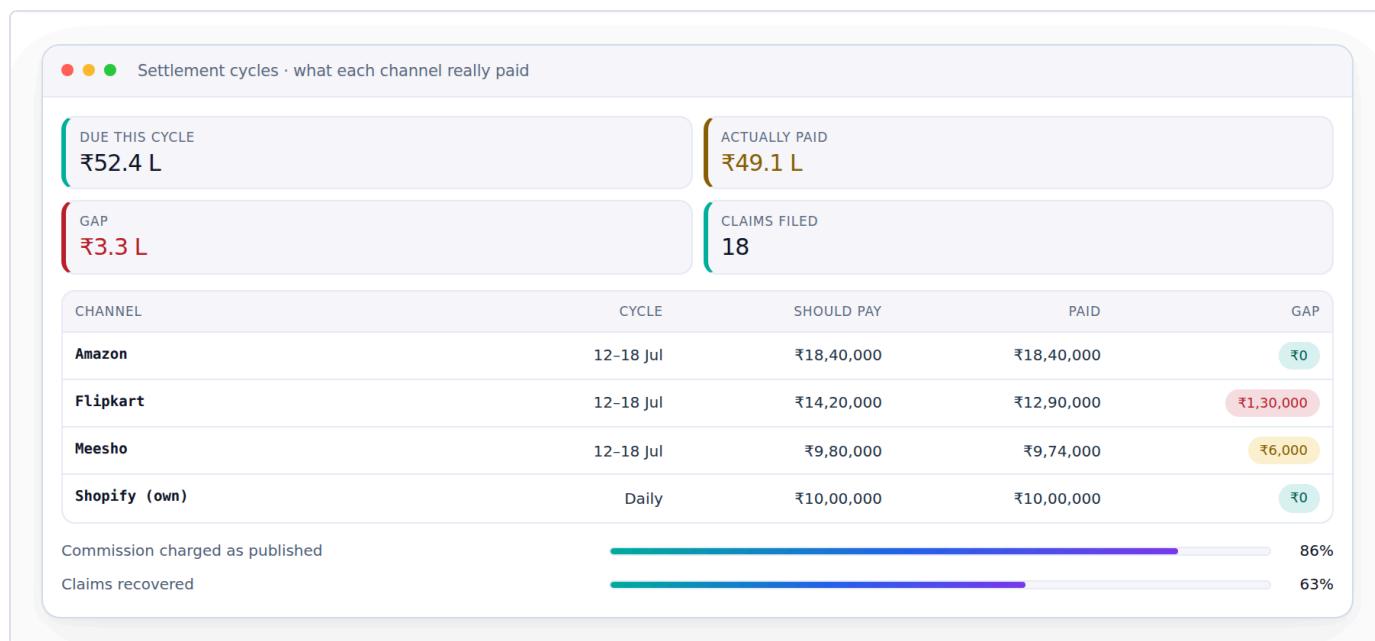
Reads from: Accounting & GST · Sales · Purchase **Writes to:** Accounting & GST

App	What it does
Cash Flow Forecast	Expected receipts and expected payments laid out by week, drawn from open invoices and open bills rather than typed in by hand — so a cash shortfall is visible weeks before the date it would actually bite.
Banking & Reconciliation	Bank statement lines matched against the ledger’s own record of what should have moved, with anything unmatched surfaced instead of silently carried forward.
Budget vs Actual	A budget set per category and period, and actual spend from Accounting tracked against it as the period runs — not only after it closes, when the only thing left to do is explain the variance.

Module 14 · Settlement

Get paid what you are owed — cycle by cycle.

Matching one payout to one order line happens in OMS. This module is the level above it: the settlement cycles each panel runs, the fees it actually charged against the fees it published, and the tax it deducted on your behalf.



Settlement cycles · what each channel really paid — illustrative figures

Reads from: E-commerce / OMS · Accounting & GST **Writes to:** Accounting & GST

App	What it does
Payout Cycles	Every settlement cycle each panel runs — what it should pay, what actually landed in the bank, and on which day — so a late payout is visible the day it is late, not at month end.
Fee & Commission Audit	The rate card a channel publishes against the rate it actually charged, category by category and SKU by SKU. A silent commission increase is caught the first time it is applied — and the tier you are rated in is on the same screen, because the tier is what the rate card hangs off, and losing one quietly costs more than any single deduction.
TCS & TDS Register	Every rupee the panels deducted as TCS and TDS, matched against the portal's own figures — so the credit you claim is the credit you are actually owed.

Module 15 · E-commerce / OMS

Every marketplace and your own website, one queue.

Stop logging into seven seller panels and your own store admin. Every order — Amazon, Flipkart, Meesho, Ajio, Nykaa, JioMart, Myntra, and your Shopify, WooCommerce, Magento or custom site — lands in one pipeline, and one stock number goes back out to all of them. Then the money side closes in the same module: what each channel paid, what it kept, what came back, and what you are still owed.



Order queue · 9 channels · dispatch cut-off running — illustrative figures

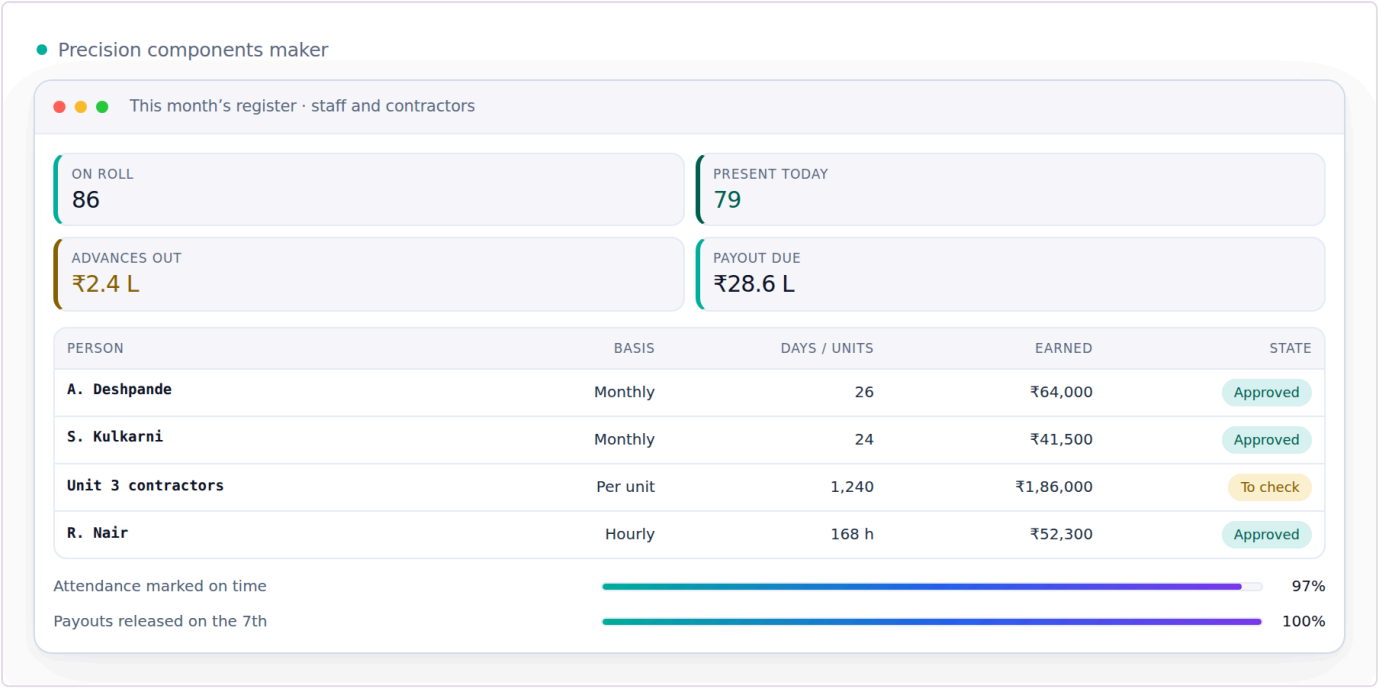
Reads from: Inventory & Catalog · CRM · Sales · Accounting & GST · Logistics · Settlement **Writes to:** Inventory & Catalog · Accounting & GST · Warehouse · Logistics · Settlement

App	What it does
Marketplace OMS	Every marketplace and every storefront in one order queue — Amazon, Flipkart, Myntra, Meesho, Ajio, Nykaa and JioMart alongside Shopify, WooCommerce, Magento, Wix and your own custom site. The stages each channel really uses — to accept, to pack, ready to dispatch, handed over, in transit — with the right cut-off counting down on every order, because a quick-commerce or air-shipped order is not due at the same hour as a standard one. Priority orders first, the day grouped by product so one item is picked once instead of once per parcel.
Order Management	One pipeline from new to delivered, whether the order came from a seller panel, your Shopify or WooCommerce site, a dealer or the counter.
Manual Data Check	Upload the sheets you already download — marketplace orders and returns, and your own counter-shop registers, one file or a whole ZIP — and read ten cross-checks back: money, month, item, state, returns, claims, ads, payouts and GST. Every figure is clickable down to the transactions behind it, and the whole result downloads as Excel.
Reconciliation	Match every marketplace payout to the order line that earned it, and expose the gap.
Claims & Disputes	Turn shortfalls, weight disputes and lost parcels into filed claims with evidence — and answer them before the clock runs out. A claim that is awaiting your response is worth money; one closed for no response is worth nothing, so the days remaining sit on the screen next to the amount.
Returns / RMA	Customer, courier and wrong returns — and the dead stock they actually cost you.
Channels & Storefronts	Connect a channel once and it stays in step: catalogue out, price out, stock out, orders in. Seven marketplaces and any website platform — Shopify, WooCommerce, Magento, BigCommerce, Wix or a custom site over its own API — each switchable without touching your data. Shopsy and any other storefront a channel runs alongside its main one counts as its own channel here. Where a channel has no open interface, its own downloaded report is a first-class way in. A channel may also know you by a different trading name — that is a label on the channel, not a second company, so it tags the order and the payout without ever splitting your books.
Labels & Documents	The channel gives you a PDF; this turns it into something a packer can work from. Cropped to your label size, your own product code printed large where the channel left it off, the invoice and the packing slip merged behind it, and the whole batch sent to the label printer in one job. Reprint a single parcel without redoing the batch — and nothing is ever uploaded to an outside website to be cropped.
Listing & Catalog Manager	Bulk-create and bulk-edit listings across every channel from the one product record in Inventory & Catalog, and catch the mismatches that quietly cost sales: listed but out of stock, or in stock but never listed.
Size / Fit Recommendation AI	A fit suggestion at the point of purchase, built from the item's own measurements and the return history of buyers who picked each size — aimed straight at the return reason that costs the most: the right item in the wrong size.
AR / Virtual Try-On	A way to see drape, fit and colour on a screen before buying, for items where a flat photo alone leaves too much to guess.

Module 16 · HR & Payroll

Pay people right, on time.

Salaries and output-based earnings in one register, with attendance driving both — whether people are on a monthly wage, an hourly rate or paid by what they finish.



Precision components maker · This month's register · staff and contractors — illustrative figures

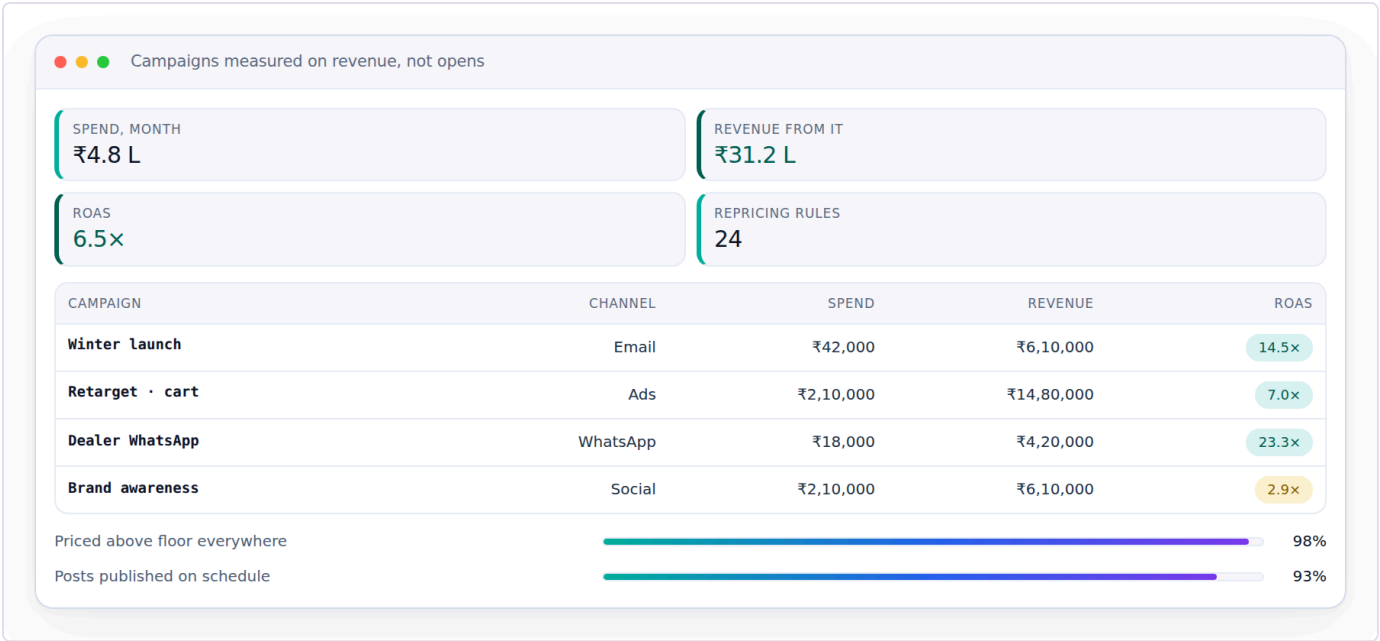
Reads from: Manufacturing Writes to: Accounting & GST

App	What it does
Staff & Contractors	Attendance, effective-dated salary and output-based earnings in a single register, whoever is on it.
Time-off & Advances	Leave, festival advances, and exactly how they change this month's payout.
Appraisal & Hiring	Performance reviews and a hiring pipeline that ends in an employee record.
Recruitment	The pipeline before someone becomes an employee — an opening, the people who applied for it, a trial piece where the work itself is the interview, and the decision with its reason kept. It matters more in a skilled trade than in most: a person is taken on for skill at one particular kind of work, and the trial output is the evidence, so it is recorded against that work and the rate that would apply rather than remembered as an impression. A candidate who is not taken on now stays findable when the same skill is needed in a busy month, and their personal documents are held under the same consent and retention rules as anyone else's, not in a folder on somebody's phone.
Payout Execution	Where the calculation in the earnings register actually turns into money leaving the business — bank batch, UPI, cash against a signed receipt — with the method and the reference recorded against every payout, so the register's total and the money that actually moved can always be checked against each other.

Module 17 · Marketing

Sell more without discounting.

Plan content, run campaigns, and let rules keep your prices competitive while protecting margin.



Campaigns measured on revenue, not opens — illustrative figures

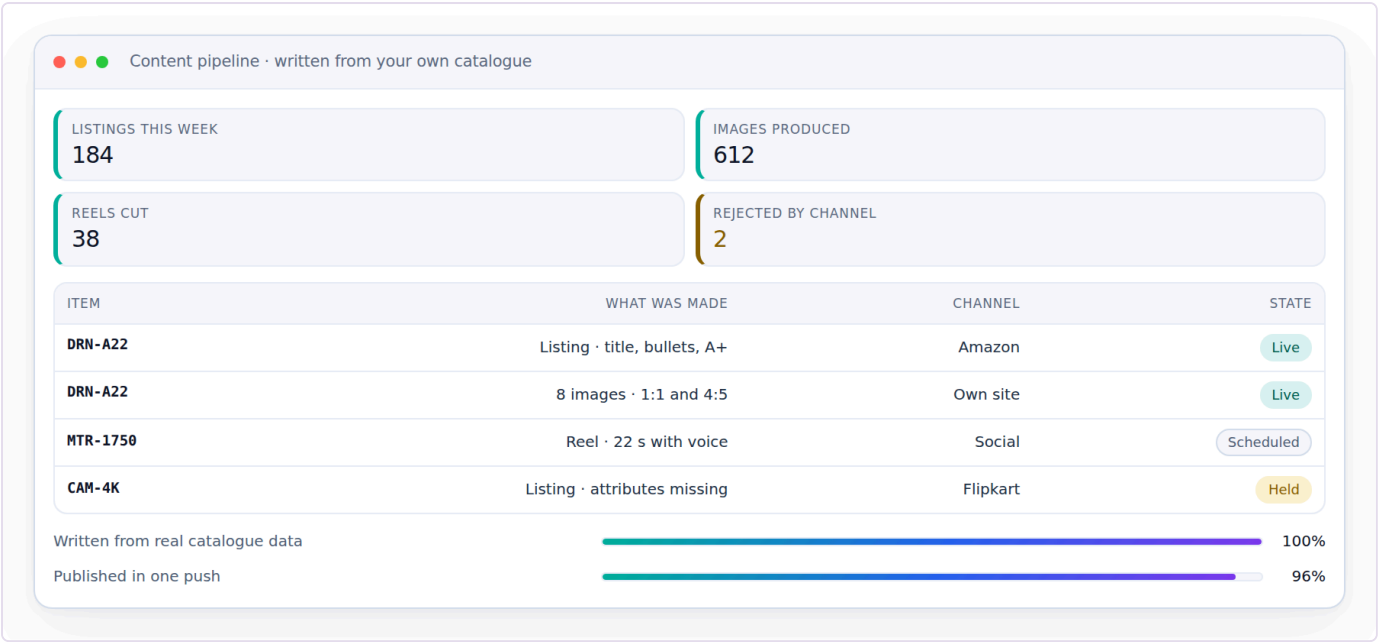
Reads from: Inventory & Catalog · CRM **Writes to:** Sales · E-commerce / OMS

App	What it does
Social Calendar	Plan and publish across every channel from one calendar.
Campaigns	Email, SMS and WhatsApp campaigns measured on real revenue, not opens.
Repricing Engine	Rules per channel and SKU — floor, ceiling, match-lowest, festival overrides — and what each change actually did. A price that went up and took the orders down with it shows as exactly that, next to the rule that raised it, so the rule can be reversed on evidence rather than on a feeling.
Automation	If this happens, do that — across any module, without writing code.
Blog & Pages	Articles, landing pages and category copy written, scheduled and published straight to your own site — Shopify, WooCommerce, Magento or a custom CMS — with the meta title, description and internal links set before it goes out.
Events	Trade shows and exhibitions worked as a channel of their own — booth, budget and every lead captured on the floor landing straight in CRM instead of on a stack of business cards.
Website & Page Builder	The storefront itself, built by dragging sections into place rather than by editing a theme file — hero, product grid, size guide, lookbook, contact form — each block reading live from the catalogue, so a price or a stock state on a landing page is the same number the order screen uses instead of a figure someone pasted in and forgot. Blog & Pages above writes articles into a site that already exists; this is for the businesses that do not have one, and it is the gap that shows up plainly when this module list is set beside a mature open-source ERP: they ship a full site builder next to the blog, and until now this did not.
Markdown / Clearance Optimization	The same rule engine that reprices for competitiveness, aimed at ageing stock instead: when to start discounting it and by how much, before it becomes a warehouse write-off rather than a sale at a lower margin.

Module 18 · AI Content Engine

Write it, shoot it, cut it — from the catalogue you already have.

Listings, ads, email, product photography and reels, all generated from your own catalogue — so the words match the product and the picture is the right size for the channel it is going to. Words, images and video sit in one module because they are one job.



Content pipeline · written from your own catalogue — illustrative figures

Reads from: Inventory & Catalog **Writes to:** Marketing · E-commerce / OMS

App	What it does
Content Engine	Fourteen stages in your own voice, from buyer research and competitor reading through hooks, channel-ready listings, ads, social posts, video scripts, song lyrics, the publishing calendar and alt text — each written from your own catalogue, so the words match the thing.
Image Studio	Layers, free transform, background removal, channel presets and SEO alt text — a phone photo becomes a channel-compliant product image.
Video Studio	Text and image to video, reels and ad cuts sized for every channel.
Design Studio	A full design surface — templates, layers, undo and redo, any colour, exact sizing, background images and stock elements — exporting PNG, JPG or PDF at whatever size the channel or the printer asks for.
Motion Renderer	A reel rendered from a page of HTML and CSS — the same layers, fonts and brand colours the Design Studio already uses — into a real MP4, on this machine, with nothing uploaded. It is not a screen recording: the clock is faked, the animation is seeked to the exact instant of each frame, and only then is that frame captured, so the render does not care whether the machine was busy. Rendering the same festival banner twice produces the same file to the byte, which is what makes a reel something you can check and re-cut rather than something you have to watch all the way through and hope about.
Narration Studio	A voice over the reel, in the language the buyer actually speaks — the same script the Content Engine wrote, spoken. The default needs nothing installed and no key, because every modern browser can already speak; a self-hosted or cloud voice sits behind it as an interchangeable provider for anyone who wants a cloned or branded one. Long scripts are split at sentence boundaries and rejoined, so a two-minute description is not cut off at whatever limit a service imposes. A voice cloned from a real person is only ever used with that person's recorded consent, filed against them in Data Privacy & Consent like any other permission.
Image Generation Slot	Generated imagery — a model on a background you do not have to shoot, a festival backdrop, a lifestyle scene — as a capability with interchangeable providers rather than a bet on one service: a queue of jobs, a preview while it works, inpainting to fix one region, upscaling and face correction. This one needs a graphics card. Image models cannot run on an ordinary office machine or on the container this system is built in, so what ships is the queue, the review screen and the provider slot, with the generating itself done by whichever engine you point it at — your own GPU box, or a cloud service, swapped without touching anything else. Said plainly here because the alternative is a screen that looks finished and produces nothing.
Publisher	One push sends the finished listing, picture and copy everywhere it has to appear — your storefront, each marketplace, each social account — and reports back what actually went live and what was rejected, with the reason.

Module 19 · SEO, AEO & AIO

Be found by a search box, an answer box and an AI.

Content already exists once this module is reached; here it is made findable — by a traditional search engine, by the answer box above the results, and by the AI assistants now answering shopping questions directly instead of sending someone to a results page.



Multi-doctor clinic · Local search · what patients actually search — illustrative figures

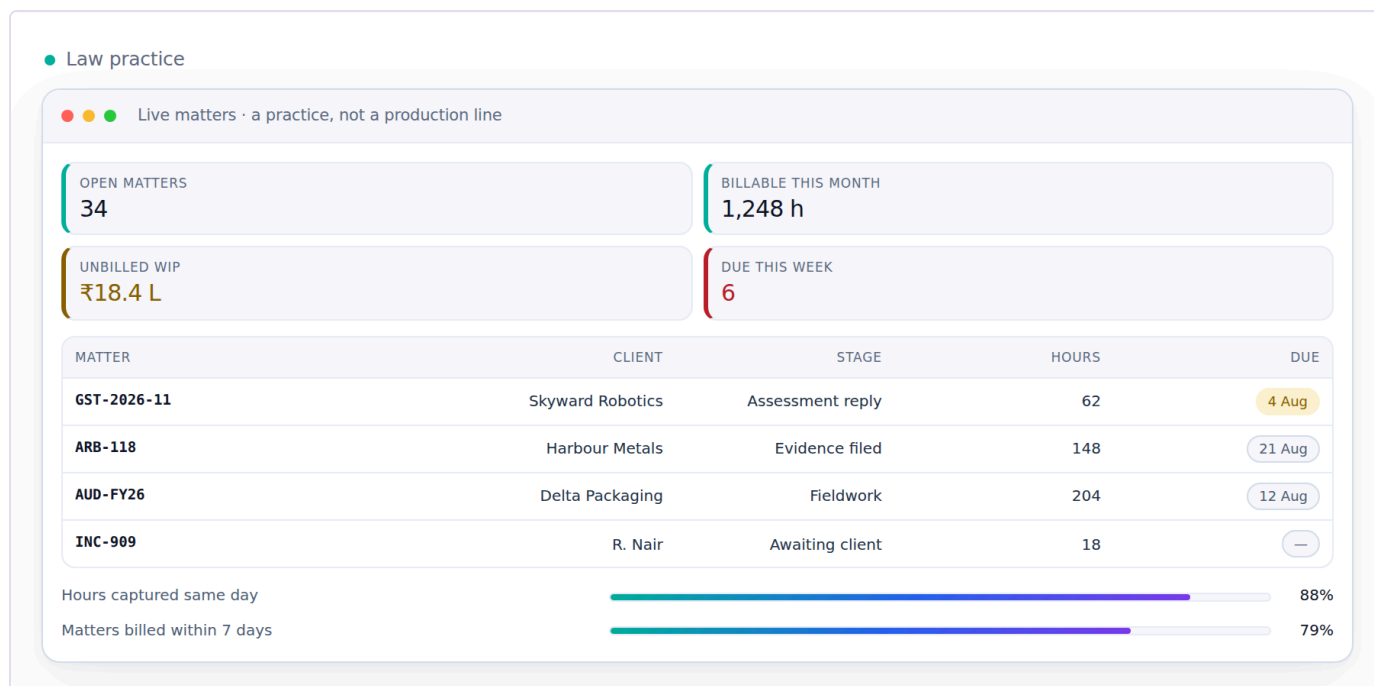
Reads from: Inventory & Catalog · AI Content Engine **Writes to:** Marketing

App	What it does
Technical SEO & Schema	Structured data, sitemaps and page-level technical checks against your own storefront and content pages, so a search engine can read what a page is actually about instead of guessing from the text alone.
Answer-Engine Optimization	Content shaped to be quoted directly by an answer box or a voice assistant — a clear, citable answer near the top of the page — rather than written only for a person to scroll through.
AI-Engine Visibility Tracking	Whether and how this business is actually cited when someone asks an AI assistant a shopping question in this category, tracked over time — the same discipline as rank tracking, aimed at a newer kind of result page.

Module 20 · Projects & Collaboration

The work that is not an order — and the talking around it.

Not every business runs on orders. A law firm runs on cases, an agency on engagements, a workshop on jobs, a builder on sites. This module holds that work on the same records as everything else, so the time, the cost, the documents and the decisions attached to it end up in the books rather than in somebody's inbox.



Law practice · Live matters · a practice, not a production line — illustrative figures

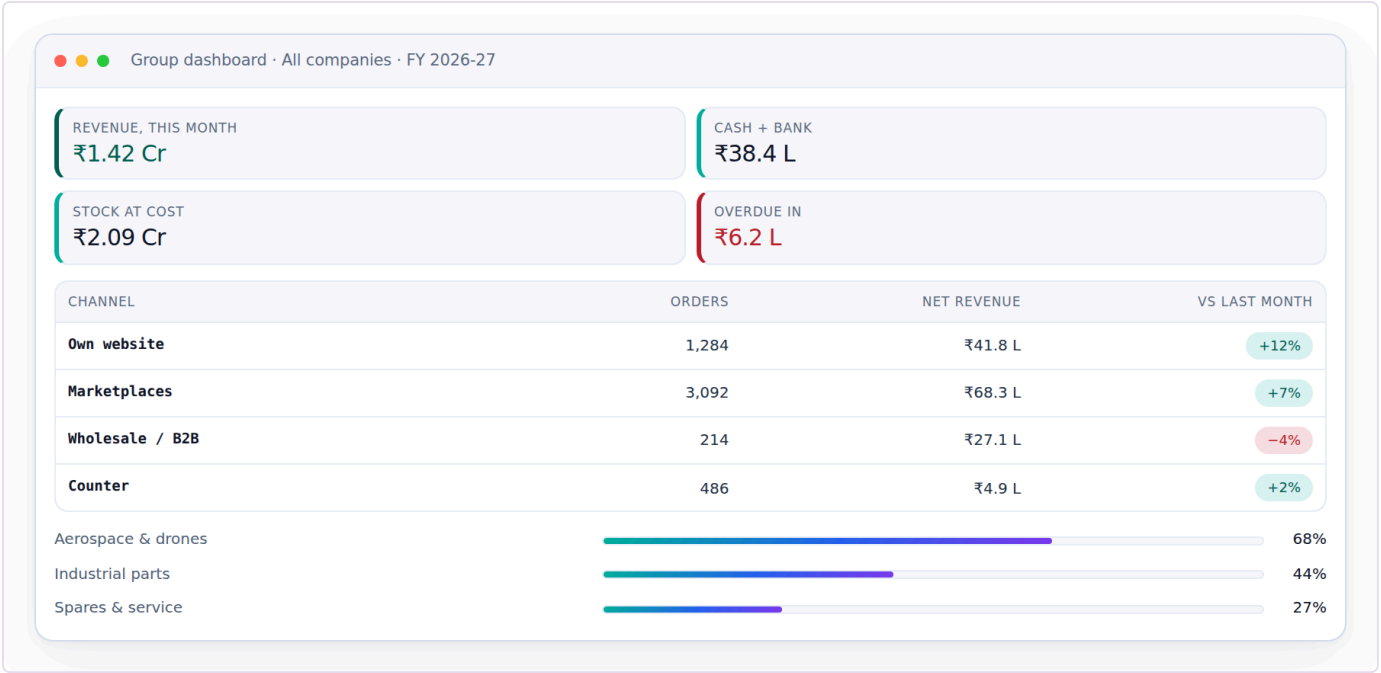
Reads from: CRM · Sales · HR & Payroll · Inventory & Catalog **Writes to:** Accounting & GST · HR & Payroll · CRM

App	What it does
Projects & Cases	A project, a case file, an engagement or a job — whatever your work is called. Stages you define, owners, deadlines, documents, billable time and real cost, all on one record the ledger can see.
Timesheets & Planning	Who is on what this week, and the hours that actually went in — against a project, a case, a job or a machine. Billable and non-billable separated, so a rate card turns straight into an invoice and a real cost.
Approvals	One queue for everything waiting on a yes — a purchase order, a discount, a leave day, a credit note, a payment. The rule that sent it there is on the screen next to it, and the decision goes to the audit record.
Forum	Questions and answers that outlive a chat — for customers, dealers or staff — with the useful ones kept where the next person will actually find them.
Automation Studio	The place a person builds “when this happens, do that” by dragging it out and watching it run — a trigger, the steps after it, a branch where the answer decides which way to go — over the same event stream every module already writes to. Marketing’s Automation aims that idea at campaigns; this is the general one, reaching any module: a payment marked short holds the next dispatch and opens a claim, a worker’s pooled output crossing a threshold raises the payout for approval, a return marked damaged writes off the piece and messages the buyer. Every run is kept — what fired it, each step, what each step returned — because an automation nobody can inspect afterwards is a rule the business cannot trust with its money.
Discuss	Conversation attached to the record it is about: this order, this bill, this case. A year later the reason for a decision is still sitting next to the decision.
Knowledge Base	A searchable internal wiki of standard operating procedures, scoped to the role it applies to, so how a task is meant to be done is written down once instead of carried in one person’s head.

Module 21 · Dashboard & BI

See the whole business without asking anyone.

Every number in Medhava rolls up here as work happens — no exports, no waiting for month-end, no asking three people for their sheet. It is the last module built for a reason: it has nothing to show until the other twenty are producing real records for it to read.



Group dashboard · All companies · FY 2026-27 — illustrative figures

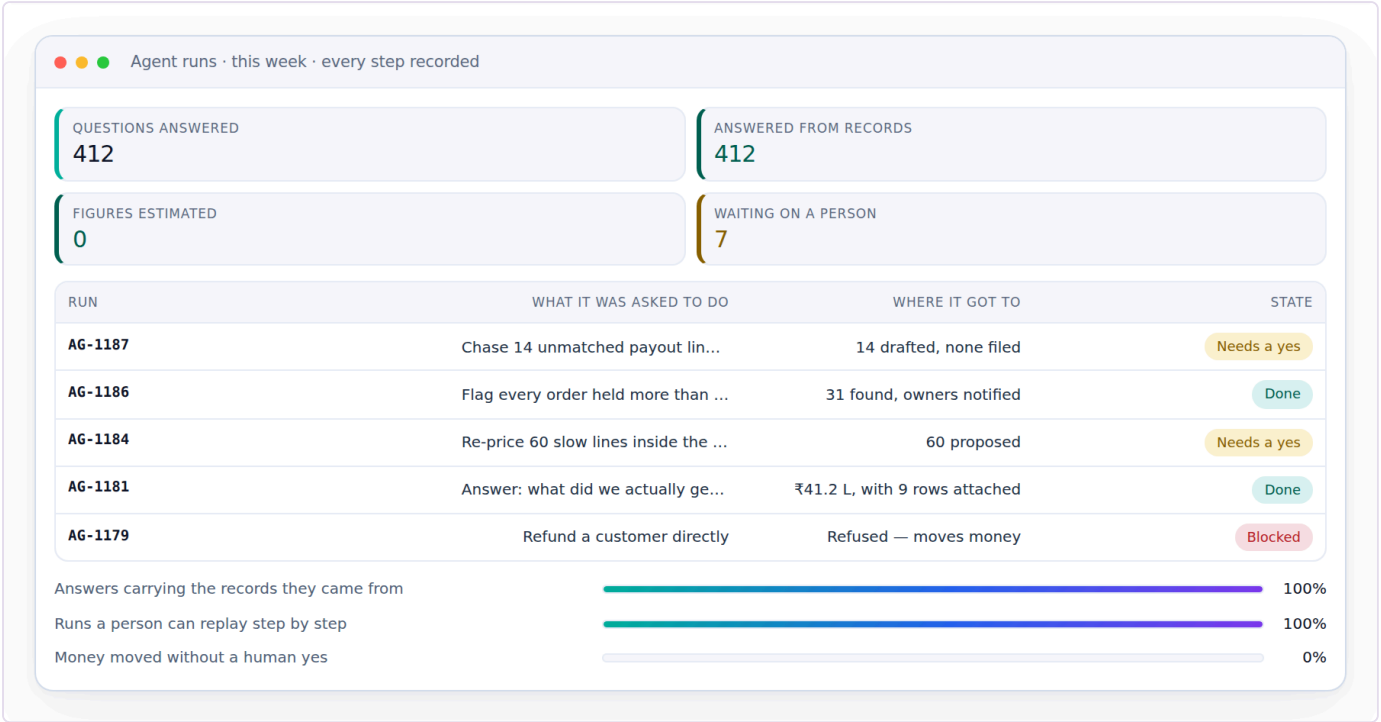
Reads from: Every module **Writes to:** —

App	What it does
CEO Dashboard	Cash, sales, stock, profit and alerts on one screen, refreshed as work happens.
Report Builder	Drag the fields you want into a report and save it for the whole team.
Group Consolidation	Several companies, one set of figures — sales, cash, stock and profit rolled up across every company you run, inter-company entries removed, with years of history to compare against. Add a company whenever the business grows one; nothing in the software caps the number, only the plan does. And a company with no tax registration of its own — a job-work arm, a new venture not yet registered — is a company like any other here, kept in the group figures without being dragged into a return it does not belong in.
Excel Dashboard Builder	A full workbook — financial summary, HR, purchase, sales, inventory and production, GST, expenses — generated from the live records behind every other screen, with each company shown as its own row and a consolidated row that is a formula over them, never a separately typed total.
ESG / Sustainability Reporting	Water usage, chemical compliance, waste and packaging, reported from the same certificate and audit records Quality & Compliance already keeps — so a sustainability report is a query over evidence already on file, not a separate exercise assembled once a year from scratch.

Module 22 · AI Assistant, Agents & Automation

Ask the business a question — and let the routine work run itself.

Last for the same reason Dashboard & BI is late: something that answers questions about the whole business can only be built once the whole business is in one place. Three different things live here and the difference between them matters. An ASSISTANT answers a question you asked, from the records, with the records attached. A CHATBOT holds the same conversation with your customer instead of you. An AGENT is given a job rather than a question and works out the steps itself. That last one is what separates this module from Automation Studio in Module 20, where a person draws the steps in advance and the rule runs the same way every time; and from Automation in Module 17, which fires marketing campaigns and nothing else. Both of those stay exactly as they are — this module sits above them and calls them, rather than replacing either. Module 18 writes content; this module answers and acts.



Agent runs · this week · every step recorded — illustrative figures

Reads from: Every module **Writes to:** Projects & Collaboration · CRM · Marketing

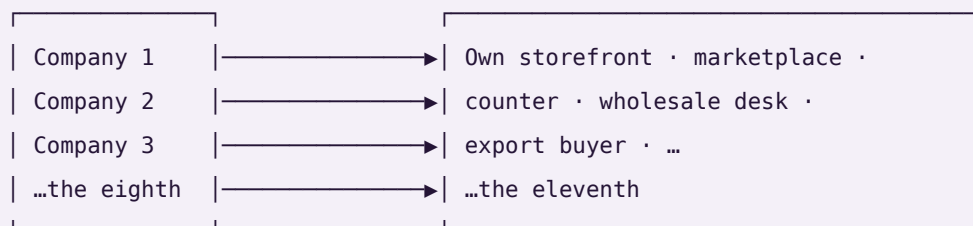
App	What it does
AI Assistant	Ask in your own words — “what did Myntra actually pay us last week, and what is still short?” — and get the answer with the rows it came from sitting underneath it, each one clicking through to the record. It reads the ledger, the stock table and the settlement lines the same way a report does, so the figure it gives is the figure the books give. When it cannot find the answer it says so and shows what it looked at; it never estimates a number and presents it as a fact, because a plausible wrong figure is far more expensive than an honest blank. It answers only from records the person asking is already allowed to open, so it can never become a way around permissions.
AI Chatbot	The same engine turned to face the customer, on your own storefront and on WhatsApp: where is my order, will this size fit me, I want to return this. It reads the real order and the real size chart rather than a script written six months ago, and it will say “let me get someone” instead of guessing at anything about money, a refund or a complaint. The handover goes into the Module 04 Helpdesk queue with the whole conversation already attached, so the person picking it up starts where the customer left off instead of asking them to explain again. It never asks a customer for a card number, a bank detail or a password — that promise does not get a chatbot-shaped exception.
AI Agents	A job rather than a question: “chase every unreconciled settlement line from last week and draft the claim for each.” The agent works out the steps, does them, and stops at the point where a person has to decide. It runs inside a scope you set — which records it may read, which it may write, and how much it may spend through the Module 01 Provider Router — and it cannot quietly widen that scope mid-run. Anything that moves money, files a claim, changes a price or sends a customer a message waits for a human yes.
Agent Guardrails & Run Log	What each agent is allowed to touch, written down as a scope rather than trusted to a prompt, and every run recorded step by step: what started it, what it read, what it proposed, what a person approved, what it actually changed. Kept in the same audit trail as everything else, with the same absence of an off switch. An agent whose working nobody can inspect afterwards is not a colleague, it is an unexplained entry in the books.
Knowledge & Retrieval	The index that makes the answers grounded: your own designs, rate cards, settlement files, standard procedures and past decisions, searchable so a reply quotes what is actually on file instead of what a model remembers about the trade in general. Permission-scoped at the row, so two people asking the same question get answers drawn only from what each of them may already see.

Companies and channels — how many is up to you

Whatever number you have, it is not a setting this system was built around, and it is not a ceiling. A company is a **row**. A channel is a **row**. Every business record carries the company it belongs to, and every sale carries the channel it came through. Ten companies selling on ten channels each is the same three tables and the same code as one company selling on one.

COMPANIES (a row each)

CHANNELS (a row each, per company)



Three things this buys you, and one it deliberately refuses.

Each company's books are its own. Its trial balance balances on its own. No report can reach across into another company's rows — not by convention, but because a journal line can only point at an account that belongs to the same company, and a test checks that no line anywhere ever does.

The group is the sum minus what you sold yourselves. When one company in a group sells to another, that is revenue in one set of books and cost in another. Adding the companies up would report a group turnover the group never earned from the outside world. Every entry that names a sister company is eliminated at group level, and the consolidation returns all three numbers — gross, eliminated, group — so you can see the elimination rather than take it on trust.

The channel is a dimension of the sale, never of the stock. You can read this month by channel, by company, or by both. What you cannot do is keep a separate stock number per channel, and that is on purpose: the last unit sold on one marketplace has to vanish from the others at that instant, which per-channel inventory cannot do.

And the reporting follows your own sheets. The report's columns come from the sheets that are actually in the workbook you give it. A fourth company is a new sheet, not a new version of the software.

This is checked, not claimed. `core/tests/core.test.js` builds ten companies with ten channels each — a hundred channels — posts an order down every one plus ten inter-company sales, and asserts every company's books balance, that no journal line points at another company's account, and that the group figure is the plain sum **minus** inter-company trade: ₹2,10,500 gross, ₹50,000 eliminated, ₹1,60,500 group. It then calls the same builder for eleven companies and eleven channels with no code changed.

The rules that hold everywhere

1. **No app depends on any single outside company.** Every capability — books, marketplaces, AI writing, couriers, payments, messaging, storage, GST, printing, barcode — has several interchangeable providers and a by-hand option, so the system works with nothing connected at all. **A provider named as the source of a figure is a bug;** providers move messages and money, the ledger originates numbers.

2. **The books are this system's own.** No other accounting package is required, ever. Tally, BUSY and Zoho remain available for anyone already running one — nothing assumes them, and no figure is ever sourced from one.
 3. **Nothing ever asks for an account password.** Outside services connect with a scoped, revocable key. *This system will never ask you for a marketplace, bank or account password. If any screen ever does, it is not this system.*
 4. **Money is integer paise, never a floating-point number**, because float arithmetic accumulates the error that eventually shows up as a trial balance that will not tie.
 5. **Values that change over time are effective-dated** — a salary, a price, a tax rate, a commission. March payroll resolves the salary in force *in March*. A missing value is an error, never silently treated as zero.
 6. **Nothing is deleted, only deactivated.** A person who leaves keeps their name attached to years of earnings, approvals and audit rows that must still resolve.
 7. **The audit trail has no off switch.** Eight years, before-and-after values, as the MCA rule requires — because an audit trail that can be switched off is one that gets silenced exactly when it matters.
 8. **Gates, not warnings.** Each app refuses one thing outright, because a warning gets clicked through on a busy afternoon. A cash-on-delivery order cannot be packed below its advance. A bill for more than was accepted cannot be paid. Every gate is also a self-test.
 9. **It is not trained. It is built.** No model learns from your data. Every rule is written down, visible on the Wiring screen, and checked by a self-test — so it is right on day one.
 10. **You can reach it from anywhere, but it cannot be reached into.** Ask & Print takes a plain line from your phone and sends a PDF back. The office reaches out; the internet never reaches in.
-

How it is verified

Nothing ships because it looked right on a screen.

1. **The arithmetic, with no screen involved.** Each engine runs in isolation and its self-tests execute against seeded data.
2. **Every screen and every control, in a real browser.** Each build opens in headless Chromium; every screen is visited and every interactive control on it is clicked. Any console error fails the build.
3. **The real job, with the result asserted.** Not "does the button click" but "did the thing happen". A control that looks alive but changes nothing fails the build.
4. **Against a business's own figures.** Where a business already knows the answer, the software has to reproduce it — and where it cannot, the reason is named rather than the number quietly adjusted. In the worked implementation carried furthest, every record in the owner's hand-made reference report is placed into a bucket with a named cause — matched exactly, changed at source, rate added since, a rule that applies, or present only in a source file that has since been restructured and can no longer be read — and the check fails on any record whose difference has no explanation. A mismatch is a bug, not a rounding difference; an unreadable input is a stated limitation, not a passing test.
5. **A structural audit.** Every "comes from" on every Wiring screen must name a module that actually exists, no vendor name may ever be the source of a figure, and the app count in every file must match this one.

6. **The shipped copy, not the working copy.** The packaged archive is extracted and re-tested in the folder a customer would open it in, because a packaging step that quietly renames a file breaks nothing until it is in somebody's Downloads folder.

The honesty charter

This is the standard the build is held to, and it is written down because a standard nobody wrote down is a standard nobody can be held to.

1. **This page describes a design.** Everything on it is what the system is being built to be, and nothing on it claims to already exist. When a part of it is finished, it will say so with the test that proves it — never before.
2. **Counts are counted, never claimed.** Every module and app figure on this page is read from the canonical module list when the page is built. No number here was typed by hand.
3. **Progress is reported as it is.** If tests fail, the failure is shown with its output. If a step was skipped, it is named as skipped. "Done" means implemented, tested and checked against the original request — not "the code has been written".
4. **Every capability names its alternatives.** No part of this system depends on a single outside company. Each layer names what it is built on, at least two replacements, and the interface the rest of the code talks to — so changing a supplier is a setting, not a rebuild.
5. **Uncertainty is surfaced, not smoothed over.** Where something cannot be verified, it is reported as unverified rather than presented as fact.

Medhava · one business, one brain · 22 modules · 113 apps · one shared data core

PART TWO — THE PLAN OF ACTION

One business operating system. Any industry. One shared data core.

22 modules · 113 apps · 285 rules (86 enforced) · 113 tables · 46 product screens across 12 sectors · 19 tool capabilities · 6 industry packs. Every count in this line is read from `brand/site/modules.js`, `brand/site/rules.js`, `brand/site/shots.js`, `brand/site/tools.js`, `core/schema.postgres.sql` and `core/packs/` by `brand/site/mkcounts.js` — no figure here was typed from memory.

What this document is. The plan for building Medhava as a product that any business can adopt, from planning through execution. It is not the Vastrangam plan with the names changed. `PLAN_OF_ACTION.md` is that: one ethnic-wear group in Surat adopting this engine, with its own karigar payroll, its own three companies, its own migration off an accounting package. That document stays as it is, and it is the most useful thing in this repository — it is the worked example of the whole exercise, an entire industry implementation carried to the level of detail that proves the engine bends.

This document answers the different question: **how does the same engine serve a dental clinic and a freight forwarder and a dairy co-operative, without becoming a different program each time?**

PART I — WHAT MEDHAVA IS

M1 · ONE ENGINE, MANY TRADES

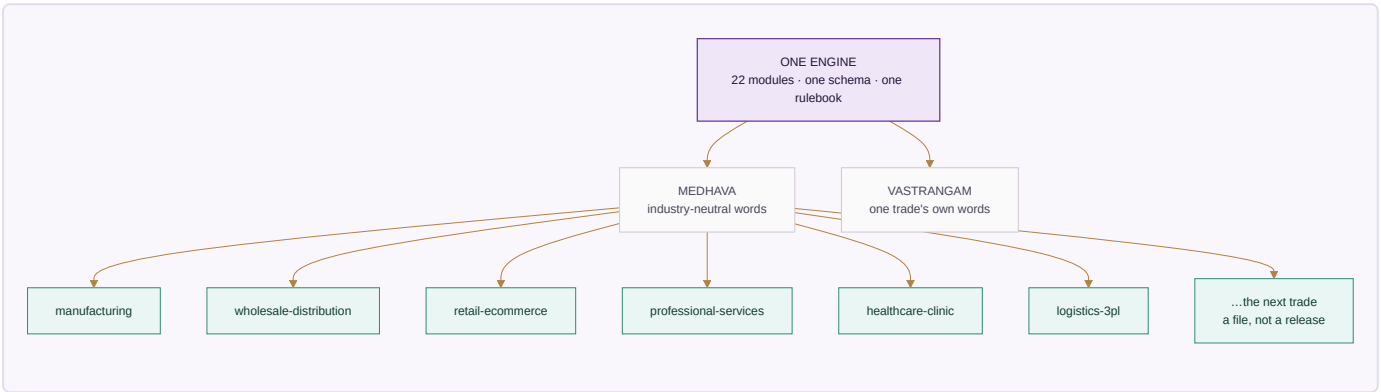
Medhava is a single application over one shared data core. A business event is entered once and flows everywhere it belongs — that law, and the cascades that follow from it, are set out in full in `PLAN_OF_ACTION.md` §A0 and are identical here, because it is the same engine. This document does not restate them.

What is specific to Medhava is the claim that the engine does not know which trade it is in. That claim is either true in the code or it is marketing, so here is where it is enforced:

The claim	What makes it true
No module assumes an industry	<code>brand/site/modules.js</code> is audited at build time; trade vocabulary in it fails the build
Trade words live in one place	The edition overlay may change wording only — <code>build.js</code> compares the structural shape before and after applying it and exits if a module number, an app name or an app count moved
The same structure ships twice already	Two editions build from one module list: MEDHAVA neutral, VASTRANGAM in one trade's own words
The number of companies is data	<code>core/tests/core.test.js</code> posts across a 10 × 10 company × channel grid, then runs 11 × 11 with no code changed

The sentence the whole product rests on. A clinic's appointment, a factory's work order, a law firm's matter, a contractor's site and a service firm's job are the same record with different words on it. Everything below is the work of making that true rather than clever.

Drawn, because the shape is the argument:



The editions differ in **wording**. The packs differ in **configuration**. Neither is a copy of the code, and that is the only reason one team can carry all of them.

M2 · THE INDUSTRY PACK ENGINE

An earlier version of this document said, in this place, that the industry pack was **specified, not built** — that a third trade meant someone writing a third overlay by hand, and that this did not scale to a product. That was the honest state of it then. It is built now, and the paragraphs below say what was built, what it refuses, and which trades it was pointed at first and why.

M2.1 · Which trades actually run this software

The pack order was not chosen by taste. It was chosen by looking up who actually buys ERP, order management and warehouse management, and building for the largest first. Published estimates differ between research houses — sometimes considerably, because "share of users" and "share of revenue" are different questions — so the figures below are attributed, and the ranking rather than any single number is what the build order rests on.

ERP — who the users are

Industry	Share of ERP users	Note
Manufacturing	~21% of users; ~32% of market revenue	The largest block on every measure. Cited elsewhere as 19.7% of revenue — the spread is why the ranking, not the decimal, is what is used
Banking, financial services, insurance	~16%	Heavily served by specialist systems rather than general ERP
Professional and financial services	13.86%	The second-largest block, and the one with no stock at all
Distribution and wholesale	9.90%	Credit and movement, not production
Healthcare	4.95%	Small today — and the fastest-growing, at 22.37% CAGR to 2030
Construction	1.98%	
Education	0.99%	

Finance and insurance, manufacturing and public administration together account for roughly 60% of total ERP spend.

WMS — who the users are

End user	Share	Note
Retail and e-commerce	~28%, about \$1.37bn in 2025, 10.1% CAGR	The largest WMS segment
Manufacturing	23.6–30.22% depending on the source	
Healthcare and pharmaceuticals	~10%	The fastest-growing, at 12.2% CAGR

The whole WMS market is put at \$3.4bn–\$5.92bn in 2025, again depending on who is counting and what they count as WMS.

OMS and third-party logistics — which markets are served

Market served by 3PLs	Share
Transportation	90%
Manufacturing	83%
Retail	83%
E-commerce	68%
Wholesale	67% (down from 83% the previous year)

And among the technology services those providers offer, **order management ranks first** — ahead of visibility (86%), transport management (84%), optimisation (77%) and ERP integration (75%). That is the finding that matters most for Medhava's shape: for a large part of this market the order, not the ledger, is the system of record.

Sources, read 2026-08-23: Grand View Research, Mordor Intelligence, HG Insights, MarketsandMarkets, Manufacturing Lead Generation, Inbound Logistics (3PL Perspectives), Electro IQ, openpr, NetSuite. These are market-research estimates, not measurements; they are used here to order a build queue, which is what they are good for, and not quoted anywhere in the product as fact.

M2.2 · What a pack is

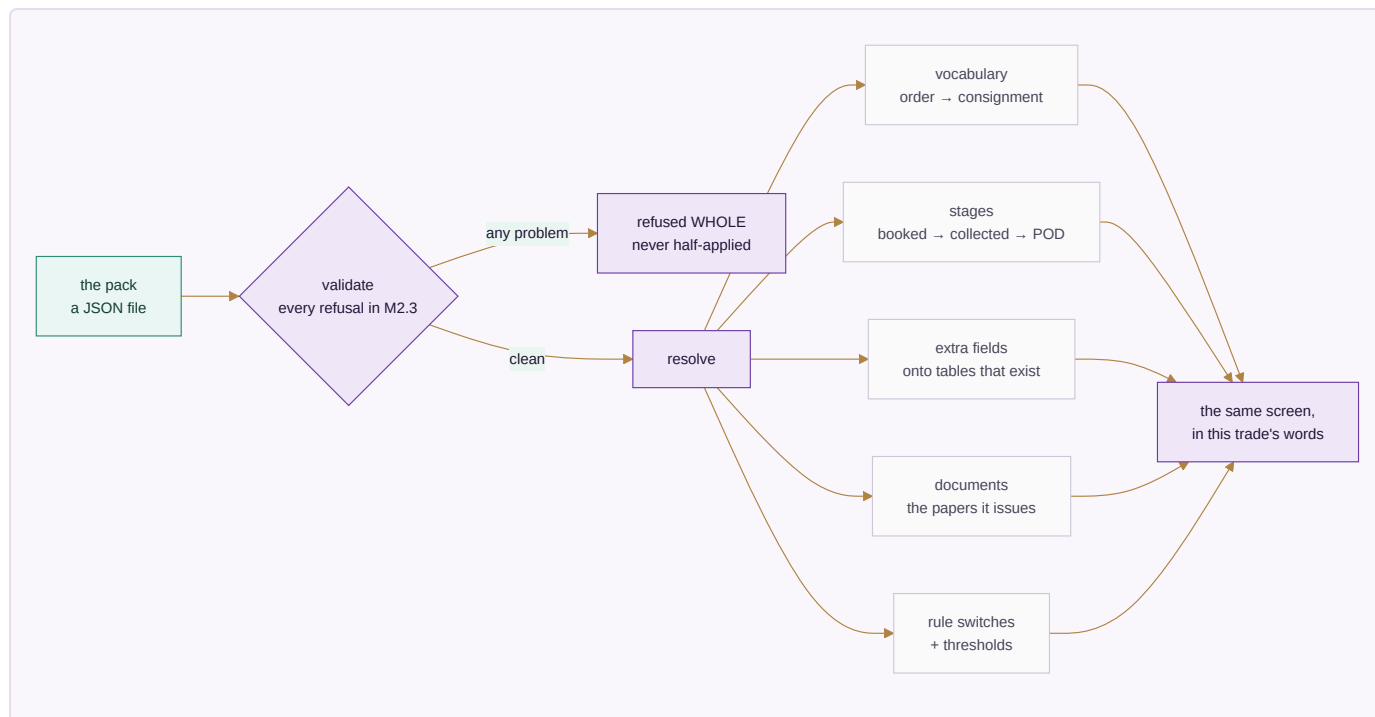
An industry pack is a row of configuration, loaded as data, that carries:

- **vocabulary** — what this trade calls an order, a customer, a unit of work, a stage
- **stages** — the pipeline a job moves through, named and ordered by the trade
- **fields** — the extra attributes this trade needs on a record, and their types
- **documents** — the papers it issues, and what has to be on them
- **rule switches** — which discretionary rules apply, and at what thresholds
- **starting data** — a chart of accounts, roles, units of measure

The engine reads a pack the way it already reads companies and channels: as rows. Nothing in `modules.js`, `core/` or the schema changes when a fourteenth trade is added — which is exactly the property the 10×10 test already proves for companies, applied to trades.

M2.2a · How a pack becomes a screen

Nothing about this is magic, and the picture is the fastest way to see that:



M2.3 • What a pack may never do

A configuration file that can do anything is not configuration, it is a hole. Six refusals are enforced in `core/packs.js` and each one is a named test in `core/tests/packs.test.js`:

A pack may not	Because
contain executable code, at any depth	the moment a pack can run code, adding a trade is a code change again and the whole guarantee is worthless
invent a concept the engine does not have	"vocabulary" would quietly become a place to put anything, and the screens would carry a name for something that does not exist
add a field to a table that does not exist	the shape of the database would move outside the reach of the schema test that guards it
declare money as a plain number	the entire schema was built to keep floating-point rupees out; a pack is not a side door
switch off an immutable rule	a trade may call an invoice a fee note; it may not decide its audit trail is optional
be applied in part	a half-loaded trade is a system whose vocabulary and rules disagree, with no way to tell which half is live

The rule ids marked immutable in `core/packs.js` cover company scoping, the audit trail, the posting rules, group elimination and roster privacy — and a test asserts that every one of them really exists in the rulebook, so the protection cannot silently guard nothing.

One default is worth stating on its own, because getting it backwards would be invisible: **a rule a pack never mentions is ON**. The rulebook is the default and a pack is an exception list, never a permission list. The other way round, every rule added after a pack was written would silently apply to nobody using it.

M2.4 • Which packs ship

Built in the order the research ranks them:

Rank	Pack	Sector	What it is really for
1	manufacturing	Manufacturing	The largest ERP user base, and the vocabulary furthest from a service firm's
2	wholesale-distribution	Distribution	Credit, not production, is the thing the software has to hold
3	retail-ecommerce	Retail	The largest WMS segment; returns and settlement are the hard part
4	professional-services	Services	No stock at all — the hardest case for the neutrality claim
5	healthcare-clinic	Healthcare	Small now, fastest-growing on every measure
6	logistics-3pl	Logistics	The most-served 3PL market, where the order <i>is</i> the product

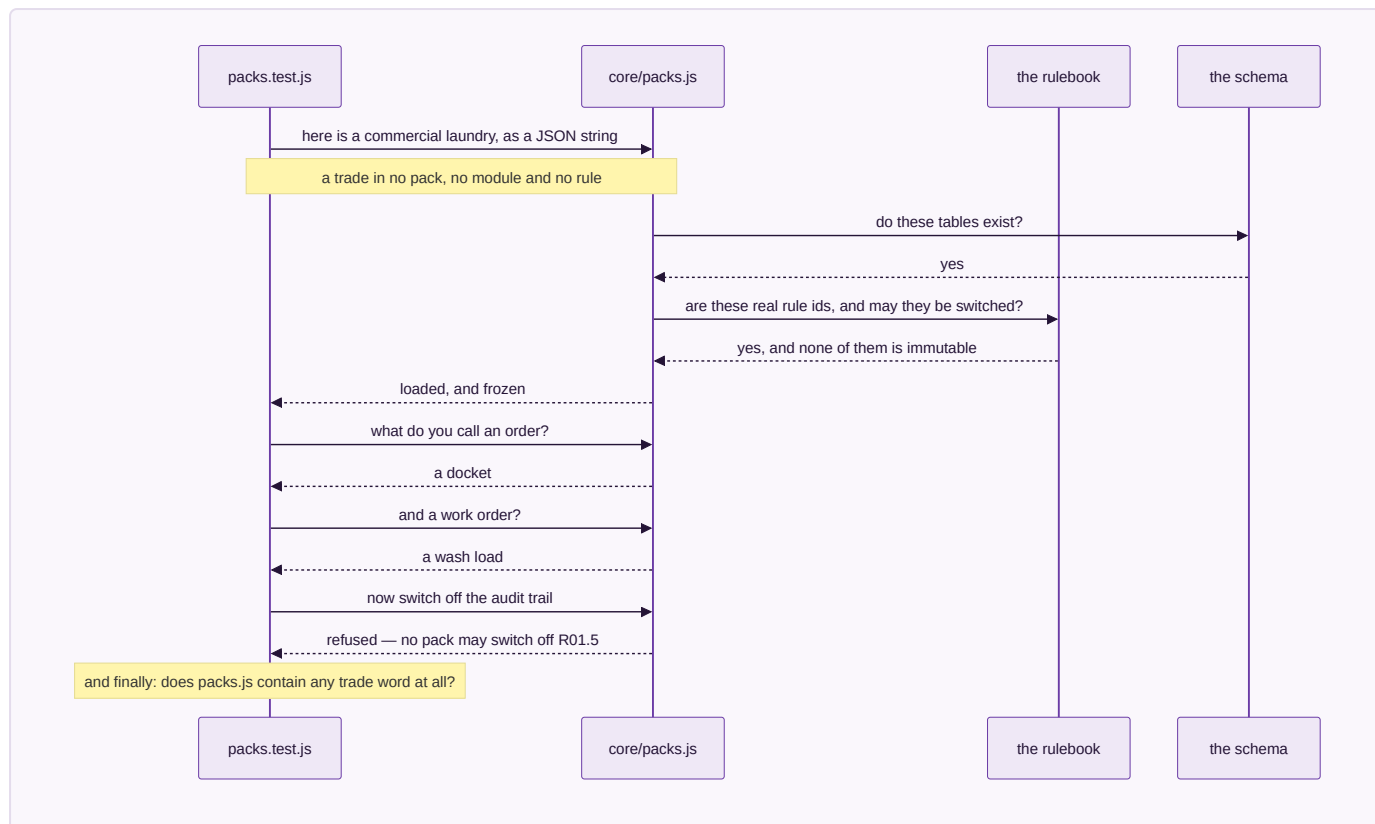
M2.5 • The gate, and what it proves

Phase 2's gate was written before the engine was: *a third trade is added without writing code*.

`core/tests/packs.test.js` §4 runs it, and runs it harder than the phase asked. During the test run it invents a **commercial laundry** — a trade that appears nowhere in this repository, in no pack, in no module, in no rule — hands the engine a JSON string, and then requires the whole system to answer in that trade's words: an order reads as a docket, a work order as a wash load, a customer as an account. Its pipeline resolves ordered and terminating, its extra fields land on real tables, its rule switches resolve against the real rulebook, and it is refused the audit trail exactly as the six shipped packs are.

A last assertion closes the loop: `core/packs.js` **may not contain a single trade word**. Not laundry, not clinic, not freight, not dealer, not matter, not patient. If the engine ever learns a trade, the test fails — because an engine that knows one trade's words has an opinion about which trades are normal, and that is the failure this whole section exists to prevent.

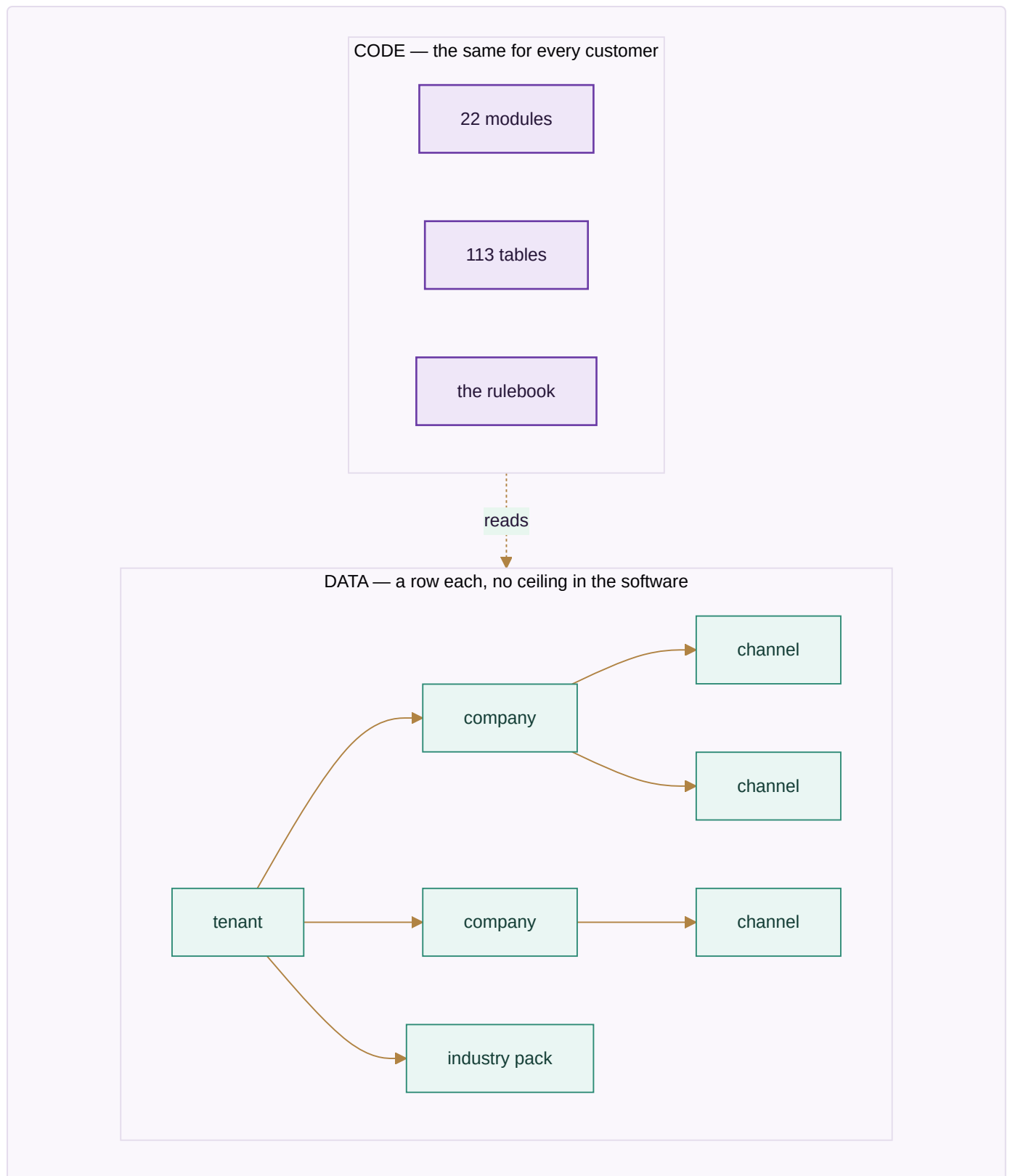
What the gate actually does, step by step:



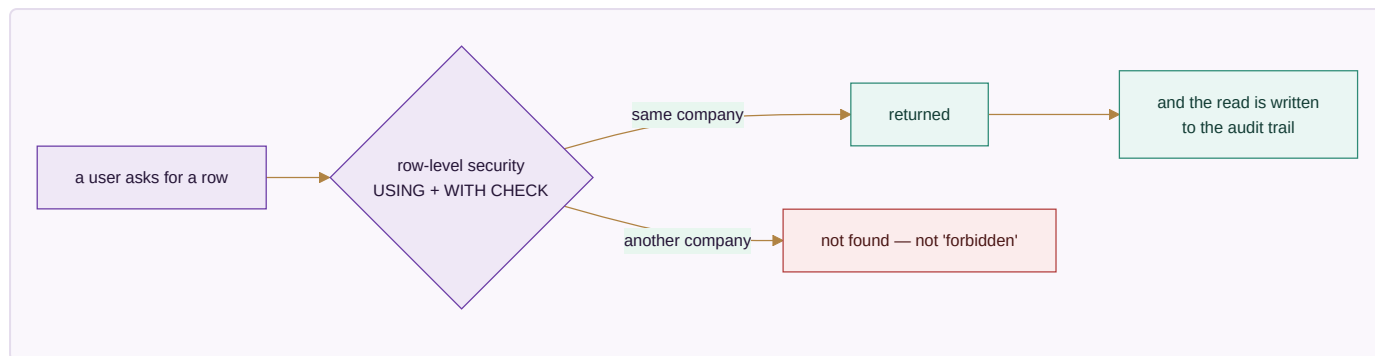
`node core/tests/packs.test.js` → **every check passes, 0 failures.**

M3 · TENANCY — WHAT IS A ROW AND WHAT IS CODE

Concept	Row or code	Consequence
Tenant (a customer of Medhava)	row	Onboarding a business is data entry, not a deployment
Company inside a tenant	row	A group with four companies is four rows, consolidated with inter-company trade eliminated
Channel	row	A new marketplace is a row; rule R15.1 makes discovery automatic
Location, stage, role	row	A warehouse, a production stage and a job title are all configuration
Industry pack	row	A trade is configuration, not a fork — <code>core/packs.js</code> , six packs shipped, a seventh added during the test run
The 22 modules	code	The structure is the product; it is the same for everyone
The rulebook	code	Which apply is configurable; what they refuse is not — and 22 of them cannot be switched off by any pack



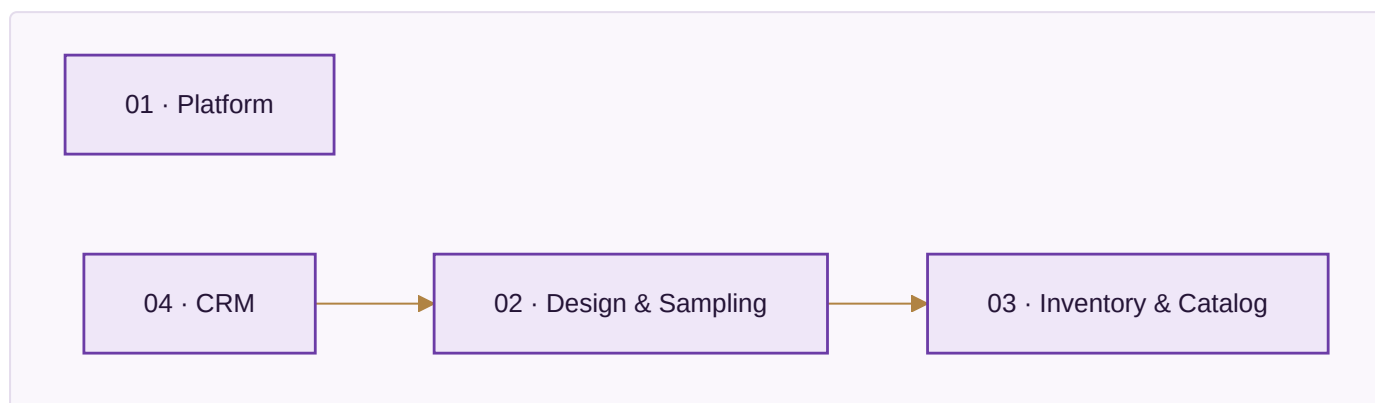
Isolation. Every business table carries `company_id` and a row-level security policy carrying both `USING` and `WITH CHECK`, so a read and a write are separately prevented from crossing a boundary. `core/tests/schema.test.js` fails the build if any company-scoped table lacks one. Cross-tenant isolation is the same mechanism one level up and is the single highest-risk item in this plan — a bug there is not a defect, it is an incident.



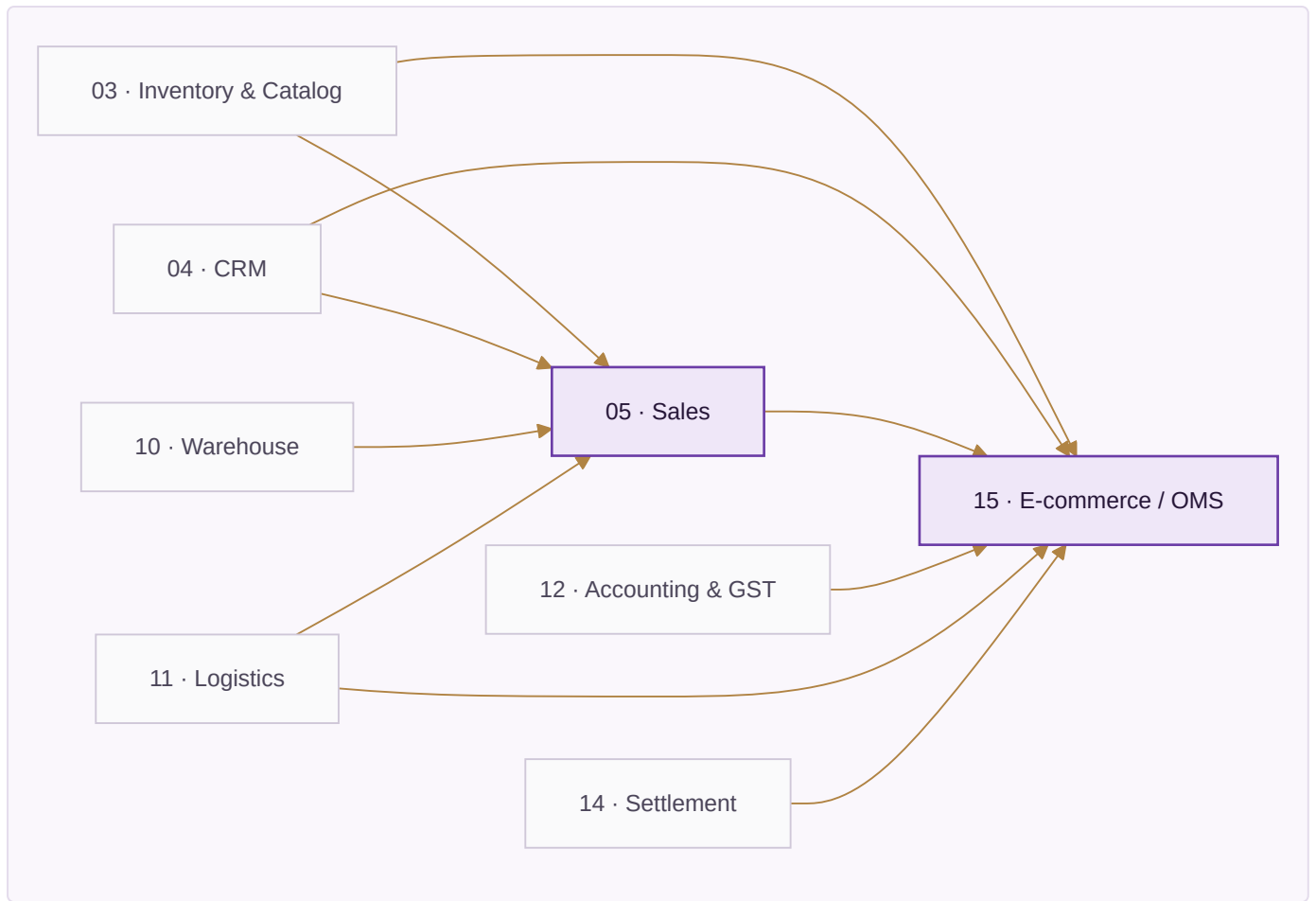
M4 • THE 22 MODULES, READ FROM TWELVE TRADES

How the modules feed each other. Generated from the `reads` field on every module in `brand/site/modules.js` by `brand/site/mkdiagrams.js` — the same field the website renders as "Reads from" — so it cannot drift from the module list. Cut into bands because all 22 modules and all 44 edges on one page rendered as an unreadable tangle; the information is the same, the page is legible.

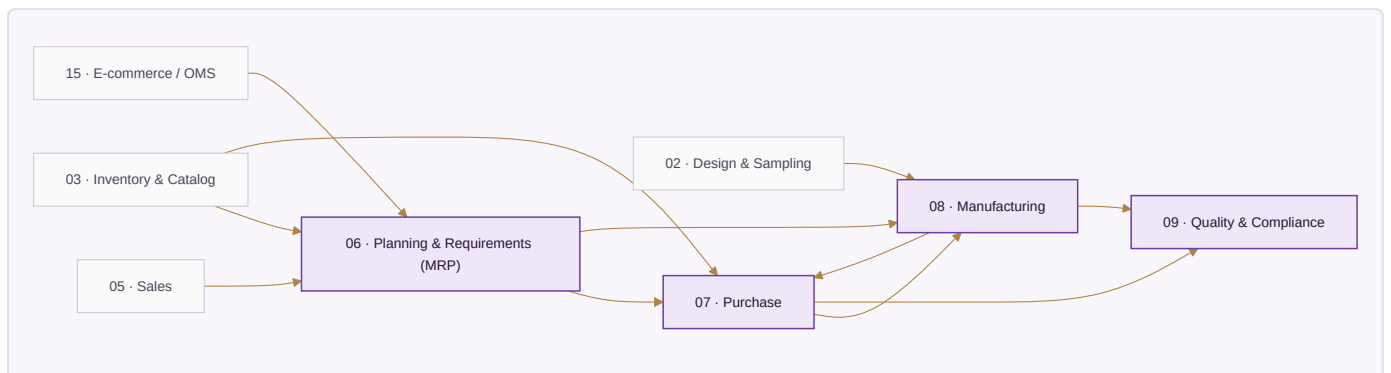
Foundation — what it reads, and from where.



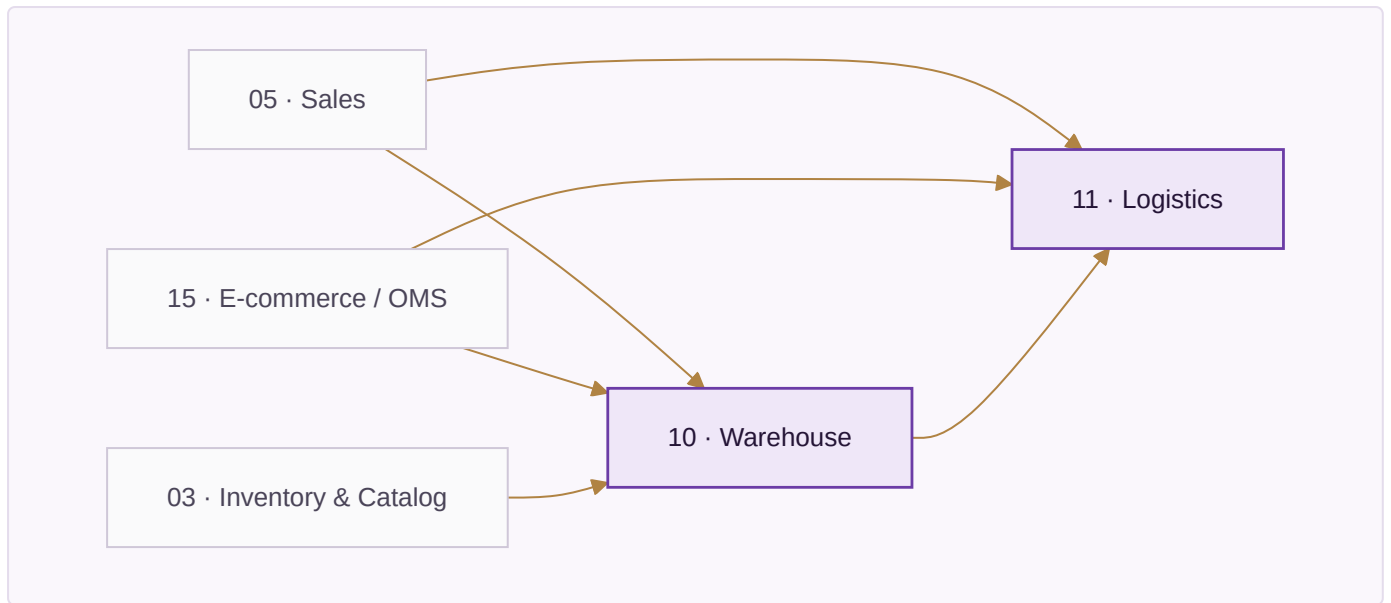
Selling — what it reads, and from where.



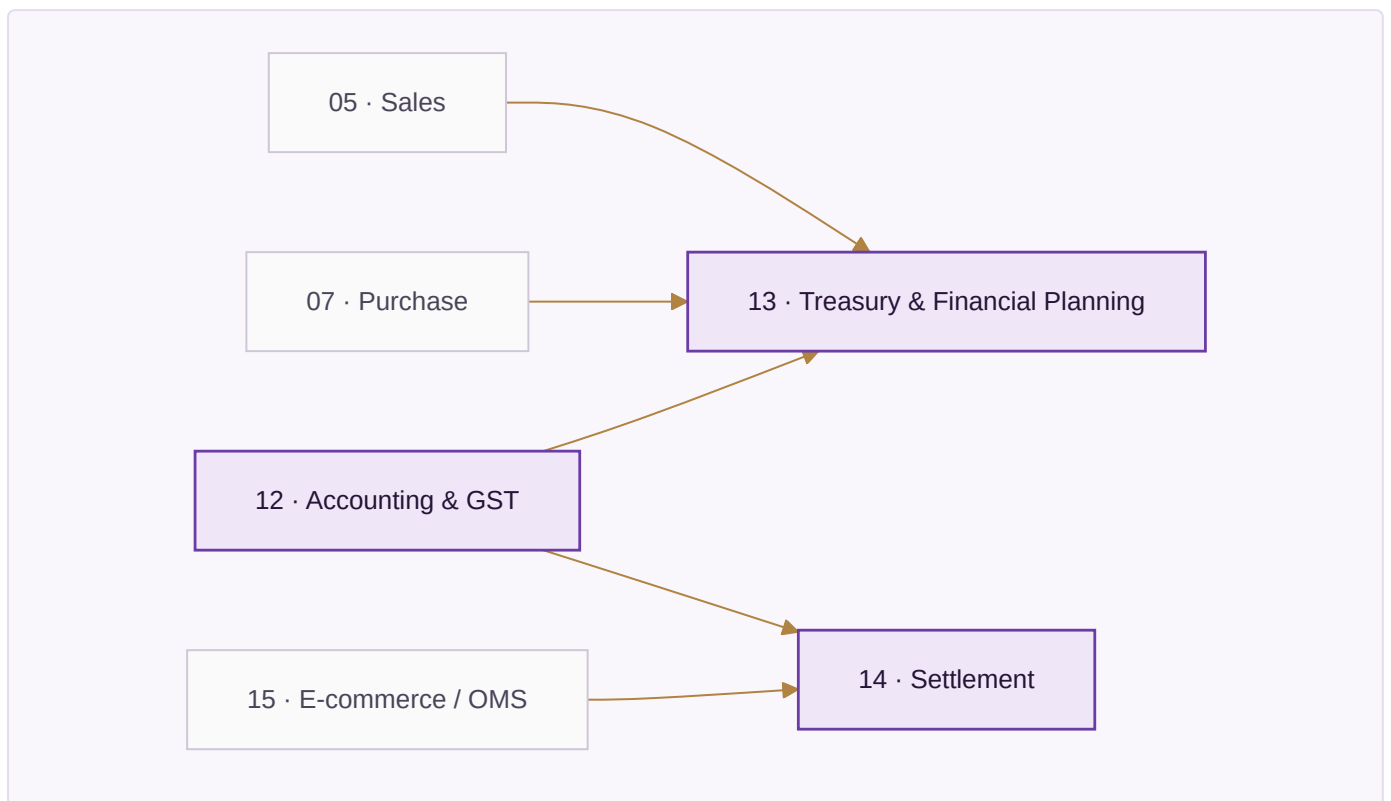
Planning & making — what it reads, and from where.



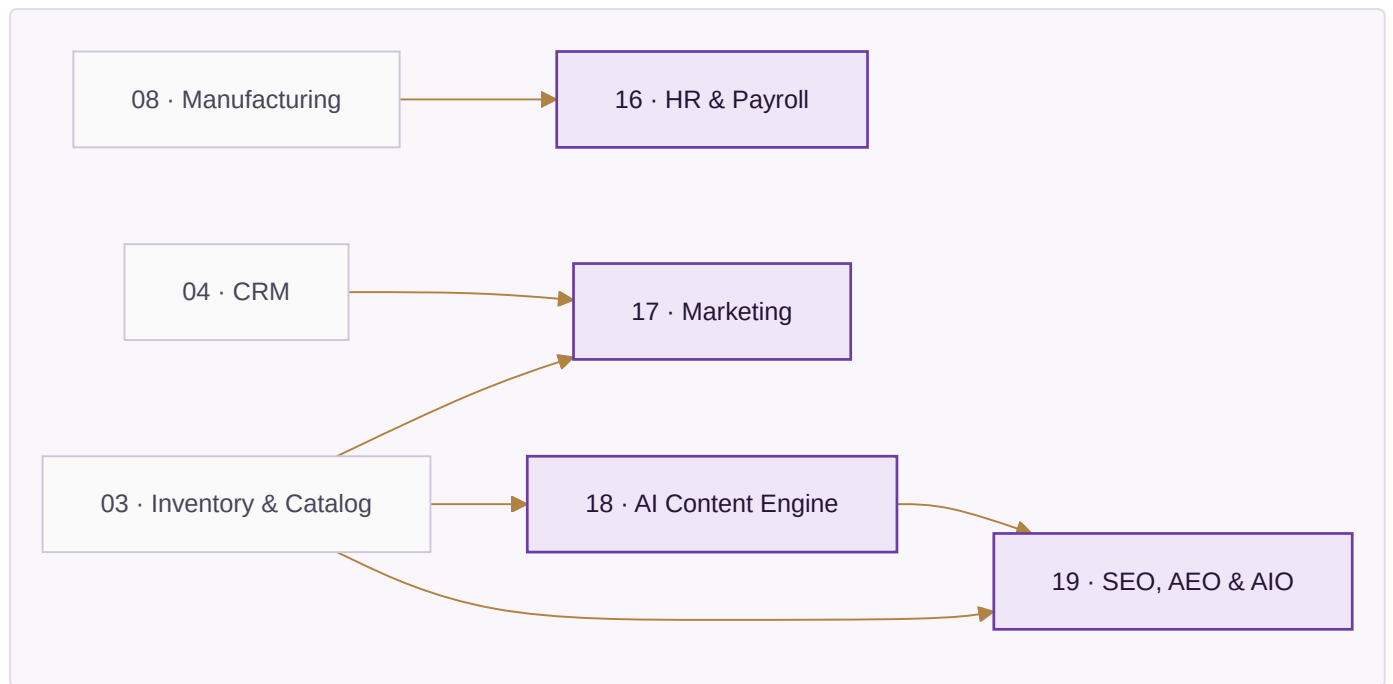
Moving it — what it reads, and from where.



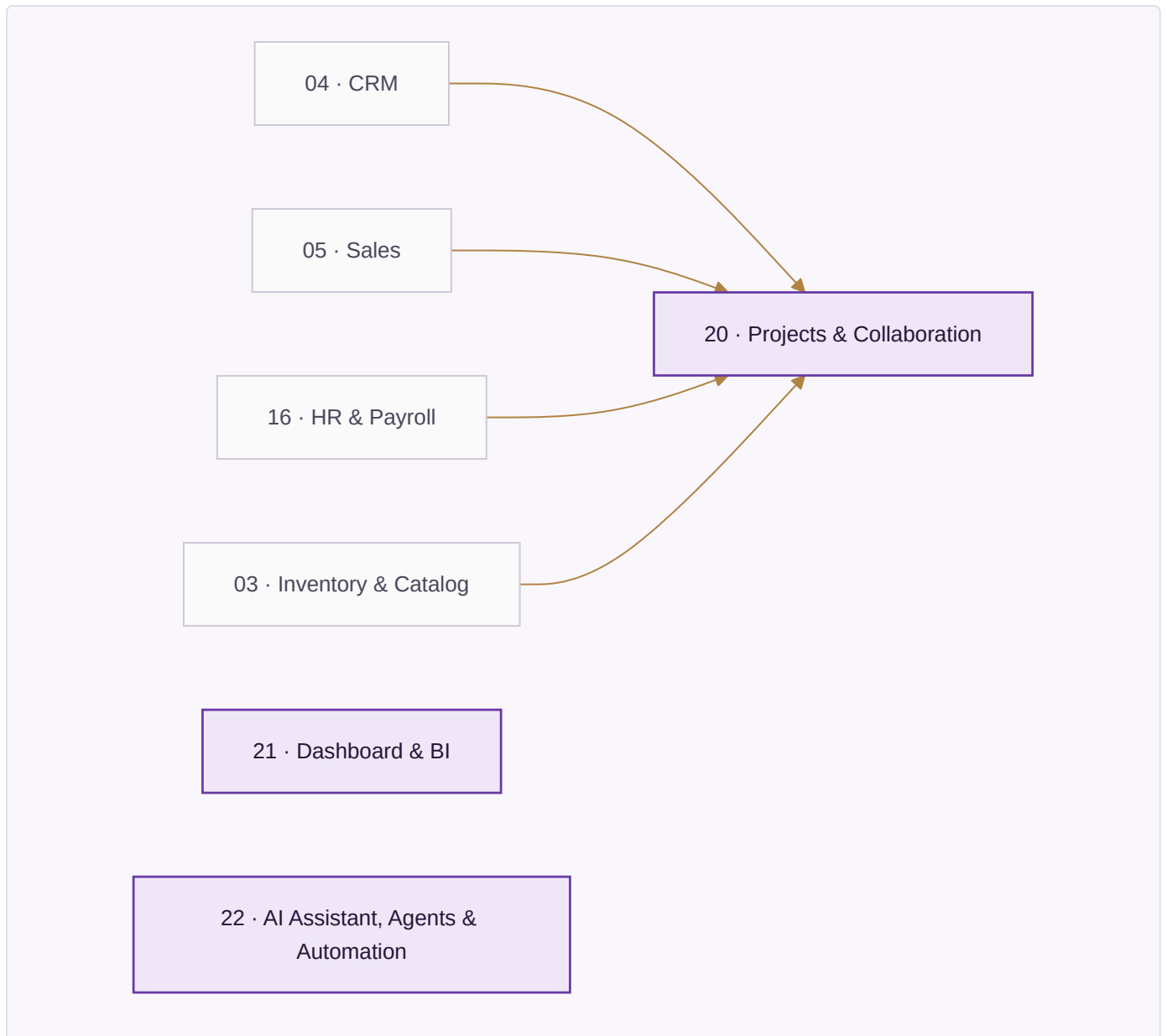
The money — what it reads, and from where.



People & demand — what it reads, and from where.



Across the business — what it reads, and from where.



01, 03, 04, 12, 21, 22 declare that they read *every module*: they sit on the shared data core rather than on any one upstream module, which is why no arrow into them is drawn above. Everything else reads exactly what the arrows show.

The module list is in `PLAN_OF_ACTION.md` Part II in full, with every app and every rule. It is not repeated here. What is here is the reading that makes it industry-neutral — the same module, seen from a different trade:

Module	Manufacturing	Professional services	Healthcare	Hospitality
02 Design & Sampling	part revision	matter scoping	protocol	recipe development
05 Sales	order book	engagement letter	appointment book	cover forecast
06 Planning	material requirement	resourcing	rota and stock	purchase plan
08 Manufacturing	work order, stages	—	—	batch and prep
09 Quality	inspection	file review	licence and calibration	food safety log
10 Warehouse	pick and pack	—	consumables	store issue
16 HR	shift and piece-rate	timesheets, chargeable	sessions and locums	rota and casuals
20 Projects	improvement work	matters	care pathways	openings

The landing page carries every one of the product screens counted at the head of this document, across all twelve sectors, doing exactly this — the same module rendered with a machine shop's figures and a clinic's figures, one under the other. That is the argument made where it can be checked rather than believed.

PART II — EXECUTION

M5 · THE STACK, AND WHAT IT COSTS

The full tools register is `brand/site/tools.js`, gated by `brand/site/checktools.js`, which fails the build if any paid tool does not name **both** the free option it replaces and the concrete condition that forces the upgrade.

"When we grow" is rejected by the checker; a number or a named event passes.

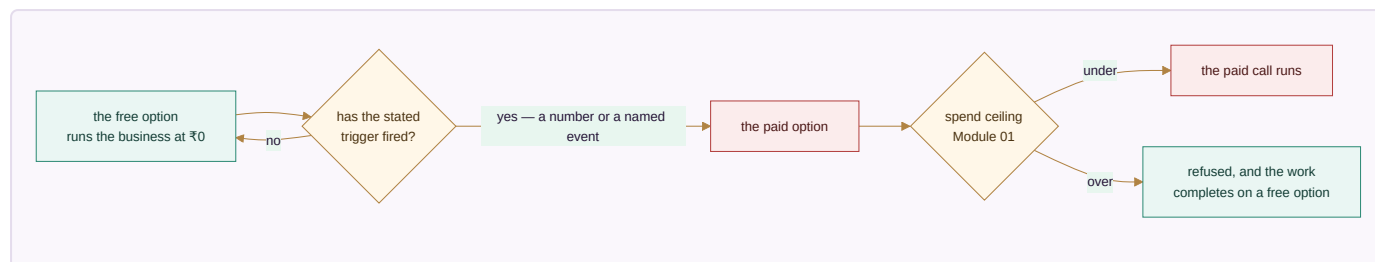
19 capabilities. Only three have no free path in existence:

Capability	Why there is no free option	What it costs
WhatsApp Business messaging	Meta requires an approved provider; there is no free tier to outgrow	~₹1,500–3,000/month plus per-conversation charges
SMS	Carrier access is the cost	~₹0.15–0.25 per message
Generated imagery	Free software, but it needs a graphics card	A GPU, or a hosted API per image

Everything else runs on free tiers or free software until a stated trigger fires: Postgres, self-hosted automation, a free hosting tier, local AI, the browser's own speech synthesis, ffmpeg and Chromium for media, UPI for payments, CSV for every integration, and a progressive web app instead of a store listing. **A business can run the whole system on zero rupees of software licence** and pay only for the three lines above, and only when it reaches them.

The Provider Router in Module 01 puts a spend ceiling in front of every paid call and refuses past it rather than warning — so the AI line in particular cannot quietly become the largest one.

How a line ever becomes a paid line. Every capability starts free and only moves when a stated condition fires — which is the whole discipline `brand/site/checktools.js` exists to enforce:



A paid entry with no free predecessor and no concrete trigger **fails the build**. "When we grow" is not a trigger; a number is.

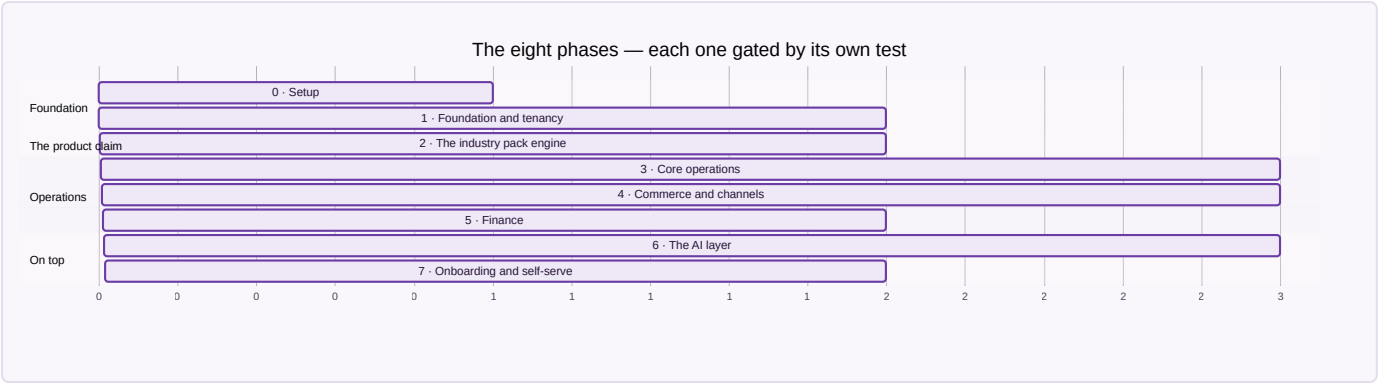
M6 · THE EIGHT PHASES

The gate is absolute: **Phase N+1 does not start until Phase N's tests pass**. A phase is done when its stated result is reproduced, not when its code is written.

Phase	What gets built	Done when
0 · Setup	Environments, CI, monitoring, the schema loaded	A commit deploys and a user logs in
1 · Foundation & tenancy	Tenants, companies, roles, RLS, masters, the SKU/item model	Two tenants exist and neither can read a single row of the other, proved by a test that tries
2 · The industry pack engine	Vocabulary, stages, fields, documents and starting data as rows	A trade nobody designed for is added without writing code , during the test run, from a plain settings file
3 · Core operations	Inventory, procurement, production, quality, warehouse	Three trades run the same operations screens on their own vocabulary
4 · Commerce & channels	Sales all channels, OMS, returns, logistics, settlement	A week of orders across channels, settled and reconciled
5 · Finance	Double-entry, tax, banking, period locks	A month closes: trial balance ties and returns generate from vouchers
6 · The AI layer	Assistant, chatbot, agents, guardrails, content	An assistant answer matches the books; an agent asked to move money stops
7 · Onboarding & self-serve	Sign-up, pack selection, import, go-live	A business onboards itself in a day without a call

Phase 2 is the one that decides whether this is a product or a project. Its gate is written before its engine — *a new trade must be addable without a developer* — and it runs on every test pass against a trade nobody designed for. Everything from Phase 3 onward stands on that claim, so it is checked rather than argued.

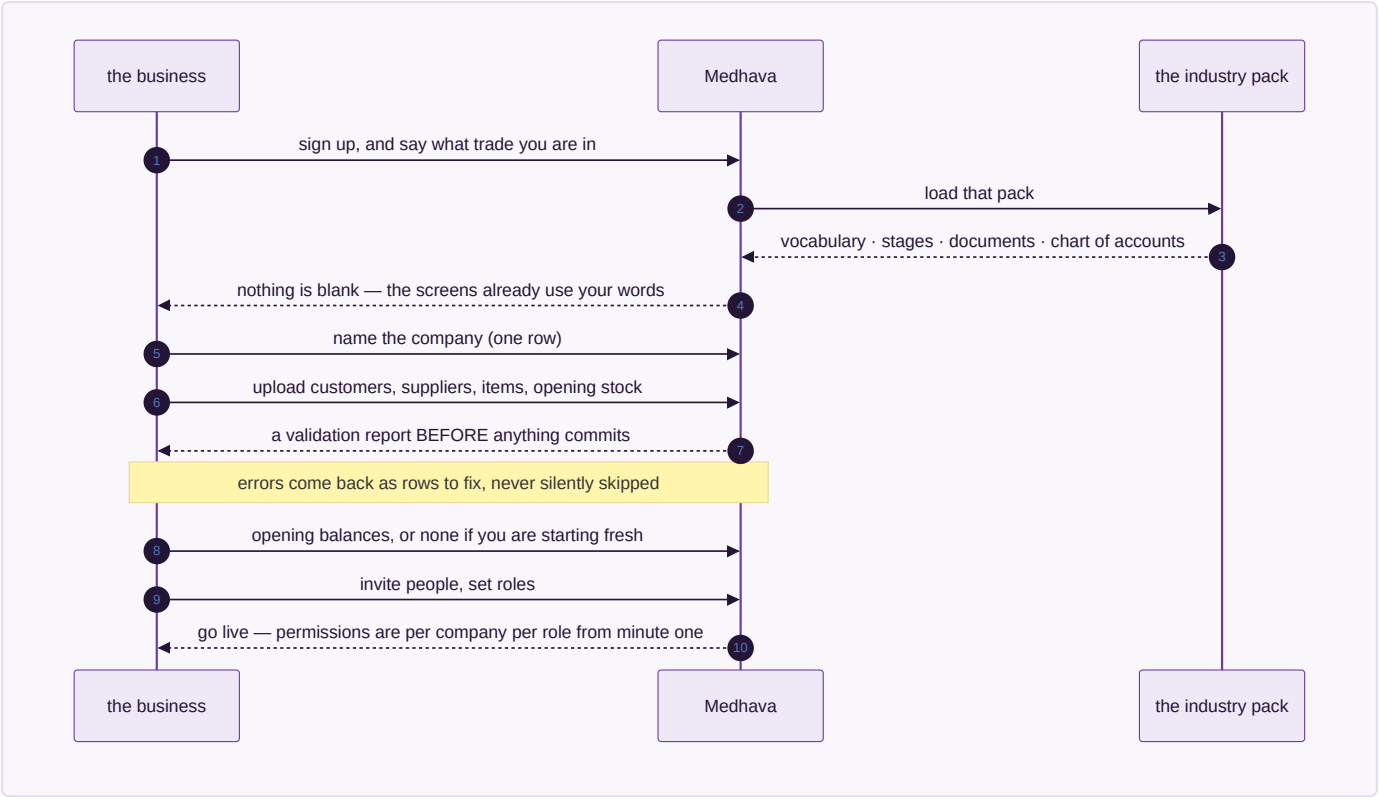
The same eight phases as a sequence:



Bars show sequence and relative size, not calendar dates — a phase ends when its test passes, and a date typed here would be a promise the gate does not make.

M7 · ONBOARDING A BUSINESS IN A DAY

For a business with no system to migrate from — which is most of the target market — the sixty-day parallel run in the Vastrangam plan is the wrong shape entirely. The sequence is:



1. **Sign up, pick a trade.** The industry pack loads vocabulary, stages, documents and a chart of accounts. Nothing is blank.
2. **Name the company.** One row. A group adds more rows now or later.
3. **Import what exists** — customers, suppliers, items, opening stock — from a spreadsheet, with a validation report **before** anything commits. Errors are shown as rows to fix, never silently skipped.
4. **Opening balances**, or none at all if the business is starting fresh.

5. **Invite people, set roles.** Permissions are per company per role from the first minute.
6. **Do one real transaction end to end** — one sale, invoiced, stock moved, posted — and click the dashboard figure down to the voucher. That is the go-live test, and it takes a minute.

For a business migrating from an existing system, the discipline in `PLAN_OF_ACTION.md` Part IV E5 applies unchanged: export, clean, load, opening balances, parallel run, and a decommission criterion agreed in advance rather than argued at the end.

M8 · SECURITY AND COMPLIANCE

Row-level security in the database and permission checks in API middleware — one layer is one mistake away from a cross-tenant read. Personal data is exportable and erasable on request, with consent and retention tracked as the two separate clocks they are. Card data never reaches this system; the gateway’s own secured field takes it, so there is no scope to protect. The audit trail has no off switch, and R16.22 keeps individual pay and personal attributes out of any document that leaves the building.

M9 · PERFORMANCE

Operation	p95
Page load, cached / uncached	< 1 s / < 3 s
Live stock or availability query	< 500 ms
Document PDF	< 2 s
Channel order pull, per channel	< 60 s
Settlement import, 1,000 lines	< 30 s
Daily profit-and-loss refresh	< 10 s

The availability query is the one that decides whether people trust the system: it runs on every screen showing a quantity or a free slot, and one that takes two seconds is one people stop asking.

M10 · RISK REGISTER

Risk	Likelihood	Impact	Mitigation
Cross-tenant data leak	Low	Critical	RLS with USING and WITH CHECK, middleware checks, a test that actively attempts a cross-tenant read
The industry pack engine slips, and trades get hard-coded instead	High	Critical	Phase 2 gate: a third trade added with no code. Hold the gate
A vertical demands a genuine structural exception	Medium	High	Add it as a module or an app for everyone, never as a branch for one customer
Free tier terms change or a tier disappears	Medium	Medium	Every capability has a self-host route recorded; the register is dated
WhatsApp provider outage	Low	High	Adapter swap; SMS and in-app as fallback
Onboarding needs hand-holding, so it does not scale	High	Medium	The day-one sequence above is a tested path, not a document
AI spend runs away	Medium	Medium	The spend ceiling refuses rather than warns

M11 · SUCCESS METRICS

Measure	Target
A new trade supported without writing code	Yes, from Phase 2 onward — binary, not a percentage
Time for a business to onboard itself	Under a day, unattended
Cost to run for a business at pilot scale	₹0 in software licence
Rules enforced by a test	the enforced-of-total figure at the head of this document; up every build
Cross-tenant incidents	Zero, and this is the only metric with no acceptable non-zero value

M12 · RUNBOOKS

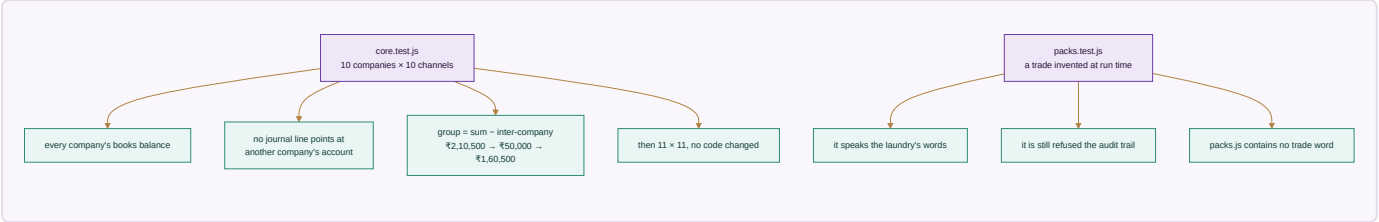
Daily — error and uptime check, failed integrations queue, onboarding funnel. **Weekly** — usage per tenant, slow queries, support themes that suggest a missing rule. **Monthly** — free-tier headroom against triggers, spend against ceilings, backup restore test. **Quarterly** — re-read the tools register against current published tiers, review which SPECIFIED rules became enforceable, and check whether any vertical has quietly grown an exception.

M13 · WHAT A CUSTOMER PAYS FOR

The free-first register is not only how Medhava is built — it is the shape of what it can offer. Because the system runs on free tiers and free software at pilot scale, a business can be given a genuinely working system before it pays anything, and the paid lines are the ones it would have had to pay anyway: messaging, marketplace access, payment processing, courier pickup.

The commercial decision — subscription, per-company, per-user, or usage — is not made in this document, because it is a business decision rather than an engineering one. What this document fixes is the constraint it must respect: **nothing in the architecture may make a plan cap technically necessary**. Companies, channels, users and trades are rows. If a plan limits them, that is a commercial choice stated in the pricing, and the software says so rather than pretending to a technical limit it does not have.

PART III — THE PROOF



The test of whether this is one system or twenty-two programs sharing a login is a single transaction followed end to end, and it is re-run at every module boundary. That walkthrough is in `PLAN_OF_ACTION.md` Part V and is identical here.

The test of whether it is *industry-neutral* is different, and it is this: **take the same transaction and run it in a trade nobody had in mind when the code was written**. That test used to be run by hand, by writing an overlay. It is now run by the machine: `core/tests/packs.test.js` invents a commercial laundry every time it executes, loads it from a JSON string, and requires the system to answer in that trade's words — while still refusing it the audit trail. The last assertion in that file is that `core/packs.js` contains no trade word at all, so the engine cannot quietly learn one trade and call it neutrality.

Every check passes, with the shipped packs and a seventh added at run time. That is the whole industry-neutrality claim, reduced to something that either passes or does not.

Counts in this document are read from `brand/site/modules.js`, `brand/site/rules.js`, `brand/site/shots.js`, `brand/site/tools.js` and `core/schema.postgres.sql` when it is generated. Product screens carry illustrative figures and are labelled with the trade they are drawn from; they are not case studies, and no business is named as a customer that is not one.

PART THREE — HOW IT IS BUILT

How this platform is designed and built.

14 parts · 49 decisions · 19 technical layers · compiled 2026-08-24

What this document is

This describes a design. Nothing in it exists yet, and nothing in it claims to. Every part is a decision to be made and built. Where it says *done when*, that means the decision is made, written down and proven by a test — not that something is running.

It is written for whoever builds the platform. A business using the platform installs nothing and needs none of this; onboarding one is a separate document written for a reader with no terminal.

Every technical word is explained the first time it appears, in plain language, with an everyday comparison where one helps. No prior knowledge is assumed anywhere in this document.

The two rules everything obeys

1 · No capability depends on one tool. Every one of the 19 layers names what it is built on, **57 named replacements** between them, and the interface the rest of the code talks to. That last part is what makes switching a settings change rather than a rewrite. A check refuses any layer with fewer than two alternatives or no interface, so the rule cannot quietly rot into a paragraph nobody kept.

2 · Nothing is static, and the past stays correct. A customer can add, edit or remove anything at any time, and it takes effect at once. Every change carries the date it starts from and who made it, and is added rather than written over. So a supervisor can leave on Tuesday and a replacement start on Wednesday, changed the same morning — and last month's payroll, already paid, does not move by a rupee. *Purana record mitta nahin; naye date se naya rule lagta hai.*

Part 13 lists all 18 things a customer can change, and the 6 that can never be switched off.

Part 0 · What you are building

One piece of software that many separate businesses use at the same time, each seeing only its own information, each seeing it in its own words.

The businesses will not resemble each other. A steel plant, a clothing manufacturer, a car maker, a retail chain, an education company, a single creator selling courses — all of them, on the same code. That is the whole design problem, and every decision in this document exists to serve it.

The trap to avoid is building one system and then bending it. The moment a customer needs a change and the answer is "we will add a setting for you", the software has started to fork, and in two years there are as many versions as customers. The way out is to decide early that the things that differ between businesses are **data**, not code — their words, their steps, their extra fields, their documents, which parts they use at all — and that the code is the same for everyone, forever.

platform — One piece of software that many separate businesses use at the same time, each seeing only its own information. *Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.* **tenant** — One business using the platform. Its people, its data and its settings are its own. *Us building mein ek office. Aapka office, aapka saamaan, aapka taala.*

module — One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together. *Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.*

0.1 · Write down what is code and what is data, before writing any code

This is the most expensive decision in the project and the cheapest one to get right at the start. Anything on the "data" side can be changed by a customer, in the app, in a minute. Anything on the "code" side needs a developer and a release. Put something on the wrong side and you either ship a rigid product or an unmaintainable one.

This is data — the customer changes it	This is code — the same for everyone
What they call things	That records have to name their owner
The steps their work moves through	That money is exact
Extra fields on any record	That every change is recorded
Which modules they use	How the modules work
Their companies, channels, locations	That one business cannot read another
Their documents and numbering	The rulebook the books rely on
Which outside services they connect	The shape of the connection

Done when: The two lists exist and the team agrees on them. Every later argument about a feature starts by asking which column it belongs in.

0.2 · Adopt the two rules that everything else obeys

Both exist because of the same fear: that in three years you cannot change something you need to change. One is about the tools underneath you. The other is about the business on top.

Rule	What it means in practice
No capability depends on one tool	Every layer names one default so work can start, at least two replacements, and the interface the rest of the code talks to. Swapping is a settings change, never a rewrite.
Nothing is static, and the past stays correct	A customer can add, edit or remove anything, any time, taking effect at once. Every change carries the date it starts from — so last month's figures do not move.

The second rule is the harder one and it is worth being blunt about why. A system that lets you overwrite freely will happily change a payroll total for a month you already paid out. A system that locks the past makes you phone a developer when a supervisor quits on a Tuesday. The effective date is what gives you both: purana record mita nahin, naye date se naya rule lagta hai.

Done when: Both rules are written into the project's working agreement, and there is a check that fails the build when either is broken.

0.3 · Decide the shape: one code base, many businesses, one database

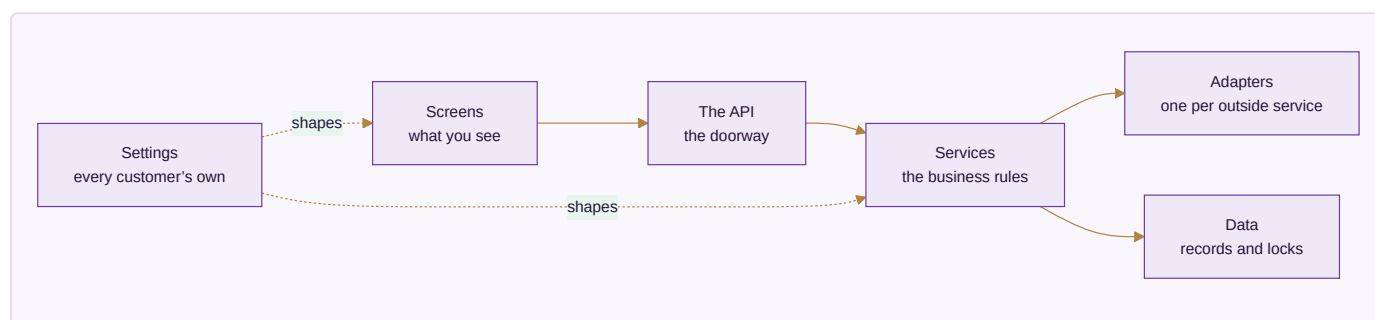
row-level security — A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug. *Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.* **database** — Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost. *Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.*

Three ways exist to serve many businesses. Give each its own copy of everything — simple at three customers, unmanageable at fifty, because every fix has to be applied fifty times. Give each its own database — safer-feeling, but a change to the shape of the data has to run everywhere and one of them will fail while the others succeed. Or keep everyone in one database with a lock at the record level, which is one system to fix, one shape to change, and one thing that must be got exactly right.

Done when: The choice is written down with its consequence stated: the record-level lock is now the single most important piece of code in the system, and it is tested before anything is built on top of it.

Part 1 · The shape of the whole thing

Before any single piece, the map. Six layers, each talking only to the one below it, so a change in one does not ripple through the rest.



1.1 · Separate the six layers and keep them separate

frontend — The part you see and click — the screens, the buttons, the forms. *Hotel ka dining hall aur menu card. Jo aapke saamne hai.* **backend** — The part of the software you never see, which does the actual work — checks the rules, saves the records, calculates the totals. *Hotel ka kitchen. Customer nahin dekhta, par khaana wahin banta hai.* **API** — The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer. *Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.* **adapter** — A small piece of code that translates between the system and one outside service, so the rest of the system never has to know which service is in use. *Travel adapter. Andar ka appliance wahi, bas plug ko us desk ke socket ke hisaab se badal diya.*

The reason for layers is not tidiness. It is that a layer with a clear edge can be replaced without touching anything else, and a layer whose edges have blurred cannot be replaced at all. Most systems that become impossible to

change did not decide to be — they just let the screens start talking directly to the database, one shortcut at a time.

Layer	What lives there	What it must never do
Screens	What the user sees and clicks	Contain a business rule, or reach the database directly
The API	The doorway the screens knock on	Decide anything — it only carries requests
Services	The business rules. The real system	Know which outside company provides anything
Adapters	One per outside service	Contain a business rule
Data	The records, and the locks on them	Trust the layers above it
Settings	Every customer's own configuration	Ever require a release to change

Done when: The layer boundaries are agreed and there is a check that fails when the code of one layer mentions another it should not know about.

1.2 · Put every business rule in one place, away from everything replaceable

The rules are the only part of this system that is genuinely yours. Frameworks change, databases get swapped, the screen library goes out of fashion. If the rule that says a dispatch cannot exceed what was ordered lives inside a screen or inside a database feature, it dies with that thing. Written as plain functions that take values and return decisions, it outlives all of them — and it can be tested without starting a database or opening a browser.

Done when: A rule can be tested by calling it directly, with no database, no browser and no network. If a test for a rule needs any of those, the rule is in the wrong place.

1.3 · Forbid any outside company's code inside the business rules

provider — A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery. *Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.*

interface — A written promise about what a part of the system does, without saying which product does it — so the product underneath can be swapped without anything above noticing. *Bijli ka socket. Socket ka size fix hai; usme koi bhi company ka plug lag jaata hai.*

The instant a service calls a payment provider or a messaging provider directly, that provider is welded into your system. Every alternative listed in any document becomes decorative, because reaching it means finding and rewriting every mention. The adapter layer exists precisely to hold that damage in one small, replaceable place.

Done when: A search for any provider's name outside the adapters folder returns nothing, and that search runs automatically on every change.

Part 2 · The database — where everything is kept

The most important layer, and the one where mistakes are least recoverable. A wrong screen is a bad afternoon; a wrong data shape is a year of workarounds.

schema — The written plan of what information the system keeps and how the pieces connect. *Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.*

The database — Keeps every record — customers, orders, stock, vouchers — and answers questions about them.

PostgreSQL is open source, runs anywhere, and has the two things this design needs built in: locks at the record level so one business cannot read another's rows, and exact whole-number arithmetic so money never drifts. Any managed Postgres service is a hosting decision, not a database decision — the same schema runs on all of them.

This layer	The database
Built on	PostgreSQL
Can be replaced with	A managed Postgres service — same database, somebody else runs the machine Postgres on your own server — the software is free, you supply the machine MySQL or MariaDB, if a team already knows them well — the schema needs rework for the record-level locks
Everything talks to	<code>DatabaseService</code>
Switching costs	Moving between Postgres hosts is a dump and a restore. Moving off Postgres entirely means rewriting the isolation layer, which is the one part worth not moving.

2.1 · Build the lock between businesses first, before anything else

table — One kind of information inside the database — all your customers in one, all your orders in another. *Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.* **row** — One single record — one customer, one order, one payment. *Register mein ek line. Ek line matlab ek entry.*

Everything else in this document assumes it. If it is added later, every table you have created by then has to be revisited, and the one that gets missed is the one that leaks. It also has to live in the database rather than only in the application, because the application will one day have a bug and the lock has to survive it.

Careful. Test the case where no business is selected at all. Depending on how the setting is read, that either refuses or quietly returns **everything** — and the second one is a silent, total leak that every other test would pass straight over.

Done when: A test creates two businesses with real records, asks for the other one's record by its exact identifier, and gets nothing back. The same test, run with the lock removed, fails — because a test that has never failed has not been shown to test anything.

2.2 · Give every business record the same standard columns

audit trail — *An automatic record of every change — what changed, who changed it, and when. Har entry ke saath naam aur time apne aap likha jaata hai. Baad mein koi bole "maine nahin kiya", toh register bata deta hai.*

Repetitive on purpose. Every business table carries the same handful of columns for who owns the record, when it was made, who made it, when it last changed, and whether it has been ended. Doing this everywhere means every feature that depends on them — history, undo, audit, reporting — works everywhere, instead of working on the tables somebody remembered.

Column	What it is for
identifier	Names this one record, unique across the whole system
company	Which of the customer's companies it belongs to
created at / created by	When, and by whom
updated at / updated by	The same for the last change
ended at	Set when a record stops applying. Never deleted
version	Stops two people silently overwriting each other

Done when: A check reads the schema and fails if any business table is missing one of these.

2.3 · Store money as whole units, never as a decimal

integer paise — *Money stored as a whole number of paise instead of a decimal, so amounts are exact and rounding can never quietly lose a rupee. Paisa hamesha poore paise mein ginte hain, aadha-adhoora kabhi nahin — isliye hisaab kabhi ek rupya idhar-udhar nahin hota.*

Decimal arithmetic on money loses fractions in ways nobody can trace. Every amount is a whole number of paise, and every column carrying money says so in its name so a value can never be read as rupees by mistake. Converting for a report is a division by a hundred of an exact number — there is no rounding decision left to get wrong.

Done when: A check fails if any money column is a decimal type, and the arithmetic is proven with a test that would fail under decimals.

2.4 · Make every changeable value effective-dated, and append-only

effective date — *The date a change starts applying from. Records made before it keep the old value; records after it use the new one. Naya rate 1 tarikh se lagu. Purane mahine ka bill purane rate se hi banega — woh apne aap nahin badlega.*

This is the mechanism that gives a customer complete freedom without breaking their history. A rate, a role, a person's position, a tax percentage — none is a single value. Each is a list of values with the date each started applying. Asking "what was the rate on the 3rd of last month" is then an ordinary question with an exact answer, rather than an archaeology project.

Column	What it is for
what	The thing being set — a rate, a role, a position
who it applies to	The person, the item, the company
value	What it became
from date	When it started applying
to date	Empty means still in force
changed by	The person who made the change

Two rows covering the same date for the same thing is a data error, not a preference. The check for it runs on write, because by the time it shows up in a report the wrong number has already been paid to somebody.

Done when: A report for a past month is run twice — once before a rate change and once after — and returns the identical figure both times.

2.5 · Record every change automatically, with no way to switch it off

Not a feature — a foundation. A dispute about what a figure was six months ago is answered by the record or it is not answered at all. Because it cannot be disabled, nobody has to remember to enable it, and no configuration mistake can quietly remove it.

Done when: Changing any record writes a history entry naming what changed, from what, to what, by whom and when — and there is no setting anywhere that stops it.

Part 3 · The backend — where the work actually happens

The part nobody sees, which does everything that matters: checks the rules, saves the records, calculates the totals, and refuses what should be refused.

The backend runtime — Runs the business rules, checks permissions, writes records and calculates totals.

The same language runs on the browser side, so one team can work across the whole system and code that validates a form can be shared with the code that validates the saved record — no rule gets written twice and no two versions of it drift apart.

This layer	The backend runtime
Built on	Node.js with TypeScript
Can be replaced with	Any container host — the code is ordinary and carries no host-specific parts
	Python or Go for a service that genuinely suits them, talking over the same API
	A different Node framework — the business logic sits outside the framework on purpose
Everything talks to	the HTTP API contract
Switching costs	Low, because the rules live in plain functions rather than inside a framework. Moving a service means moving the functions and putting a different door in front of them.

3.1 · Organise the backend by what it does, not by what technology it uses

Group the code by business area — sales, stock, payroll, accounts — rather than by technical type. A person fixing how a discount works then opens one folder instead of five, and a whole area can be lifted into its own service later without unpicking it from everything else.

Done when: Someone new can find where a business rule lives from the name of the business area alone, without being told.

3.2 · Make one action do all of its consequences, or none of them

A sale reduces stock, raises an invoice, posts to the ledger and updates what the customer owes. If three of those succeed and one fails, the books are wrong and nobody knows. All of it happens together or none of it does — and the middle state never exists, even for a moment, even if the machine loses power in between.

Done when: A test interrupts an action half way through and confirms the records are exactly as they were before it started.

3.3 · Design the API so a screen never decides anything

Screens exist on phones, on laptops, and eventually in places nobody planned for. Every one of them must reach the same rules. The moment a screen calculates a total or decides whether an approval is needed, that logic has to be repeated in the next screen — and the two will disagree.

Done when: Every calculation and every permission decision can be reproduced by calling the API directly, with no screen involved.

3.4 · Make the API answer honestly when something is refused

A refusal is information. "Not allowed" tells a user nothing and generates a support call; "this dispatch is 12 more than the order allows" tells them what to do. And a refusal caused by somebody else's change must say so, rather than looking like their own mistake.

Done when: Every refusal names what was refused and why, in words a user can act on without phoning anyone.

Part 4 · The frontend — the screens, drawn from settings

The single idea that makes one system serve every industry: **screens are described as data, not written one by one**. A screen definition says which fields, in what order, with what labels, under what conditions. Change the definition and the screen changes — no code, no release.

The frontend — Everything a person sees and clicks — screens, forms, tables, dashboards.

Screens are drawn FROM SETTINGS rather than written one by one. A tenant that renames a field, adds a column or turns a module off gets a different screen with no new code written — which is the only way one system can serve a steel plant and a single creator without becoming two systems.

This layer	The frontend
Built on	React with TypeScript, screens generated from configuration
Can be replaced with	Vue or Svelte — the screen definitions are plain data and do not care what draws them
	Server-rendered pages where speed on a weak connection matters more than interaction
	A native mobile shell reading the same screen definitions
Everything talks to	the screen definition format
Switching costs	Moderate, and bounded: what a screen contains is data, so a rewrite replaces the painter, not the paintings.

4.1 · Describe screens as settings rather than building them individually

Hand-built screens are the reason most business software cannot be customised. Every customer request becomes a code change, and the code grows a branch for each customer until nobody can safely change anything. If the screen is a description, a customer adding a field is a new line in their own settings — and it affects nobody else at all.

A screen definition says	So a customer can
Which fields appear, and in what order	Hide what they do not use, promote what they do
What each field is called	Use their own trade's words
Which are required	Enforce their own discipline
Which extra fields they added	Record what only they need
What the columns and filters are	See their work the way they think about it
Which actions the buttons offer	Match their own process

Done when: Adding a field to a screen for one customer is done in the app, takes effect at once, and changes nothing for any other customer.

4.2 · Build one design system and use it everywhere

Every screen drawn from the same set of parts means the system feels like one product rather than twenty. It also means an improvement to a table — better sorting, better behaviour on a phone — arrives everywhere at once instead of being reimplemented per screen.

Done when: A new screen can be assembled from existing parts without writing new visual code.

4.3 · Design for a bad connection and a small screen first

The people entering most of the data are not at a desk. They are on a shop floor, in a godown, on a site, on a phone, on a connection that comes and goes. A screen that only works on a fast laptop connection is a screen that does not get used, and the data it should have captured gets written on paper instead.

Done when: Every screen that captures data is usable one-handed on a phone, and says clearly what happened if the connection dropped mid-save.

4.4 · Let a customer turn whole modules on and off

A creator selling courses has no godown. A steel plant has no reels to publish. Showing everybody every module makes the product look bloated to all of them and correct for none. The menu is a setting, so each business ends up with a system that looks built for it.

Done when: Turning a module off removes it from the menu and keeps every record it ever held — tidying a menu never destroys data.

Part 5 · Storage and memory — files, speed, and what is remembered

Three different things that get confused with each other: where files live, what is kept handy for speed, and what the assistant is allowed to know.

storage — Where files are kept — photographs, invoices, scanned documents. Different from the database, which keeps information rather than files. *Almari ke bagal wala godown. Register almari mein, par bade dabbe aur photo godown mein.* **cache** — A small, fast copy of information that was just looked up, kept ready in case it is asked for again. *Counter pe rakha hua sabse zyada bikne wala saamaan. Har baar godown tak jaana nahin padta.*

File storage — Keeps photographs, invoices and scanned documents — the things too big to sit in the database.

Almost every file service speaks the same request format, so one adapter reaches most of them. That makes this the cheapest layer in the whole system to change your mind about.

This layer	File storage
Built on	Any S3-compatible object store
Can be replaced with	A different S3-compatible provider — usually a URL and a key change
	Files on your own server's disk, with a backup copy elsewhere
	A self-hosted object store such as MinIO, which speaks the same format
Everything talks to	<code>FileStore</code>
Switching costs	Copy the files across and change the address. Nothing above this layer notices.

5.1 · Keep files outside the database, and never trust their names

Photographs and scans are large, and a database is an expensive place to keep large things. They go in a file store, with the database holding only a reference. A file also arrives from outside, so its name and its claimed type are somebody else's input: both are checked, and the file is stored under a name the system chose.

Done when: A file can be uploaded, fetched and deleted through one interface, and swapping the file store underneath changes one setting.

5.2 · Make every file access ask permission, every time

The most common serious leak in business software is a file link that works for anybody who has it. A photograph of a signed document is as sensitive as the record it belongs to, and it must inherit exactly the same permission, checked on every single fetch.

Done when: A link to another business's file, used by someone from a different business, is refused — proven by a test that tries it.

5.3 · Use the cache only for things that can be safely lost

The cache exists to make common screens fast. The moment anything is kept **only** in the cache, restarting it loses data — and caches get restarted routinely. Everything in there is a copy; losing the whole thing costs a slow minute and nothing else.

Done when: The cache can be wiped completely while the system is running, and nothing is lost but speed.

5.4 · Decide exactly what the assistant may remember, and for how long

model — *The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question. Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.*

An assistant that answers questions about a business needs to see that business's data — and must never see another's, must never keep it after the question is answered, and must never learn from it in a way that could surface it elsewhere. This is a decision to make deliberately at design time, because discovering it later means discovering it the wrong way.

May remember	May never
The current conversation, until it ends	Cross a business boundary, ever
What the user is looking at right now	Retain business data after answering
Settings and vocabulary for this business	Be used to train anything
A saved answer the user chose to keep	Hold a password, a key or a card number

Done when: The retention rules are written down, enforced in code, and a test proves one business's question cannot reach another's data.

Part 6 · Sign-in and permissions

Two separate questions kept deliberately apart: who are you, and what may you do. The first can be handed to somebody else. The second never can.

authentication — *Proving you are who you say you are, usually by signing in. Gate pe pehchaan dikhana. "Main kaun hoon" wala sawaal.* **role** — *What a person is allowed to see and do — a manager sees more than a counter staff member. Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.* **permission** — *One specific thing a role is allowed to do, like approving a discount or viewing salaries. Guchhe ki ek chaabi. Ek chaabi ek darwaza.*

Sign-in and permissions — Proves who somebody is, then decides what they are allowed to see and change.

Who you are and what you may do are kept apart deliberately. Sign-in can be handed to an outside service — or to a customer's own company login — while permissions stay ours, because they depend on the company and role structure no outside service knows about.

This layer	Sign-in and permissions
Built on	Sessions issued by the platform, with permissions checked in the backend and again in the database
Can be replaced with	An identity provider for sign-in only, with permissions still decided here A customer's own company sign-in, for enterprises that require it Self-hosted Keycloak or Authentik, when nothing may leave the building
Everything talks to	IdentityService
Switching costs	Low for sign-in, by design. Permissions never move, so the expensive half is never in play.

6.1 · Separate proving who somebody is from deciding what they may do

Large customers will insist on using their own company sign-in, and that is reasonable — it is how they remove access when somebody leaves. But no outside sign-in system knows that this person may approve purchases up

to a limit in one of your companies and only view stock in another. That decision stays here, always.

Done when: Sign-in can be switched to an outside provider without any change to how permissions work.

6.2 · Make permissions specific to the company, not just the person

Somebody who works across two companies in a group is not the same person in both. Giving one blanket level of access across a group is how a figure from one company ends up in a report for another, and it cannot be untangled afterwards.

Done when: A user working in two companies sees exactly what their role allows in each, and this is proven by a test that tries to cross.

6.3 · Check permission in the backend and again in the database

Hiding a button is not security — it is tidiness. The check that matters happens where the data is. Two layers, because one layer is one mistake away from an incident, and the layers fail independently.

Done when: A request that bypasses the screens entirely is still refused, proven by calling the API directly with a role that should not be allowed.

Part 7 · Talking to the outside world

Messages, storefronts, marketplaces, couriers, payments. Every one is somebody else's system, every one will change without warning, and every one will be down at some point. The design assumes all three.

Messages to customers and staff — Sends WhatsApp messages, text messages and email — reminders, confirmations, statements.

Each tenant connects its own accounts. The platform is built with a place for them to plug in and never holds one central account of its own — a business's conversations with its own customers belong to that business. The platform's job is the plug, not the account.

This layer	Messages to customers and staff
Built on	A message service with one adapter per provider, per tenant
Can be replaced with	Any WhatsApp provider — the adapter changes, the code that decides what to send does not
	Text message and email as fallbacks when a message cannot be delivered
	A shared inbox or an export, for a tenant with no messaging account at all
Everything talks to	<code>MessageService</code>
Switching costs	One adapter per provider. Switching is a settings change made by the tenant, not a release made by us.

7.1 · Build the plug, not the account — every connection belongs to the customer

This is worth being exact about. The platform needs no messaging account, no marketplace seller account and no payment account of its own. A business's conversations with its own customers, and its own selling accounts,

belong to that business. What the platform provides is the place to plug them in, and the code that knows how to talk to each kind.

Building it the other way — one central account that everyone shares — makes the platform the account holder for other people's customers, and makes every customer dependent on a relationship they have no control over.

Done when: A customer connects their own accounts in the app, and the platform holds no account of its own for any of these.

7.2 • Give every capability an ordered fallback, ending somewhere that needs nothing

fallback — *The next option the system automatically moves to when the first one fails or is unavailable. Bijli gayi toh inverter. Inverter gaya toh mombatti. Andhera kabhi nahin hota.* **circuit breaker** — *A switch that takes a repeatedly failing service out of use for a while, instead of retrying it endlessly and slowing everything down. Ghar ka MCB. Baar-baar fault aa raha hai toh woh line hi kaat deta hai, poora ghar band nahin hota.*

A courier service stops answering at nine at night. A messaging provider hits a limit mid-broadcast. A model provider runs out of quota half way through. In each case the work must continue down the list rather than stop — and the last item must be something that needs no outside service at all, even if that means a manual step. That last item is what turns an outage into an inconvenience.

Done when: Every capability has a written fallback order whose final entry needs nothing bought or connected, and a test proves the work completes when the first choice is unavailable.

7.3 • Stop hammering a service that keeps failing

When an outside service is broken, retrying it constantly makes everything slow while achieving nothing. After a few failures it is taken out of the list, the work moves to the next option, and it is tried again once after a pause.

Done when: A provider failing repeatedly is taken out of use automatically, and returns by itself once it recovers.

7.4 • Never let an outside system be the source of a figure the business reports

Numbers come from your own records. An outside service can tell you a payout happened; what that payout **means** to your books is decided here, from your own data, against what you expected. Otherwise a mistake in somebody else's system silently becomes a mistake in your accounts.

Done when: Every figure in every report can be traced to a record in this system, never to an outside response that was taken on trust.

Part 8 • Work that happens on its own, and finding things

Nobody should watch a progress bar while a thousand messages send or a month closes.

queue — *A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report. Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer*

khada intezaar nahin karta. **job** — One piece of work taken off the queue and done in the background. Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.

Background work — Does the things nobody should have to wait for — sending a thousand messages, building a month-end report, pulling orders overnight.

Every job is written so that running it twice does the same thing as running it once. That single discipline is what makes it safe to retry after a failure, and it is worth more than any particular queue product.

This layer	Background work
Built on	A queue backed by the database, with named workers
Can be replaced with	A Redis-backed queue when volume outgrows the database A hosted queue service, behind the same interface
	An external workflow tool such as n8n for steps a non-programmer should be able to edit
Everything talks to	JobQueue
Switching costs	Low. Jobs are plain functions with a name; the queue only decides when they run.

8.1 · Make every background job safe to run twice

Machines restart, connections drop, and a job that was half done gets picked up again. If running it twice sends the message twice or posts the payment twice, every failure becomes a cleanup. Written so that running it again reaches the same result, a failure becomes a retry.

Done when: Every job is run twice deliberately in a test, and the result is identical to running it once.

8.2 · Let a job that fails be seen, understood and retried

A job that fails silently is worse than one that fails loudly — the work simply never happened and nobody finds out until a customer asks. Failures are visible, keep the reason, and can be retried without a developer.

Done when: A failed job appears in a screen with its reason, and an admin can retry it.

8.3 · Start with the database for search, and keep records the source of truth

search index — A prepared list that makes finding things fast, the way the index at the back of a book beats reading every page. Kitaab ke peeche wali index. Poori kitaab padhne ki zaroorat nahin, seedha page number mil jaata hai.

A separate search engine is another thing to run, back up and keep in step. The database can search well enough for a long time. When a separate engine is eventually needed, it is a faster copy — never the place the records live — so it can be rebuilt from scratch at any time.

Done when: Search can be turned off entirely and every record remains reachable, if less conveniently.

Part 9 · The artificial intelligence layer

Useful for writing descriptions, tagging photographs, summarising and answering questions. Dangerous when it becomes something the business cannot operate without, or a bill nobody capped.

spend ceiling — *A maximum amount the system is allowed to spend on paid services, after which it refuses to spend more instead of warning you. Jeb mein utne hi paise leke nikle jitna kharch karna hai. Khatam matlab khatam — udhaar nahin.*

Artificial intelligence — Writes descriptions, tags photographs, summarises, and answers questions about your own data.

Ordered fallback, a breaker on anything failing repeatedly, and a spend ceiling that REFUSES rather than warns. Because every capability also has an option that costs nothing, a spent budget can stop the spending without ever stopping the business.

This layer	Artificial intelligence
Built on	A router in front of several providers, ending on one that needs nothing bought
Can be replaced with	Any hosted model provider — an entry in the router, not a change to the system A model running on your own machine, for work that is routine or private Templates and rules with no model at all, which must always remain the last resort
Everything talks to	<code>ModelRouter</code>
Switching costs	A list entry. The router exists precisely so changing provider is never a project.

9.1 · Put a router in front of every model, never call one directly

Providers change price, change quality, change terms and disappear. A router means the system asks for a capability — "write a description", "tag this photograph" — and the router decides who does it, in what order, and what happens when one fails. Adding or removing a provider is a list entry.

Done when: Adding a new provider requires no change to any business rule, and removing one changes nothing but the list.

9.2 · Cap the spending, and make the cap refuse rather than warn

A warning arrives after the money is gone. The ceiling is checked before each paid call, and over it the paid provider is simply refused — the work then completes on an option that costs nothing. Because every capability is guaranteed a free path, a spent budget stops the spending without ever stopping the business.

Done when: With the ceiling set to zero, every capability still completes its work, proven by a test that sets it to zero and runs the full set.

9.3 · Never let a model decide anything that moves money or stock

A model is good at language and unreliable about facts. It may draft, suggest, classify and summarise. It may not approve a payment, adjust a stock figure, post to the ledger or change a price by itself. The line is not about how

good the model is — it is that a wrong number produced by a person can be traced to a decision, and a wrong number produced by a model cannot.

Done when: An assistant asked to move money declines and produces a request for a person to approve. This is tested by asking it to.

9.4 · Make every answer traceable to the records it came from

An assistant that answers "your best-selling item last month" must be answerable when somebody disagrees. Every answer carries what it looked at, so a wrong answer is a question about the data rather than a mystery.

Done when: Every assistant answer can be expanded to show the records behind it, and those records can be opened.

Part 10 · Running it

Getting it built is half. Being able to change it every week for years without fear is the other half, and it is the half that decides whether the product survives.

environment — A separate running copy of the system — one for trying things, one that customers actually use. *Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.* **deployment** — Putting a new version of the software in place so people start using it. *Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.* **continuous integration** — A robot that checks every change automatically, before anyone can put it live. *Quality-check wala banda gate pe khada. Har maal nikalne se pehle usse guzarta hai.* **rollback** — Putting the previous working version back, quickly, when a new one turns out to be wrong. *Naya taala kharab nikla toh purana taala wapas laga do — do minute ka kaam.* **observability** — Being able to see what the system is doing and what went wrong, without guessing. *Dukaan mein CCTV aur register. Kuch gadbad ho toh dekh sakte ho ki hua kya, andaaza nahin lagana padta.*

Source control and automatic checks — Keeps the history of every change and runs every test before anything goes live.

Git itself is the thing that matters, and git is not owned by anybody. The host is a convenience.

This layer	Source control and automatic checks
Built on	Git, with automatic checks on every change
Can be replaced with	A different hosting service — a git repository moves with one command Self-hosted Gitea or Forgejo
	A separate build service reading the same repository
Everything talks to	the test commands themselves
Switching costs	Very low. The checks are ordinary commands, so any system that can run a command can run them.

10.1 · Keep separate copies for trying things and for real customers

Nobody should learn that a change breaks payroll by watching it break a real payroll. A practice copy carries realistic but not real data, so mistakes cost an afternoon rather than a customer.

Done when: A change can be tried end to end somewhere that no customer can see.

10.2 · Make a robot check every change before a person can release it

Human review catches design mistakes. It does not reliably catch that a change broke something three modules away. Automatic checks do, on every single change, without getting tired or being in a hurry on a Friday evening.

Done when: No change reaches customers without every check passing, and this cannot be skipped by anyone.

10.3 · Be able to put the previous version back in minutes

Something will get through. What separates a scare from an incident is how fast the last working version can return. If going back is difficult, the pressure will be to fix forward under stress, which is how a small problem becomes a large one.

Done when: Going back to the previous version is one command, practised at least once before anyone depends on it.

10.4 · Package it so it can run anywhere

The moment something host-specific gets in, the hosting choice is locked and moving means a project. Packaged as an ordinary container with nothing host-specific inside, moving is a decision rather than an undertaking.

Done when: The same package runs on a laptop, on a rented server, and on a managed platform, with only settings differing.

10.5 · Be able to see what is happening without guessing

When something is slow or wrong at four in the afternoon with customers waiting, the question is where — and guessing is expensive. Structured records of what happened, how long it took and what failed turn that into a lookup.

Done when: A failure can be traced from the user's click to the exact operation that failed, without adding new logging first.

10.6 · Back it up, and prove the backup by restoring it

backup — *A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work. Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.*

An untested backup is a belief, not a protection. The only proof is a restore into a scratch copy, done deliberately, before it is ever needed.

Done when: A backup has been restored into a scratch environment and checked, and that is repeated on a schedule.

Part 11 · Security, stated plainly

Short, because these are absolutes rather than preferences.

11.1 · Never ask anyone for a marketplace, bank or account password

Every connection is made with a key the customer creates and can withdraw. A password hands over an account that cannot be taken back and cannot be limited. This is a promise the product makes, so nothing in the software, the documents or a support conversation may ever break it.

Done when: No screen, no form and no support process anywhere asks for one, and the product says so openly.

11.2 · Keep keys out of the code, always

encryption — *Scrambling information so that even somebody who steals the file cannot read it. Apni hi code-bhasha mein likhna. Chori bhi ho jaaye toh padha nahin jaata.*

A key written into the code is in every copy of that code, forever, including copies you no longer control. Kept outside, a key can be replaced in a minute.

Done when: A search of the whole history finds no key, and that search runs automatically on every change.

11.3 · Treat identity documents and bank details as read-once, never stored

Identity and bank numbers may be needed for a calculation or a payment file. They are used and not written into anything that is kept, because a stored copy is a liability that grows quietly until the day it is stolen.

Done when: No committed file and no exported document contains an identity number, a bank account or a card number.

11.4 · Let a person's data be corrected and removed on request

Keeping a record for the law and removing a person's data on request are two different obligations that resolve differently, and a system with only one of them will breach the other.

Done when: Both are separate, recorded settings, and a request of either kind can be carried out and evidenced.

Part 12 · What order to build it in

The order is not a preference. Each stage exists because the next one cannot be trusted without it, and each finishes when a test proves it rather than when the code is written.

12.1 · Finish a stage only when its test passes, never when its code is written

"Done" is the most abused word in software. A stage that is finished because somebody believes it is finished will be discovered later, from the far side of three stages built on top of it. A stage finished because a test proves it can be built on.

Done when: Every stage has one written test that decides it, agreed before the stage starts.

12.2 · Build the modules in the order they are numbered

They are numbered in the order their dependencies allow. A product exists before it is stock; a customer exists before a sale; demand exists before a purchase; stock exists before it moves; the books exist before they close. Building out of order means inventing the thing you need and correcting it later.

Done when: No module is started before the ones it reads from can supply real records.

The modules, in the order they are built

22 modules. Module 01 is the spine — not something you open, the layer everything else stands on — which is why 22 modules is also 21 you use plus one underneath them.

Each row says what has to exist before it can start, and how many rules it must satisfy before it is finished.

#	Module	Needs first	Rules to satisfy
01	Platform (<i>spine</i>)	Every module	25
02	Design & Sampling	CRM	7
03	Inventory & Catalog	Design & Sampling, Every module	14
04	CRM	Every module	9
05	Sales	Inventory & Catalog, CRM, Warehouse, Logistics	18
06	Planning & Requirements (MRP)	Sales, E-commerce / OMS, Inventory & Catalog	8
07	Purchase	Inventory & Catalog, Planning & Requirements (MRP), Manufacturing	12
08	Manufacturing	Purchase, Planning & Requirements (MRP), Design & Sampling	20
09	Quality & Compliance	Purchase, Manufacturing	7
10	Warehouse	Sales, E-commerce / OMS, Inventory & Catalog	8
11	Logistics	Sales, E-commerce / OMS, Warehouse	11
12	Accounting & GST	Every module	24
13	Treasury & Financial Planning	Accounting & GST, Sales, Purchase	8
14	Settlement	E-commerce / OMS, Accounting & GST	13
15	E-commerce / OMS	Inventory & Catalog, CRM, Sales, Accounting & GST, Logistics, Settlement	19
16	HR & Payroll	Manufacturing	22
17	Marketing	Inventory & Catalog, CRM	10
18	AI Content Engine	Inventory & Catalog	11
19	SEO, AEO & AIO	Inventory & Catalog, AI Content Engine	6
20	Projects & Collaboration	CRM, Sales, HR & Payroll, Inventory & Catalog	9
21	Dashboard & BI	Every module	9
22	AI Assistant, Agents & Automation	Every module	15

A module is finished when every rule for it is satisfied and proven by a test — not when its screens exist. Screens can be demonstrated; rules are what the books rely on.

Part 13 · What a customer can change without you

The measure of whether this design succeeded. Everything in this table is changed by the customer, in the app, taking effect immediately — and none of it requires a developer, a release or a phone call.

18 things, across 4 areas. For each one, the column that matters most is the last: what happens to records already made.

People

What changes	Who	Immediately	Records already made
Somebody joins — a worker, a supervisor, an office staff member, a contractor	Admin, or an HR role	They exist from their start date and can be assigned work the same minute.	Nothing before their start date mentions them, because they were not there.
Somebody leaves, with or without notice	Admin, or an HR role	Marked as left from a date. Their sign-in stops, and no new work is assigned to them. Their record is kept, not deleted.	Every hour they worked, every piece they made and every rupee they were paid stays exactly as it was. Deleting the person would blank all of it and change months already closed — so the record remains and simply has an end date.
A replacement starts immediately, in the same position	Admin, or an HR role	Added with their own start date and given the position. Work in progress is reassigned to them from that date. No waiting, no release, no developer.	Work completed under the previous person stays credited to the previous person. Two people held the same position at different times, and every record knows which one applied when.
A pay rate, a piece rate or a salary changes	Admin, or an HR role	Applies from the date you set — which may be today, a future date, or a past one you are correcting.	Every completed period recalculates to the rate that applied then, not the new one. This is the single most important line in this register: a rate that silently applied backwards would change payments already made to real people.
What somebody is allowed to see and do	Admin	Takes effect on their next action. Screens they may no longer open stop opening.	Everything they did while they held the old permissions stays recorded, with the permissions they had at the time.

Structure

What changes	Who	Immediately	Records already made
A new company is opened, or an existing one is closed	Admin	It exists immediately with its own name, trading name, code and document numbering. The group view includes it from that date.	Group figures for earlier periods are unchanged, because the company did not exist in them.
A new way of selling is added — a marketplace, a shop, a counter, an export desk	Admin	Orders can arrive through it the same day. It appears in every report that breaks figures down by channel.	Earlier reports keep their own columns. A channel that did not exist then does not appear then.
A godown, a shop, a unit or a stock point is added, renamed or closed	Admin	Stock can move to and from it immediately.	Stock movements already recorded keep pointing at it, under the name it had at the time.
Which parts of the system this business uses at all	Admin	Turned on, a module appears in the menu with its screens ready. Turned off, it disappears from the menu. A steel plant, a clothing brand and a single creator each end up with a different system built from identical code.	Turning a module off hides its screens and keeps its records. Nothing is destroyed by tidying a menu.

Your words

What changes	Who	Immediately	Records already made
What the system calls things	Admin	Every screen, every report and every document changes wording at once. One business says order, another says job, another says matter, consignment, batch or booking — the record underneath is identical.	Documents already issued keep the wording they were issued with, because that is what the customer received.
Extra information you want to record that nobody else needs	Admin	Added to the screen immediately, with the type you choose — text, number, date, a list to pick from, a yes or no. Reportable from the moment it exists.	Older records simply have no value for it, which is the truth. They are never back-filled with a guess.
The steps your work moves through	Admin	Add a stage, rename one, reorder them or remove one. New work follows the new list from that moment.	Work already part-way through keeps the stage it is in, even if that stage has since been removed — a job does not teleport because somebody edited a list.
The layout and numbering of invoices, statements, labels and reports	Admin	The next document uses the new layout or the new numbering.	Documents already issued are never re-rendered. What the customer holds and what you hold stay identical.

Rules

What changes	Who	Immediately	Records already made
Turning a discretionary rule on or off	Admin	Applies to the next transaction. One business requires an approval below a price floor; another does not — same software, different setting.	Transactions already posted are not re-judged against a rule that did not apply to them.
Who has to approve what, and above which amount	Admin	The next request follows the new path.	Requests already approved keep the path they went through, and the names of who approved them.
Tax rates and the categories they attach to	Admin, or an accounts role	Applies from its effective date, which for tax is set by law rather than by you.	Every invoice keeps the rate that applied on its own date. A return filed for an earlier period recalculates to that period's rate — this is not a convenience, it is the only correct behaviour.
Which outside service is used for messages, payments, delivery or artificial intelligence	Admin	The next message, payment or shipment goes through the new one.	Everything already sent keeps the record of which service carried it, which is what you need when you query one.
The most the system may spend on paid outside services	Admin	Enforced immediately. Over the ceiling, the paid option is refused and the work completes on one that costs nothing.	Spending already recorded is unchanged.

What can never be switched off

Short on purpose. Every line is something a bank, an auditor, a customer or an employee relies on, and a setting that could remove it would remove their protection too.

Never changeable	Why
The audit trail	Who changed what, and when. A system where this can be switched off cannot be used to answer a dispute, so it cannot be switched off.
Every record naming the company it belongs to	Without it, figures from two companies merge and no report can be trusted again.
One business being unable to read another's records	This is not a preference. It is the promise that makes a shared platform usable at all.
Money kept as exact whole units	The alternative loses fractions of a rupee in ways nobody can trace afterwards.
Deleting nothing — records are ended, never erased	An erased record changes a period that was already closed, filed and possibly audited.
Never asking for a marketplace, bank or account password	The system connects through proper keys that you can withdraw. A password would hand over an account you cannot take back.

Every layer, and what replaces it

The whole of Rule 1 on one page.

Layer	Built on	Alternatives	Talks to
The database	PostgreSQL	3	DatabaseService
File storage	Any S3-compatible object store	3	FileStore
Cache and short-term memory	Redis, or a Redis-compatible store	3	CacheService
The backend runtime	Node.js with TypeScript	3	the HTTP API contract
The API	REST over HTTPS, with a written schema	3	the published API schema
The frontend	React with TypeScript, screens generated from configuration	3	the screen definition format
Background work	A queue backed by the database, with named workers	3	JobQueue
Search	PostgreSQL full-text search	3	SearchService
Sign-in and permissions	Sessions issued by the platform, with permissions checked in the backend and again in the database	3	IdentityService
Keys and passwords the system uses	Environment variables on the server, readable only by the service account	3	ConfigService
Messages to customers and staff	A message service with one adapter per provider, per tenant	3	MessageService
Storefronts and marketplaces	A channel adapter per storefront or marketplace	3	ChannelAdapter
Taking payments	A payment adapter per provider, with the card field hosted by the provider	3	PaymentService
Delivery and couriers	A courier adapter per carrier	3	CourierService
Artificial intelligence	A router in front of several providers, ending on one that needs nothing bought	3	ModelRouter
Where it runs	Containers on a virtual server	3	the container image
Source control and automatic checks	Git, with automatic checks on every change	3	the test commands themselves
Watching it	Structured logs and error reporting, in an open format	3	Logger and the metric format
Making documents	HTML templates printed to PDF by a headless browser	3	DocumentRenderer

Generated by `brand/delivery/website/mkguide.js` from `brand/site/guide.js`, `stack.js`, `plainwords.js`, `dynamic.js` and the canonical lists. Every count is read from its source at generation time — no module count, layer count or rule count is typed by hand. Nothing here is maintained by editing this file: edit the source and regenerate.

Medhava · One business operating system · Any industry